UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

Bachelor of Science (Honours) in Computer Science



Deep Learning Breast Cancer Detection

Author: Wong Zheng Jie

10246254

Goldsmiths, University of London

Singapore Institute of Management

Module: CM3015 Machine Learning and Neural Networks

Supervisor: Mr Hai Kian Loh

Singapore 20 September 2025

Abstract

Breast cancer continues to be a major cause of death in women globally, highlighting the importance of early diagnosis. This report presents the findings and evaluations of deep learning models on mammography images with the aim to improve breast cancer detection. It highlights the project's aim and objectives, motivations, methodological approach, an extent of pertinent literature review, implementation, and evaluation.

The best model, a hybrid dual branch architecture containing narrower CNN model and EfficientNetB0 transfer learning, obtained an overall accuracy of 65%, with especially strong performance in identifying Normal cases (F1 Score = 0.82) and reasonable balance across Benign and Cancer classes.

Although the models are not yet suitable for clinical use, the findings shed light on promising research directions such as leveraging ROI masks in model architectures as well as integrating advanced segmentation for improving classification performance.

Acknowledgements

Firstly, I would like to acknowledge and express my sincerest gratitude to my supervisor, Mr. Hai Kian Loh, for his invaluable guidance and support in achieving every milestone in this project. His experience and wisdom shared during our online meetings have helped shaped the direction of this work.

Secondly, I would like to thank my friend and course mate Boomiraj Lokesh for his constant inspiration, friendship, and knowledge he has generously shared during this project and over the course of our studies.

Finally, I want to thank my family members for their unwavering support and patience throughout this journey. Their emotional encouragement has kept me motivated from start to finish.

This project marks the culmination of my three years of study. As I graduate, I look forward to applying the knowledge I have gained to contribute meaningfully to society, advance the field of computer science, and create a positive impact on the world.

Table of Contents

Abstract

Acknowledgements

1. Introduction 1

1.1 Project Background 1

1.2 Data Source and Relevancy 1

1.3 About Breast Cancer 2

1.5 Motivations 3

1.4 Aim and Objectives 4

1.6 Stakeholders 4

1.7 Methodological Approach 5

1.8 Planning 7

2. Literature Review 8

2.1 Breast Cancer DL Overview 8

2.2 Similar Projects 9

2.3 Image Classification Approaches 9

2.4 Model Validation 10

2.5 Transfer Learning 10

2.6 Hybrid Architectures and Approaches 11

3. Scope 13

4. Project Design 14

4.1 Background on Neural Networks 14

4.2 CNN Architecture Overview 15

4.3 About Stratified K-fold Cross Validation 16

4.4 Stratified K-fold Cross Validation: Dataset Overview 17

4.5 Application of Transfer Learning 18

4.6 Design Considerations on Hybrid Models and Multi-Branching Architecture 19

4.7 About Multi-Branching Architecture 20

5. Implementation 21

5.1 Defining the problem and assembling a dataset 21

5.2 Choosing a measure of success 21

5.3 Deciding on an evaluation protocol 23

5.4 Preparing your data 24

5.5 Developing a model that does better than a baseline 25

5.6 Scaling up: developing a model that overfits 27

5.7 Regularizing model and tuning hyperparameters 28
5.8 Wider, Deeper, Narrower Models 30
5.9 Transfer Learning Model 34
5.10 Exploring Models and Architectures 35
6. Evaluation 38
6.1 Naïve Baseline Model Evaluation 38
6.2 Baseline Model Evaluation 40
6.3 Overfitting Model Evaluation 42
6.4 Regularized Model Evaluation 44
6.5 Wider, Deeper, Narrower Models Evaluation 45
6.6 Transfer Learning Model Evaluation 49
6.7 Narrower and Transfer Learning Multi-Branch Model Evaluation 50
6.8 Modified U-Net Model Evaluation 51
6.9 Model Comparison 52
7. Conclusion 56
7.1 Implications 56
7.2 Future Directions 57
8. Artefacts 59
9. Authors' Declarations 60
9.1 Conflict of Interest 60
9.2 Consent 60
9.3 Authors' Notes 60
10. References 61
11. Bibliography 70
Appendices
12. Appendix A – Supplementary Planning Content 71
12.1 Kanban System 71
12.2 Gantt Chart 72
13. Appendix B – The Complete Mini-DDSM archive Folder Structure 75
14. Appendix C – Supplementary Literature Review 77
14.1 Breast Cancer Diagnosis 77
14.2 Social Factors 78
14.3 Economic Factors 78
14.4 Technological Factors 78
15. Appendix D – CNN Layers 80

15.1 Convolutional Layers 80
15.2 Pooling Layers 81
15.3 Fully Connected / Dense Layers 83
16. Appendix E – User Interface 85
16.1 Ethics on Application Interaction 85
16.2 UI Design 86
16.3 UI Implementation 89
16.4 UI Output Results 91
16.5 UI Further Advancement Strategy 91
17. Appendix F – Ethics in Data Collection 93
18. Appendix G – Codes 94
19. Appendix H – Supplementary Materials 144
19.1 Supplementary Figures 144
19.2 Supplementary Tables 149
20. Appendix References 150

List of Figures

Figure 1.3.1 External Anterior and Cross-Sectional View of the Human Breast [9] 2
Figure 1.8.1 This GitHub Org Kanban Scheduling system [20] 7
Figure 2.1.1 Receiver operating characteristics for detection and risk by algorithm [22] 8
Figure 2.6.1 Processing of Hyperspectral Imagery (HSI) by Tanver Ahmed et al. (2025) [47] 11
Figure 4.1.1 DNN with the input x a probability of being associated to a certain concept ωc [49] 14
Figure 4.2.1 Simple 5 Layered CNN Architecture [50] 15
Figure 4.3.1 This Stratified K-Fold Cross Validation Pipeline 16
Figure 4.4.1 Stratified K-fold Cross Validation Process Illustration 17
Figure 4.5.1 Transfer Learning CNN Pipeline 18
Figure 4.7.1 Dual Branch Architecture Example Overview 20
Figure 5.2.1 Confusion Matrix Illustration 23
Figure 5.4.1 CC and MLO Breast Cancer Mammography Image Preview 24
Figure 5.4.2 Baseline Model Training Set Class Distribution 25
Figure 5.7.1 K-fold Cross Validation Results 28
Figure 5.10.1 Narrower and Transfer Learning Multi-Branch Model Architecture 36
Figure 6.1.1 Naive Baseline Model Results 38
Figure 6.2.1 Baseline Model Results 40
Figure 6.3.1 Overfitting Model Results 42
Figure 6.4.1 Regularized Model Results 44
Figure 6.5.1 Wider, Deeper, Narrower Models Accuracy Plot Comparison 45
Figure 6.5.2 Wider, Deeper, Narrower Models Confusion Matrix Comparison 46
Figure 6.5.3 Wider, Deeper, Narrower Models Scores Summary Comparison 46
Figure 6.5.4 Narrower Model Two Results 48
Figure 6.6.1 Transfer Learning Model Results 49
Figure 6.7.1 Narrower and Transfer Learning Multi-Branch Model Results 50
Figure 6.8.1 Modified U-Net Model Results 51
Figure 6.9.1 Model Comparison on Normal Classification Scores Chart 53
Figure 6.9.2 Model Comparison on Benign Classification Scores Chart 53
Figure 6.9.3 Model Comparison on Cancer Classification Scores Chart 54
Figure 6.9.4 Model Comparison on Macro Averages and Accuracy Chart 54
Figure 6.9.5 Model Comparison on Weighted Averages and Accuracy Chart 55
Figure 7.1.1 UI App localhost Implementation 56
Figure 7.2.1 4-Branch Model Architecture for Detecting Cancer Presence and ouput Predicted Cancer ROIs 58

List of Tables

Table 1.2.1 Commonly Used Publicly Available Datasets in Mammography for Breast Screening [6] 1
Table 5.5.1 Naive Baseline Model Summary Table 25
Table 5.5.2 Baseline Model Summary Table 26
Table 5.6.1 Overfitting Model Summary Table 27
Table 5.7.1 Regularized Model Summary Table 29
Table 5.8.1 Wider Model Summary Table 30
Table 5.8.2 Deeper Model Summary Table 31
Table 5.8.3 Narrower Model Summary Table 32
Table 5.8.4 Narrower Model Two Summary Table 33
Table 5.9.1 Transfer learning Model Summary Table 34
Table 5.10.1 Narrower and Transfer Learning Multi-Branch Model Summary Table 35
Table 6.9.1 Model Evaluation Results Comparison 52

List of Abbreviations

AI: Artificial Intelligence

ANN: Artificial Neural Network

AUC-ROC: Area Under the ROC Curve

CAT: Computerized Axial Tomography

CC: Craniocaudal View

CNN: Convolutional Neural Network

CSS: Cascading Style Sheets

CT: Computed Tomography

DCE-MRI: Dynamic Contrast-Enhanced Magnetic Resonance Imaging

DBT: Digital Breast Tomosynthesis

DDSM: Digital Database for Screening Mammography

DICOM: Digital Imaging and Communications in Medicine

DL: Deep Learning

DNN: Deep Neural Network

FPN: Feature Pyramid Network

GPU: Graphics Processing Unit

HDF5: Hierarchical Data Format 5

HTML: Hypertext Markup Language

IPYNB: IPython Notebook

IRT: Infrared Thermography

JPEG: Joint Photographic Experts Group

LFS: Large File Storage

LJPEG: Lossless Joint Photographic Experts Group

MLO: Mediolateral Oblique View

MLP: Multilayer perceptron

MRI: Magnetic Resonance Imaging

MSER: Maximally Stable Extremal Regions

NN: Neural Networks

ReLU: Rectified Linear Unit

ROI: Regions of Interests

TNM: Tumour, Nodes, and Metastasis

TPU: Tensor Processing Unit

VGG: Visual Geometry Group network

Nomenclature

Symbol	Description
x	Input feature map or pixel value
y	Output feature map after convolution or pooling
w	Weight of a convolutional filter
b	Bias term applied after convolution
$*$	Convolution operator
$f(x)$	Activation function (e.g., ReLU) applied to x
$max\ (0, x)$	ReLU function
max	Max pooling operator
m, n	Indices within pooling window
i, j	Spatial position indices in feature maps
i', j'	Output indices after pooling
H, W	Height and width of the pooling window
d	Channel / Depth index
l	Layer index
D	Depth (number of channels) of the input and output feature maps
TP	True Positives – Correctly predicted AI tests
TN	True Negatives – Correctly predicted human texts
FP	False Positives – Human texts incorrectly predicted as AI texts
FN	False Negatives – AI texts incorrectly predicted as human texts
Sp	Sum of all positives ranked
Np	Number of positives
Nn	Number of Negatives

List of Appendix Figures

Appendix Figure 12.1.1 A Kanban Board [85] 71
Appendix Figure 12.1.2 Kanban Priority (Left) and Size (Right) Options [19][20] 72
Appendix Figure 12.2.1 Gantt Chart 72
Appendix Figure 13.1 The Complete Mini-DDSM archive Folder Structure 75
Appendix Figure 13.2 MINI-DDSM-Complete-PNG-16 Internal Cancer Folder Structure 76
Appendix Figure 14.1.1 Breast Screening Routine Steps [86] 77
Appendix Figure 14.4.1 This Number of Publications Vs Modalities [97] 79
Appendix Figure 15.1.1 Convolution on a Padded Single-Channel Image [101] 80
Appendix Figure 15.1.2 Single Convolution Step Element-wise Computation [101] 80
Appendix Figure 15.1.3 This Convolutional Filter Moving Across a Single Channel Image with a Stride of 1 [101] 81
Appendix Figure 15.2.1 2x2 Max Pooling Operation [101] 82
Appendix Figure 15.2.2 Average Pooling Operation [102] 82
Appendix Figure 15.3.1 Softmax Activation [101] 83
Appendix Figure 15.3.2 Regular Connections and Dropout Applied Comparison [101] 84
Appendix Figure 16.2.1 Simple UI Design Version 1 86
Appendix Figure 16.2.2 Simple UI Design Version 2 87
Appendix Figure 16.3.1 UI Implementation Folder Structure 89
Appendix Figure 16.3.2 UI Implementation Batch File Start Up 90
Appendix Figure 15.4.1 Agile Methodology Cycle [105] 92
Appendix Figure 19.1.1 Gantt Chart Upscaled 146
Appendix Figure 19.1.2 The Complete Mini-DDSM archive Folder Structure Enlarged 147
Appendix Figure 19.1.3 6 x 6 Grid Comparison of Randomly Selected Mammogram Images 148
Appendix Figure 19.1.4 Preliminary Report Dual Input Hybrid CNN Rough Workflow Discussion Idea 149

List of Appendix Tables

Appendix Table 13.1 MINI-DDSM-Complete-PNG-16 Internal Cancer Folder File Information 76
Appendix Table 16.4.1 UI Output Results on 6 Mammography Images 91
Appendix Table 19.2.1 Preliminary Report Naïve Baseline and Baseline Table Comparison 149

1. Introduction

1.1 Project Background

In early 2025, the UK National Health Service (NHS) initiated a significant research project to investigate whether deep learning–assisted X-ray mammography can improve the accuracy of breast cancer screening. If proven effective, a Deep Learning system could replace one of the two radiologists presently reviewing mammograms, leading to faster diagnostic turnaround and allowing specialists to focus on other critical clinical tasks [1][2].

At the core of various deep learning methods for image classification lies Convolutional Neural Networks (CNNs). CNNs apply a succession of filters to the input layer, facilitating the extraction of hierarchical features, making them highly effective for image classification tasks including detecting abnormalities in medical images.

This project aims to improve deep learning breast cancer detection on x-ray mammography images using deep learning methods like CNNs in hopes of relieving specialists for other essential clinical responsibilities.

1.2 Data Source and Relevancy

The dataset selected is the Mini-DDSM dataset [3][4] based on the Digital Database for Screening Mammography (DDSM) dataset. [5]

Several factors preceded the decision to use Mini-DDSM over the original DDSM dataset and other available datasets:

Year	Country	Dataset	Sample Number	Human Number	Task
1998 (41)	US	Digital Database for Screening Mammography (DDSM)	2,620	N/A	Breast cancer detection
1998 (42)	US	Mammographic Image Analysis Society (Mini-MIAS)	322	N/A	Breast cancer classification
2012 (43)	US	INbreast	410	N/A	Breast cancer detection
2017 (44)	US	Breast Cancer Digital Repository (BCDR)	1,224	N/A	Breast cancer classification
2017 (45)	US	Curated Breast Imaging Subset of DDSM (CBIS-DDSM)	753	N/A	Breast cancer detection
2011 (46)	US	BCDR-F01	362	N/A	Breast cancer classification
2018 (47)	USA	DDSM	2,620	N/A	Classification, segmentation
2016 (48)	Netherlands	Mammographic Image Analysis Society (MIAS)	322	N/A	Classification, segmentation
2012 (49)	Multi-country	BCDR	1,875	N/A	Classification, density

N/A, Not Applicate.

Table 1.2.1 Commonly Used Publicly Available Datasets in Mammography for Breast Screening [6]

- The original DDSM dataset on itself is one of the most accessible and cleanest breast cancer related public datasets available [6]. (See Table 1.2.1)

- While the Digital Database for Screening Mammography, also known as CBIS-DDSM [7] also provides images, these datasets are stripped from their original identification filename and from the age attribute, limiting its use for certain types of analysis.

- The original DDSM image files are compressed with lossless JPEG (LJPEG) encoding that are generated using broken software (or acknowledged as an outdated tool on the DDSM website), this complicates the direct use of the DDSM dataset. Retrieving the original DDSM dataset for use would lead to a repetition in steps following the Mini-DDSM dataset (e.g. ensuring uncorrupted images, correct annotations, manually examining to remove outliers)

- The Mini-DDSM dataset was carefully assembled from DDSM, involving manual inspection to eliminate anomalies and outliers (such as incorrect entries like mammograms designated for a one-year-old female). Consequently, Mini-DDSM provides a high-quality, well-annotated, and accessible collection of mammographic images, metadata, and annotations, making it especially valuable due to the overall lack of publicly accessible, well annotated healthcare imaging datasets.

1.3 About Breast Cancer

Breast cancer stands as one of the most common types of cancer impacting people globally, occurring more frequently in women, though it is also found in men. As per the World Health Organization (WHO), in 2024, while lung cancer remained as the most commonly occurring cancer (12.4% of the total cases), breast cancer was the second most frequently diagnosed cancer worldwide (11.6% of the total cases) [8]. Timely identification and intervention greatly enhance outlook and survival probabilities, which is why screening methods like mammography are vital in public health programs.

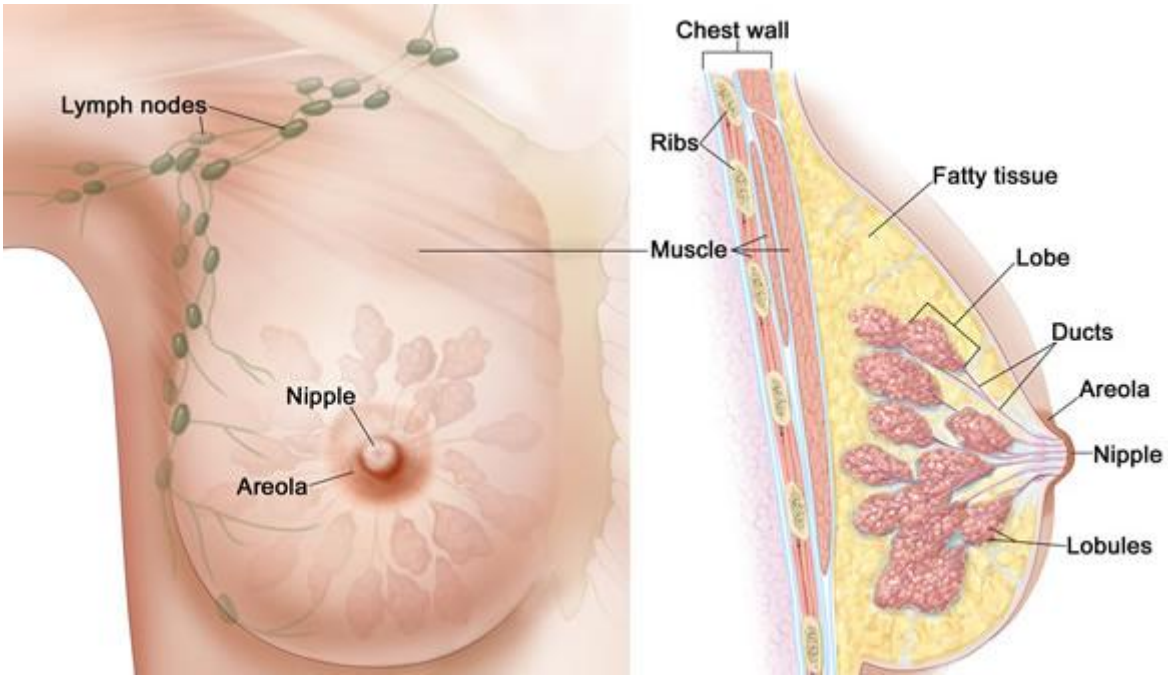


Figure 1.3.1 External Anterior and Cross-Sectional View of the Human Breast [9]

Breast cancer occurs when breast cells grow uncontrollably, leading to the formation of clusters of cancer cells or a tumour. These formations usually begin in the Lobules (milk-producing glands) or the Ducts (thin tubes that carry milk from the lobules to the nipples). (See Figure 1.3.1) [9]

The staging of breast cancer, known as the Tumour, Nodes, and Metastasis (TNM) system, expressed as Roman numerals from 0 through IV, ranges from [10]:

- Stage 0: non-invasive breast cancers, where abnormal cells are present but confined to the part of the breast in which they started.
- Stage I: Invasive cancers are at an early stage where the tumour has extended to adjacent tissues or lymph nodes but is still confined. (tumour less than 2 cm)
- Stage II: Invasive cancers are at an early stage where the tumour has extended to adjacent tissues or lymph nodes but is still confined. (tumour between 2 and 5 cm, and may have spread to nearby lymph nodes)
- Stage III: More advanced local spread, frequently engaging regional lymph nodes.
- Stage IV: Metastatic breast cancer, invasive breast cancer that has spread beyond the breast and lymph nodes to distant organs.

Mammography is a non-invasive imaging method employing low-dose X-rays to investigate breast tissue for indications of cancerous formations [11][12]. Although it continues to be the best method for screening, interpreting it is inherently difficult because of elements such as breast density and the subtlety of early-stage indicators, resulting in both false negatives and false positives. By employing AI and DL methods to mammography images, this study hopes to enhance detection accuracy and support early cancer intervention.

1.5 Motivations

Breast cancer continues to be a significant global health issue, with its incidence increasing by around 1% each year from 2012 to 2021, especially in women [13]. Although female breast cancer has been the primary focus of most research and screening initiatives, male breast cancer (MBC) continues to be notably underrepresented in population studies and clinical trials, despite its acknowledgment as a separate clinical entity. [14]

There is also a need to create tools that reduce human mistakes and speed up diagnostic procedures, particularly for the early detection of breast cancer, as timely intervention can significantly improve patient outcomes.

As mentioned by the UK government as well, every four minutes, a person in the UK succumbs to cancer, and an independent investigation of the NHS in England [15] revealed that survival rates for certain cancers in the UK are lower than in comparable countries. AI DL assistance can relieve 1 of 2 radiologists needed for mammogram screenings. This could enable many

radiologists and various specialists nationwide to attend to more patients, address increasing cancer rates, save additional lives and reduce waiting lists. [2]

By utilizing deep learning methods, this project seeks to improve the accuracy and effectiveness of breast cancer screening, providing the opportunity to assist radiologists and improve diagnostic consistency.

1.4 Aim and Objectives

Although the Mini-DDSM dataset was originally meant for Mammography-based Automatic Age Estimation, it has been repurposed in several works shared on Kaggle with the same or similar objective of breast cancer detection with Convolutional Neural Networks [3].

Several limitations have been identified in existing projects as areas for improvement:

- Many projects utilizing Mini-DDSM do not provide clear documentation of essential steps like data preprocessing and evaluation methods, making it complicated for other researchers to replicate results or expand on their findings.

- Some projects may not apply proper regularization; this can lead to overfitting and poor generalization since the Mini-DDSM dataset is relatively limited.

- Current projects frequently concentrate on model performance metrics, neglecting how these models can be effectively integrated into clinical workflows, restricting real-world implementation insights.

This project seeks to overcome these limitations by conducting comparative analysis between models for breast cancer detection on mammographic images, with the objective of enhancing breast cancer screening accuracy.

1.6 Stakeholders

The primary stakeholders of the project are Radiologists, but it is not limited and includes Doctors, Surgeons, Students, Healthcare Administrators, Medical Researchers and Academics.

While the project is driven towards relieving radiologists, the project could enable non-specialist medical personnels to conduct an initial assessment that may result in true positive outcomes leading to further specialist involvement. Stakeholders include:

- **Doctors / General Healthcare Practitioners**: Utilizing the system as a triage instrument for early identification for faster referrals to specialists.

- **Surgeons**: Using it as an secondary external tool for accurate diagnostic details to devise surgical procedures and treatment plans.

- **Students and Medical Trainees**: Engaging with AI-driven diagnostic systems, offering learning opportunities and access to new technologies.

- **Healthcare Administrators**: who can enhance efficiency, minimize diagnostic delays, and decrease expenses related to later-stage cancer identification.

- **Medical Researchers and Academics**: Utilize the system to analyse model effectiveness, create new techniques, and aid in the progress of AI within healthcare.

Overall, the project's impact lies in efficient breast cancer diagnosis for timely intervention but hopes to also improve future research and aid in the development of medically relevant AI tools.

1.7 Methodological Approach

The François Chollet's Deep Learning Workflow [16][17] is adopted as the guiding framework as it is a structured and systematic deep learning workflow that shows how model performance evolves. Listed below are the key sections mentioned in the workflow:

1. **Defining the problem and assembling a dataset**

   The project defines the task, compiling the Mini-DDSM as the main data source, ensuring labelling uniformity and data integrity.

2. **Choosing a measure of success**

   Multiple performance metrics are addressed here, guaranteeing both false positives and false negatives are carefully evaluated.

3. **Deciding on an evaluation protocol**

   Due to the limited size of the Mini-DDSM dataset, stratified k-fold cross-validation is being proposed, accounting for the dataset's size and potential class imbalance.

4. **Preparing your data**

   Dataset is pre-processed and normalized to ensure that images fit the input needs of pretrained models.

5. **Developing a model that does better than a baseline**

   A simple CNN model is being developed as a baseline, acting as a reference for us to understand and build upon, aiding us in analysing the improvements provided by more complex models.

6. **Scaling up: developing a model that overfits**

   The model is intentionally trained to overfit the data to discover the model's maximum capacity and assist in regularization methods.

7. **Regularizing model and tuning hyperparameters**

   Regularization techniques are applied to mitigate previously identified overfitting, improving performance and generalization.

Besides adopting François Chollet's Deep Learning Workflow [16][17], the project also explores areas beyond the original framework:

- **Wider, Deeper, Narrower Models**

  These models help examine the impact of capacity on learning on breast screening mammography images.

- **Application of Transfer Learning**

  Leveraging pretrained models can help reduce training time and benefit from prior knowledge.

- **Exploring Models and Architectures**

  Implementing alternate models suggested by Lulu Wang (2024) [6] allows for comparison and benchmarking.

  Alternative models and architecture allow us to explore potential improvements in screening capabilities.

  This section also explores implementing image masks and markers to pinpoint cancer regions in mammography images. Potentially aiding in the identification of cancer biomarkers and support specialist investigations and interpretability.

These sections explore alternative models and architectures that may provide better outputs and insights for breast cancer screening.

Together, these methods seek to pinpoint the most efficient strategies for detecting breast cancer via mammography and evaluate their possible impact on enhancing actual screening practices.

1.8 Planning

Selecting suitable technologies is critical for tasks to be carried out effectively within the project deadlines set. The tools used to complete the project include:

- Communications and Guidance: WhatsApp, Google Meet.
- Project Proposal: Microsoft PowerPoint, OBS Studio.
- Documentation: Microsoft Word Document.
- Design: Figma [18].
- Timeline and Version Control: Figma [18], GitHub [19].
- Dataset Preparation, Coding and Manipulation: Jupyter Notebook, Visual Studio Code.

These tools have been selected to facilitate effective communication for feedback and guidance as well as ensure organized progress monitoring and code management. This organized planning approach aims to promote systematic advancements and ensure that the deliverables adhere to academic and technical standards.

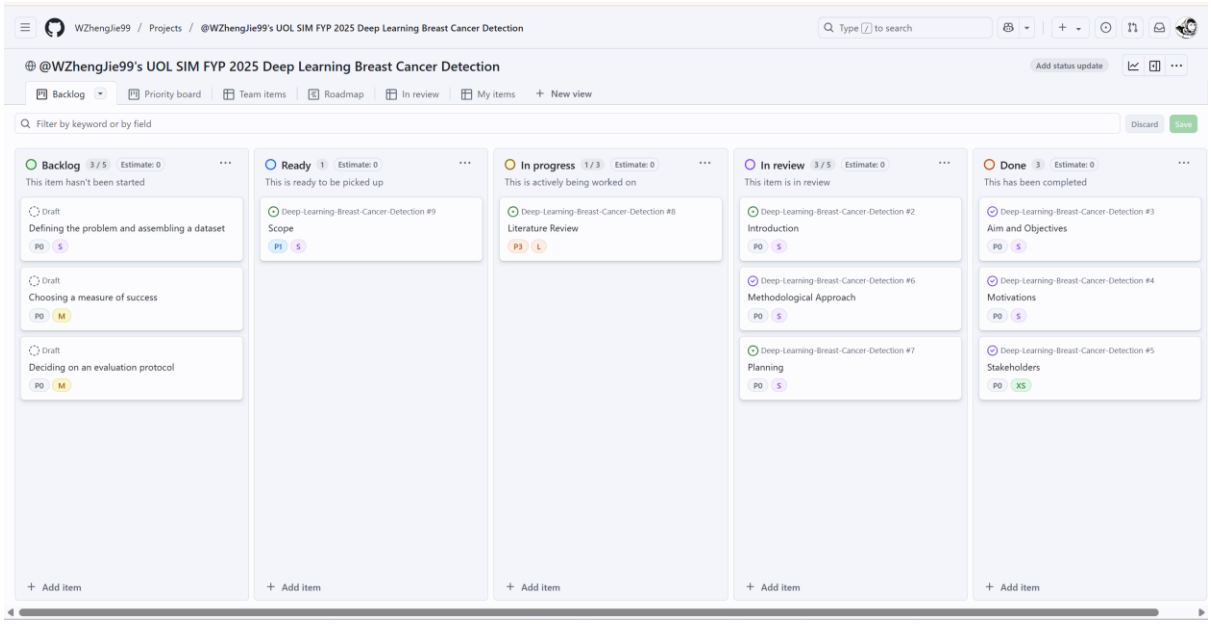


Figure 1.8.1 This GitHub Org Kanban Scheduling system [20]

GitHub [19] is chosen for managing timelines, version control, and prototyping, to safeguard organized access to code and resources throughout the project. By employing GitHub Projects' kanban-style scheduling system [20], the project timeline and tasks can be clearly visualized and managed effectively. (See Figure 1.8.1 and 12. Appendix A – Supplementary Planning Content for further details.)

2. Literature Review

The review of literature evaluates the present condition of AI DL in breast cancer screening, focusing on the difficulties and functionalities of comparable tools. This section examines common and recent methods used in DL image classification aimed towards medical imagery. (See 14. Appendix C – Supplementary Literature Review for further details.)

2.1 Breast Cancer DL Overview

Many articles have compared various imaging techniques such as digital breast tomosynthesis (DBT), ultrasound and magnetic resonance, but mammography continues to be the main tool when it comes to breast cancer diagnosis [6][21].

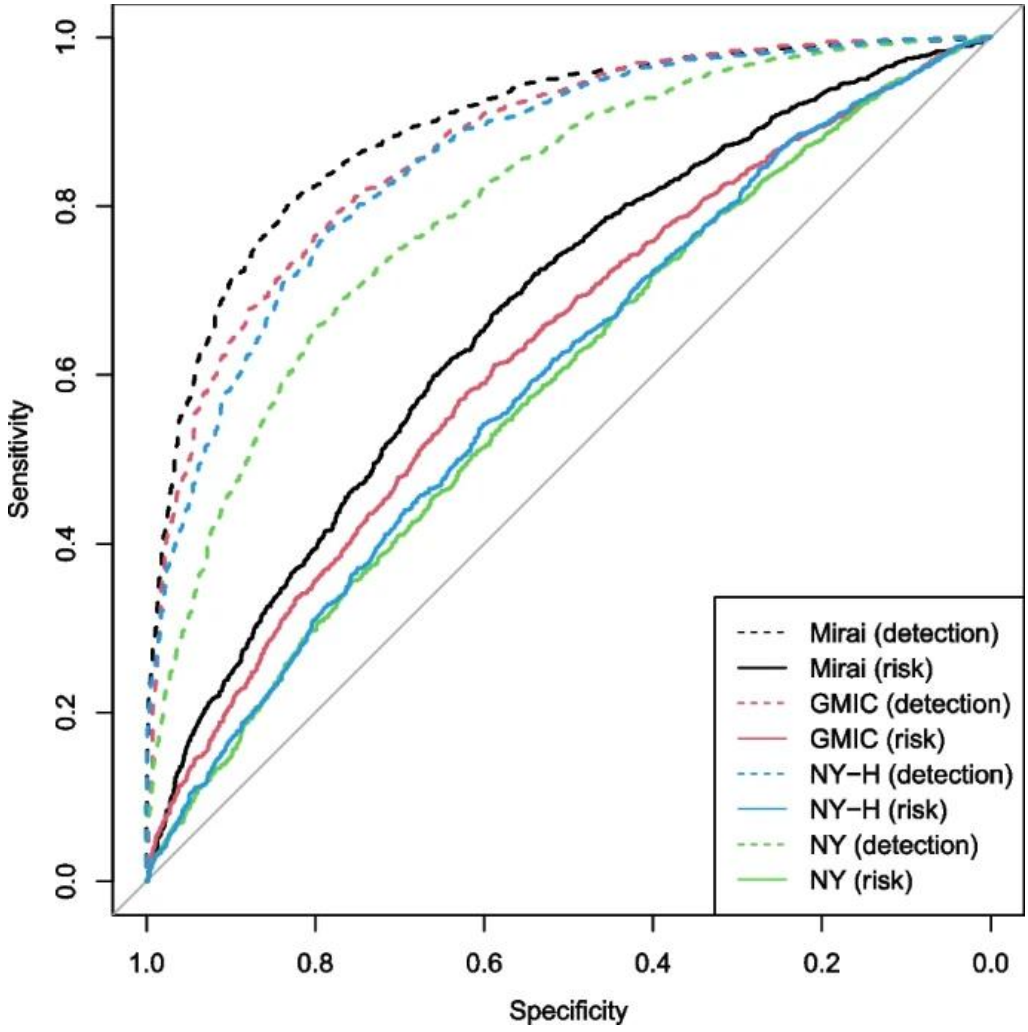


Figure 2.1.1 Receiver operating characteristics for detection and risk by algorithm [22]

Additionally, integrating AI into screening programs has demonstrated potential in reducing the workload of radiologists and enhancing diagnostic reliability. A study by Brentnall et al. (2024) [22] discovered that larger cancers, which are "easier to detect" at the time of diagnosis, were also associated with a higher likelihood of malignancy three years before diagnosis,

indicating their role in risk assessment. The paper applied 3 open-sourced algorithms for cancer detection, GMIC, NY [23], and NY-H, where H denotes heatmap and the algorithm is an extension of NY model using heatmaps [23]. These models employ CNN with the paper indicating that, more broadly, deep learning computer vision algorithms can identify complex patterns in breast scans that radiologists are not presently addressing [22].

The paper compares sensitivity (how good the test is at detecting cancer when it is there) and specificity (how good the test is at ruling out cancer when it's not there). Besides standard CNNs, the paper also used Mirai version 0.3.1 as a risk assessment algorithm, proving to have the strongest performance for both risk assessment and detection. Their finding explains that Mirai works well as it finds early signs of cancer. This shows that hybrid DL models can outperform CNN, offering different and better output and results [22][24]. (See Figure 2.1.1)

2.2 Similar Projects

Similar projects include the Uk's National Institute for Health and Care Excellence (NICE) developing AI technologies to aid in detecting fractures in X-rays [25][26][27]. Comparable to ongoing government plans to implement AI technologies in the medical sector, this project sims to integrate AI to improve diagnostic accuracy.

A paper by Yogesh Kumar et al. (2024) [28] had also addressed automating cancer detection using AI based techniques. Various cancer dataset images were used (brain, oral, breast, kidney, etc), and focused on several deep learning models (CNN, DenseNet121, VGG19, etc). The paper highlights DenseNet121 as the model that achieves the highest validation accuracy (99.94%), loss (0.0017), and the lowest Root Mean Square Error (RMSE) (0.036056) for training and validation (0.045826).

2.3 Image Classification Approaches

In a research paper by Kim, Chaeyoung et al. (2023) [29], named "Convolution Neural Network Approaches for Cancer Cell Image Classification," it was noted that tackling the classification of various cancer cell types using a single CNN model proved to be difficult because of their inconsistent sizes and differences in environmental factors

The paper discussed various approaches for image classification such as Traditional CNN [30], Transfer Learning [31], Meta learning [32], Few-shot Learning / N-way K-shot [33], Binary Classification, and observed that a variety of CNN models is necessary to effectively handle the diverse datasets of cancer cell images, and suggests exploring the use of transfer learning and meta-learning techniques.

Similarly, a study by Archana, R., and Jeevaraj, P. S. E (2024) [34] examined DL models for image processing, offering an in-depth analysis of DL techniques used for two key forms of image segmentation: semantic segmentation and instance segmentation. While various datasets where used, most models achieved high accuracy percentages. Most notable approaches include Shahab Ahmad's AlexNet-GRU (99.5% accuracy) [35] and VGG16 (100% accuracy).

2.4 Model Validation

Strong validation is crucial for ensuring generalization, dependability, and protection against overfitting. Typical validation processes consist of dividing data into training, validation (for hyperparameters), and test sets for the final performance evaluation. A ML practitioner within the community notes that "The test set is a safeguard to evaluate the true generalization power of your model and to check if it has robust fitting" [36].

A taxonomy derived from a Lalli Myllyaho et al. (2021) [37] describes approaches such as trial-based validation, redundancy, and safety channels. Aside from traditional splits, systematic validation frameworks explore multiple strategies from simulation and expert assessment to tracking post deployment performance, this is especially crucial for AI systems built for real-world situations.

Several model validation methods have been suggested to enhance the dependability of performance evaluations. Holdout validation is straightforward and requires little computation, but it is very sensitive to the way the dataset is divided and can produce biased outcomes, especially with small or unbalanced datasets [38]. K-fold cross-validation overcomes this constraint by dividing the dataset into k segments, training on k-1 segments in rotation, and validating on the left-out segment [39]. This lowers variance in comparison to holdout validation. Nonetheless, typical k-fold cross-validation does not guarantee that each fold maintains the initial class distribution. In datasets with imbalance, this can lead to deceptive evaluation metrics.

Stratified k-fold cross-validation enhances k-fold cross-validation by preserving class ratios throughout all folds, guaranteeing that both majority and minority classes are sufficiently represented in every iteration [39]. This method yields more consistent and precise performance assessments, especially for classification tasks involving infrequent or inadequately represented classes, like medical image analysis [37]. Stratified k-fold cross-validation minimizes the chances of overfitting to a specific fold while being more efficient in terms of computation than methods such as leave-one-out cross-validation, which can be excessively costly for larger datasets. Stratified k-fold cross-validation is frequently viewed as the favoured approach for validating AI models designed for practical use due to its blend of robustness, accuracy, and efficiency.

2.5 Transfer Learning

Transfer learning continues to be a fundamental approach in contemporary deep learning, allowing models to leverage existing knowledge and reduce the requirement for large datasets. Within the variety of architectures, EfficientNet consistently stands out for its balance between performance and efficiency. Adebayo Rotimi Philip (2024) [40] showed that by fine-tuning EfficientNetB0 [53][54] on the Food101 dataset [41], it reached an accuracy of 97.54%, along with comparable precision (98.21%) and recall (97.02%) all in only five epochs, indicating that

a compact and well-optimized model can rival or outperform larger models with significantly lower computational costs.

In situations where privacy and decentralization are prioritized, like “Distributed Federated Learning-Based Deep Learning Model for Privacy MRI Brain Tumor Detection” by Lisang Zhou et al. (2024) [42], EfficientNetB0 [53][54] surpassed larger CNN models, achieving superior accuracy and reduced loss, even in the presence of client heterogeneity. This highlights its strength in deployment settings where central data is not accessible and computational limitations are common.

Studies by Islam Nur Alam et al (2022) [43] on Transfer Learning Technique with EfficientNet for Facial Expression Recognition System show that EfficientNetB0 [53][54] achieved remarkable outcomes: 99.57% accuracy on CK+ [44] and 100% on the JAFFE [45] datasets following fine-tuning. These results highlight that a limited network, paired with customized transfer learning and strategic layer unfreezing, can perform exceptionally well in highly precise, low-data scenarios.

2.6 Hybrid Architectures and Approaches

Modern literature increasingly highlights the effectiveness of multi-branch and hybrid architectures for deriving comprehensive, multi-scale representations in image classification tasks particularly in critical areas such as medical imaging, remote sensing, and biological classification. A significant instance is Eff-CTM proposal by Shiwei Liu et al. (2024) [46], which combines CNN, Transformer, and MLP components throughout various branches. Initial parallel CNN branches seize local details, succeeded by dual-branch CNN-Transformer-MLP integration in intermediate layers, culminating in an effective Transformer fusion phase. This design attains high accuracy using reduced parameter budgets, showcasing efficient integration of local and global features.

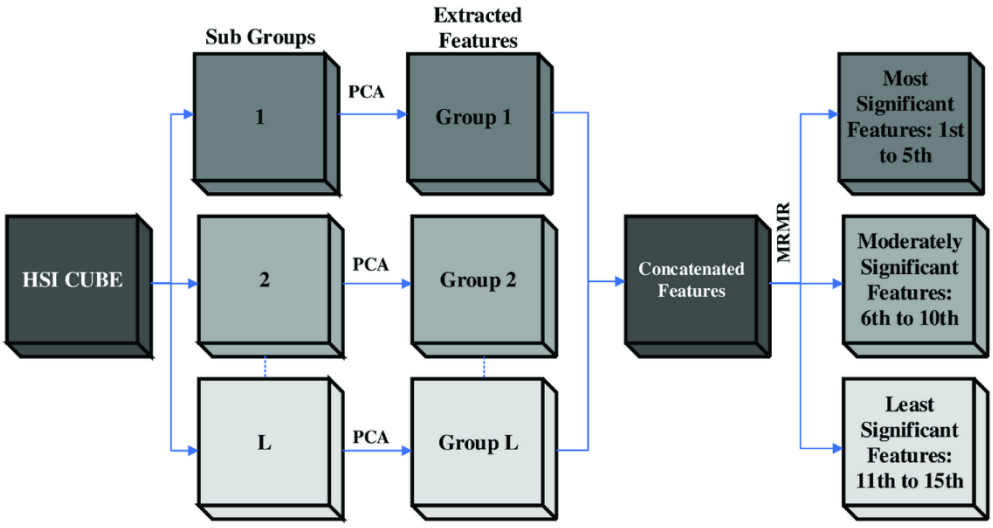


Figure 2.6.1 Processing of Hyperspectral Imagery (HSI) by Tanver Ahmed et al. (2025) [47]

In addition to classification, research by Tanver Ahmed et al. (2025) [47] hyperspectral imaging has gained advantages from a multi-branched 2D-3D CNN architecture. By employing spectral-feature ranking and segmented processing through shallow, medium, and deep branches, this model attained nearly flawless classification on benchmark datasets, highlighting the importance of depth-oriented branching for high-dimensional data. (See Figure 2.6.1)

Hybrid approaches are also present in segmentation tasks. Yundong Zhang et al. (2021) [48] TransFuse utilizes concurrent CNN and Transformer pathways alongside a BiFusion module to merge local and global characteristics, achieving top-tier outcomes in 2D and 3D medical imaging with reduced number of layers.

Overall, these research efforts demonstrate that multi-branch architectures promote deeper feature learning, improved efficiency, and flexibility to fit domain-specific limitations.

3. Scope

The project's scope clarifies what is included and excluded from the development process and sets the limits for its development. This ensures that the project adheres to its aim while addressing resource limitations.

Project Inclusions

The focus of the project is on building a deep learning-based model for breast cancer detection using mammographic images. Implementation includes CNN as it involves image classification. François Chollet's Deep Learning Workflow [16][17] is adopted as the project's development guide but also incorporates other experimentational models that could potentially produce better outcomes and insights.

Additionally, a simple user interface (UI) setup is involved to showcase the model's functionality while prioritizing patient privacy. This would allow radiologists and specialists to upload data, execute, and obtain desired results efficiently.

Project Exclusions

As the project is currently purely academic, clinical deployment in real world settings are excluded and require feedback from industry experts before testing and implementation. Furthermore, the project will exclude the design of a full-featured UI focused on user experience, as the primary objective of the project is to enhance detection performance instead of the UI. DL methods that do not align with the aim of the project are also excluded or avoided from consideration (e.g. architecture designed for NLP or sentiment analysis).

Limitations

The main limitation of the project is the limited dataset size. While there are steps to overcome this limitation, a larger and more diverse dataset is still more advantages. Time constraints due to set deadlines also limits the amount of experimentation and hyper parameter tuning. Lack of computational resources also affects training durations and model scalability. Reproducibility is also another limitation, models trained on the Mini-DDSM dataset may not perform as well with alternative datasets.

Project Estimations

The François Chollet's Deep Learning Workflow [16][17] serves as the foundational structure for the project. However, due to limited resources, only a limited number of model variations will be explored during the experimentation phase.

4. Project Design

Although a simple user interface is included in the project, it is not the main emphasis. Considering that the project deals with X-ray mammography images, the fundamental deep learning strategy revolves around CNNs which are well suited for image analysis. (See 15. Appendix D – CNN and 16. Appendix E – User Interface for further details.)

4.1 Background on Neural Networks

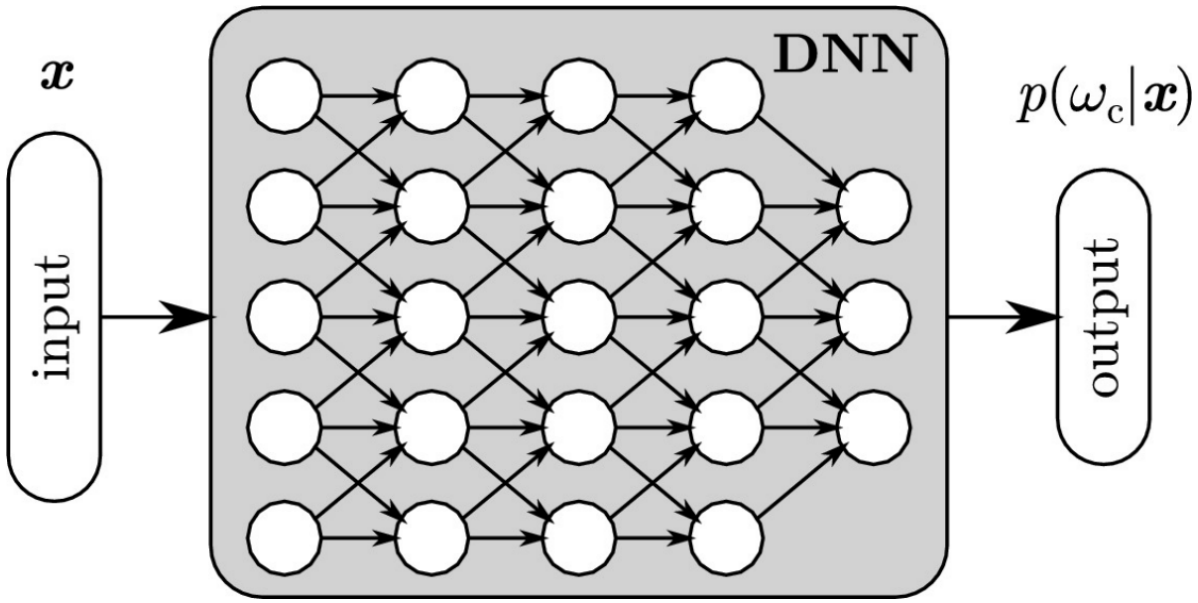


Figure 4.1.1 DNN with the input x a probability of being associated to a certain concept ωc [49]

Artificial Neural Networks (ANNs) are a type of machine learning model that draws inspiration from the human brain. When these networks have several hidden layers between the input and output, they are called Deep Neural Networks (DNNs) [49]. (See Figure 4.1.1)

Convolutional Neural Networks (CNNs) are a specialised type of DNN that is especially effective for handling data with a spatial arrangement, like images. CNNs in recent years have emerged as the leading approach in computer vision tasks, such as image classification, detection, and medical image analysis.

4.2 CNN Architecture Overview

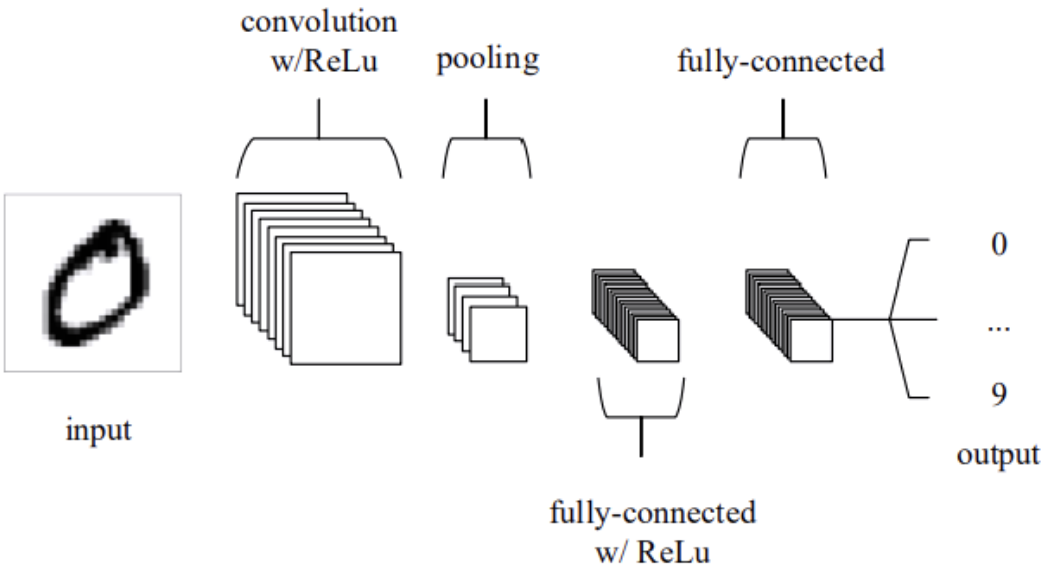


Figure 4.2.1 Simple 5 Layered CNN Architecture [50]

Like ANNs and DNNs, CNNs have an input layer that holds the pixels values of the image, and an output layer that produces the result. While common DNNs are made up of hidden layers, CNNs are made up of 3 distinct layers [50]:

- Convolutional Layers

  The convolutional layers generate the output of neurons that are connected locally to certain areas of the input, by performing the dot product between the filter weights and the related segment of the input volume. The Rectified Linear Unit (ReLU) activation function, commonly applied post-convolution, adds non-linearity by utilizing an elementwise transformation like the sigmoid or ReLU function on the resulting activations.

- Pooling Layers

  The pooling layers conducts spatial down-sampling on the input feature maps, decreasing their size and thereby reducing the number of parameters and computational complexity.

- Fully Connected / Dense Layers

  The fully connected layers, akin to those in standard ANNs, receive the flattened activations from preceding layers and calculate class scores for the final classification. ReLU activations can also be utilized here between these layers to improve performance and learning effectiveness.

Through the iterative use of convolution and down-sampling processes, CNNs can modify the initial input across several layers, eventually producing class scores for tasks like classification and regression. (See Figure 4.2.1 and 15. Appendix D – CNN Layers for further details.)

Although CNNs are powerful, they do have their drawbacks. CNNs generally need substantial quantities of labelled data to function effectively, and their black box nature creates issues regarding reliability and explanation in choices, especially in critical fields such as cancer detection. While the black box nature remains a complex challenge to overcome, this project aims to improve overall accuracy before tackling the black box challenge.

4.3 About Stratified K-fold Cross Validation

Stratified K-fold Cross Validation [39] is a method of resampling employed to assess the performance and generalizability of machine learning models. In contrast to a traditional train–test split that assesses performance on one data partition, K-fold Cross Validation splits the dataset into k folds of equal size. During each iteration, a single fold is employed for validation, while the other k-1 folds are utilized for training, and this continues until all folds have acted as the validation set. The outcomes are subsequently averaged over all folds to deliver a more dependable performance assessment.

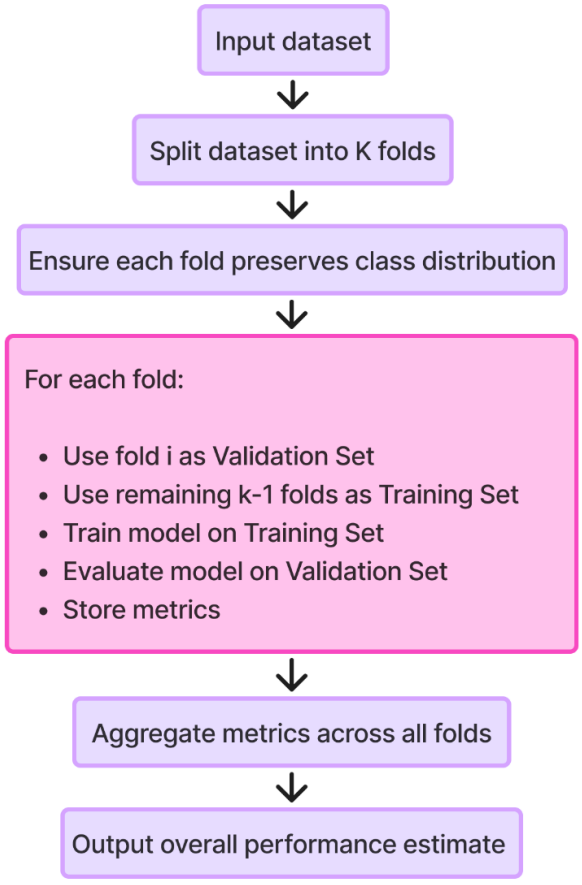


Figure 4.3.1 This Stratified K-Fold Cross Validation Pipeline

The Stratified K-fold guarantees that the class distribution in every fold mirror's the complete dataset. This holds significant importance in medical imaging or other fields with class imbalance, where specific classes might appear less often. In the absence of stratification, certain folds might not sufficiently represent minority classes, resulting in skewed evaluation outcomes.

When utilized for CNNs, Stratified K-fold Cross Validation has multiple functions. Firstly, it aids in minimizing overfitting by confirming that the model undergoes validation across various, distinctive data subsets. Secondly, it offers a broader evaluation of model resilience, as CNNs trained on medical or visual data frequently encounter differences in image quality / resolution, patient related attributes, and class distribution. Thirdly, it enables a more equitable comparison of models by averaging metrics like accuracy, loss, and AUC-ROC across folds instead of relying on just one split.

In general, Stratified K-fold Cross Validation provides a more robust statistical foundation for assessing CNN models, particularly in critical areas such as medical diagnosis where maintaining balanced performance across all classes is essential.

4.4 Stratified K-fold Cross Validation: Dataset Overview

The dataset contains three categories: Benign (B), Cancer (C), and Normal (N). In stratified k-fold cross validation (i.e. k=5), every fold keeps a similar class distribution as that of the entire dataset.



Figure 4.4.1 Stratified K-fold Cross Validation Process Illustration

For example, if the dataset contains Benign (B), Cancer (C), and Normal (N) at 50%, 30%, and 20% respectively, each fold will preserve this distribution. During each iteration, the model undergoes training with four of the folds while the leftover fold is used for validation. This

procedure persists until every fold has been utilized for validation, after which the performance outcomes are averaged. Maintaining class balance in this manner, stratification avoids biased evaluations and offers a more dependable estimate of the model's ability to generalize to new data.

4.5 Application of Transfer Learning

Transfer learning [51][52] is a machine learning technique where a model that is created for a specific task is utilized as the foundation for another similar task. Instead of building a deep neural network from the ground up, transfer learning utilizes the insights acquired from models trained on extensive benchmark datasets. This technique is especially beneficial in medical imaging, where labelled data is frequently scarce. By moving acquired features like edges, textures, and shape recognizers from the source task, the target model gains strong representations, which shortens training duration and decreases the chance of model overfitting.

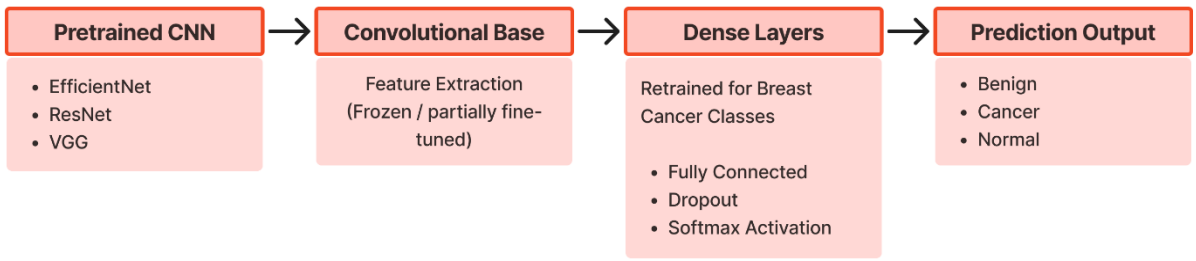


Figure 4.5.1 Transfer Learning CNN Pipeline

In practical applications, transfer learning is often utilized with CNNs by leveraging pre-trained feature extraction layers and modifying the final classification layers to fit the new task. For example, a CNN such as EfficientNet [53][54], ResNet [52], or VGG [55] that has been trained on ImageNet [56] can be adapted for breast cancer image classification by keeping its convolutional base and re-train the final dense layers for the target classes.

Fine-tuning can be utilized to selectively retrain the deeper layers of the pre-trained network to more effectively identify domain-specific characteristics. This architectural framework allows the model to generalize well even with minimal domain data, resulting in better performance rather than starting from the ground up.

Transfer learning is especially useful in areas with scarce data like medical image analysis where building a deep network from the ground up can result in overfitting. Some commonly used pretrained models comprise of ResNet50 [57], InceptionV3 [58], and EfficientNetB0 [53][54], with each offering a balance among accuracy, computational efficiency, and parameter complexity.

Among these, EfficientNetB0 [53][54] provides an appealing equilibrium for tasks related to medical image classification. Its compound scaling method consistently modifies depth, width, and resolution, enabling it to reach cutting-edge performance with fewer parameters than

deeper networks like ResNet50 [57]. This efficiency results in quicker convergence during fine-tuning and improved generalization on smaller datasets. In comparison, although ResNet [57] function effectively as feature extractors, they might demand additional computational resources and extended training periods.

By leveraging EfficientNetB0 [53][54] in combination with stratified k-fold cross-validation, this project ensures both robust evaluation and efficient learning, justifying the implementation choices over other architectures.

4.6 Design Considerations on Hybrid Models and Multi-Branching Architecture

CNNs serve as the core framework for image classification and medical image analysis thanks to their capacity to autonomously extract hierarchical features from unprocessed image data. Numerous established CNN architectures have been extensively utilized in both research and practical applications, each offering distinct advantages and compromises. VGG [55] networks utilize deep, consistent layers paired with small convolutional filters, showcasing excellent feature extraction abilities but demanding significant computational resources because of the large number of parameters. ResNet [57] incorporates residual links that allow very deep networks to converge efficiently, alleviating the vanishing gradient issue and enhancing precision on intricate datasets. Architectures like Inception [58] enhance multi-scale feature extraction through parallel convolutional paths, while MobileNet [59] focuses on computational efficiency for environments with limited resources.

Although these architectures have shown exceptional performance, they typically utilize a single-pathway strategy, restricting the variety of learned features, especially in tasks involving heterogeneous or multi-scale data like medical imaging. In contrast, multi-branching architectures allow the network to acquire complementary representations via parallel branches that analyse the same input with different feature extraction methods. Every branch might emphasize various elements of the data like texture, edge patterns, or high-level semantic details and the features obtained are subsequently combined to improve overall representational strength. This design demonstrates enhanced classification accuracy, robustness, and flexibility in tasks where minor class differences are crucial, like differentiating between Normal and Cancer tissues in mammograms.

The primary benefit of a multi-branching method is its capacity to integrate various processing techniques into one architecture. Though it brings added computational complexity, this expense is balanced by substantial improvements in feature richness and predictive accuracy. In contrast to conventional single-path CNNs, multi-branch architectures are more effective at capturing subtle and multi-scale characteristics, which lessens the risk of overfitting to a particular pattern. Additionally, multi-branch architectures work well with transfer learning, enabling pretrained networks like EfficientNetB0 [53][54] to function as backbone branches, thereby speeding up convergence and enhancing generalization on domain-specific datasets.

4.7 About Multi-Branching Architecture

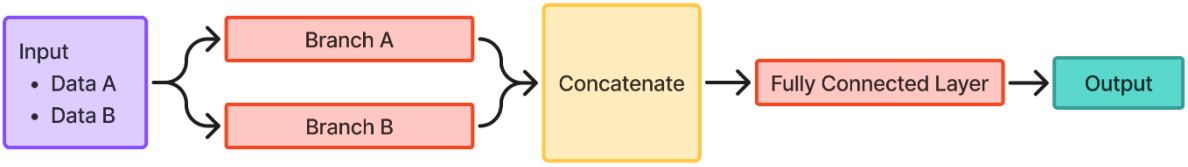


Figure 4.7.1 Dual Branch Architecture Example Overview

Multi-branching is a deep learning architecture approach where a model divides into several parallel branches [60], with each branch acquiring complementary representations of the input data before converging into a single layer. This contrasts with conventional single-route CNNs, which handle data in a linear fashion through layered stacking. (See Figure 4.7.1)

The reason for multi-branching is to capture different features at various depths or scales. Enabling various branches to handle the same input in distinct ways, such as different kernel sizes, resolutions, or methods of feature extraction, allows the model to obtain more complex feature representations and enhance generalization.

Disadvantages:

- Increased computational costs such as training time and greater memory usage
- Risks of overfitting due to large / complex model relative to dataset size
- Training instability when trying to balance learning across multiple branches

Advantages:

- Diverse learning paths helps to improved feature representation
- Greater flexibility in combining multiple processing strategies
- Potential for higher accuracy with complex datasets

While multi-branching introduces certain disadvantages, its advantages often outweigh these challenges. Through diverse learning paths that improve feature representation, offering greater flexibility in combining multiple processing strategies, allowing greater flexibility in merging different processing techniques, and facilitating increased accuracy with intricate datasets, multi-branching continues to be an important approach in deep learning architectures.

5. Implementation

5.1 Defining the problem and assembling a dataset

The primary objective outlined in this project is a image classification problem, where an initial model is developed to differentiate between mammographic images with and without indications of breast cancer, and is progressively improved upon. Classification is carried out using diagnostic labels found in the dataset, particularly distinguishing between malignant and non-malignant instances.

For this project, the Mini-DDSM dataset [3][4] is utilized as the primary dataset. As outlined in Section 1.2 Data Source and Relevancy, it includes annotated mammogram images in standard formats and has been prepared for seamless integration into deep learning processes. The dataset should be split into training, validation, and testing subsets, guaranteeing a stratified class distribution to keep balance throughout each set.

5.2 Choosing a measure of success

In a medical setting, especially for potential clinical settings, like breast cancer detection, depending exclusively on accuracy as a performance measure can be deceptive, particularly in datasets with class imbalance or when the cost of misclassification is asymmetric. In this project, a true positive prediction (detecting cancer when it exists) holds considerably more clinical significance than a true negative prediction. On the other hand, a false negative (wrongly indicating that cancer is absent) might result in postponed diagnosis and treatment, making it crucial to minimise such probable mistakes.

While accuracy will be reported, it is not the main success criterion. To achieve a more comprehensive insight into model behaviour, confusion matrix will be used to illustrate the tallies of true positives, true negatives, false positives, and false negatives. This allows for immediate understanding of the kinds of mistakes the model is making and their frequency. To guarantee a thorough assessment, the following set of applicable performance indicators will also be applied [61]:

- **Accuracy**

  Determines the proportion of correct predictions relative to the overall predictions made. While beneficial for an overall grasp of performance, it can mislead in irregular datasets. A classifier that reliably predicts the dominant class could achieve a high accuracy rate but may fail to perform well on the minority class. Accuracy Equation:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Sensitivity (Recall)**

  Evaluates the ratio of correctly identified actual positive cases (malignant). Having high sensitivity is essential for reducing the number of false negatives. Sensitivity Equation:

$$Sensitivity = \frac{TP}{TP + FN} \qquad Recall = \frac{TP}{TP + TN}$$

- **Specificity**

  Evaluates the model's capacity to accurately recognize negative instances (benign or normal), aiding in the decrease of false positives. Specificity Equation:

$$Specificity = \frac{TN}{TN + FP}$$

- **Precision**

  Shows the fraction of accurate positive predictions, taking false positives into account. Precision Equation:

$$Precision = \frac{TP}{TP + FP}$$

- **F1-Score**

  The harmonic mean of precision and recall is beneficial for assessing performance when both false positives and false negatives are relevant. F1-Score Equation:

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- **Area Under the ROC Curve (AUC-ROC)**

  Represents the balance between true positive and false positive rates at different thresholds and offers a solid metric of separability. AUC Equation:

$$AUC = \frac{Sp - Np\,(\,Nn + 1\,)\,/\,2}{NpNn}$$

- **Confusion Matrix**

  Summarizes the classification models performance by showing correct and incorrect classifications.

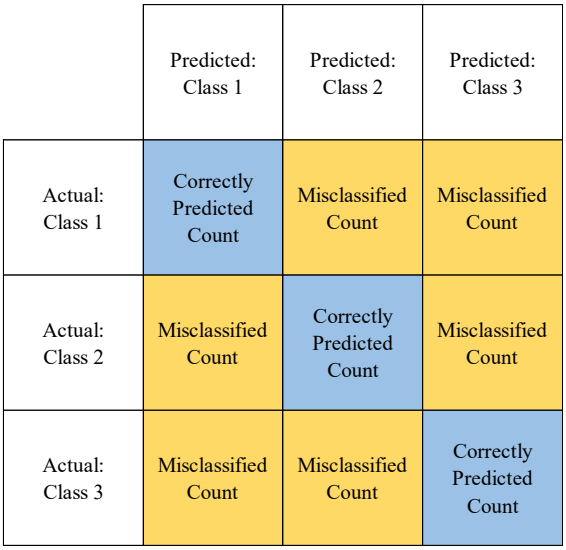


Figure 5.2.1 Confusion Matrix Illustration

These metrics will be evaluated on the validation and testing datasets to determine both model performance and generalization. The model's effectiveness will not solely be determined by its numerical outcomes across these metrics but also by its consistency and dependability in reducing significant errors.

5.3 Deciding on an evaluation protocol

To guarantee a strong and impartial assessment of model effectiveness, a clearly defined evaluation protocol should be created. The data is divided into three separate subsets: training, validation, and testing. This permits distinct stages of learning, model adjustment, and ultimate performance assessment without data contamination or overfitting to the test dataset.

The stratified splitting approach alongside class weights, is planned to be utilized to preserve the identical ratio of malignant and non-malignant cases throughout all three subsets. This is crucial in medical imaging tasks, where imbalanced datasets frequently occur and might otherwise result in distorted or misleading performance metrics.

Due to the moderate scale of the Mini-DDSM dataset, approaches like data augmentation may be employed to enhance the effective training size and minimize overfitting. Moreover, early stopping could be considered to guarantee that the model generalizes effectively and is not overly tailored to the validation dataset.

This evaluation protocol guarantees that performance metrics accurately represent the model’s generalization capability, especially in real-world clinical screening and medical settings, by distinctly separating training, validation, and testing phases.

5.4 Preparing your data

Before preparing the dataset for training, we should understand the data we are dealing with. The Mini-DDSM file structure was examined and included a JPEG-8 and PNG-16 version. The PNG-16 format was chosen because it means 16-bit depth, preserving significantly more pixel intensity details, which is crucial for identifying slight tissue variations. (See 13. Appendix B – The Complete Mini-DDSM archive Folder Structure and Appendix Figure 19.1.2 for further details)

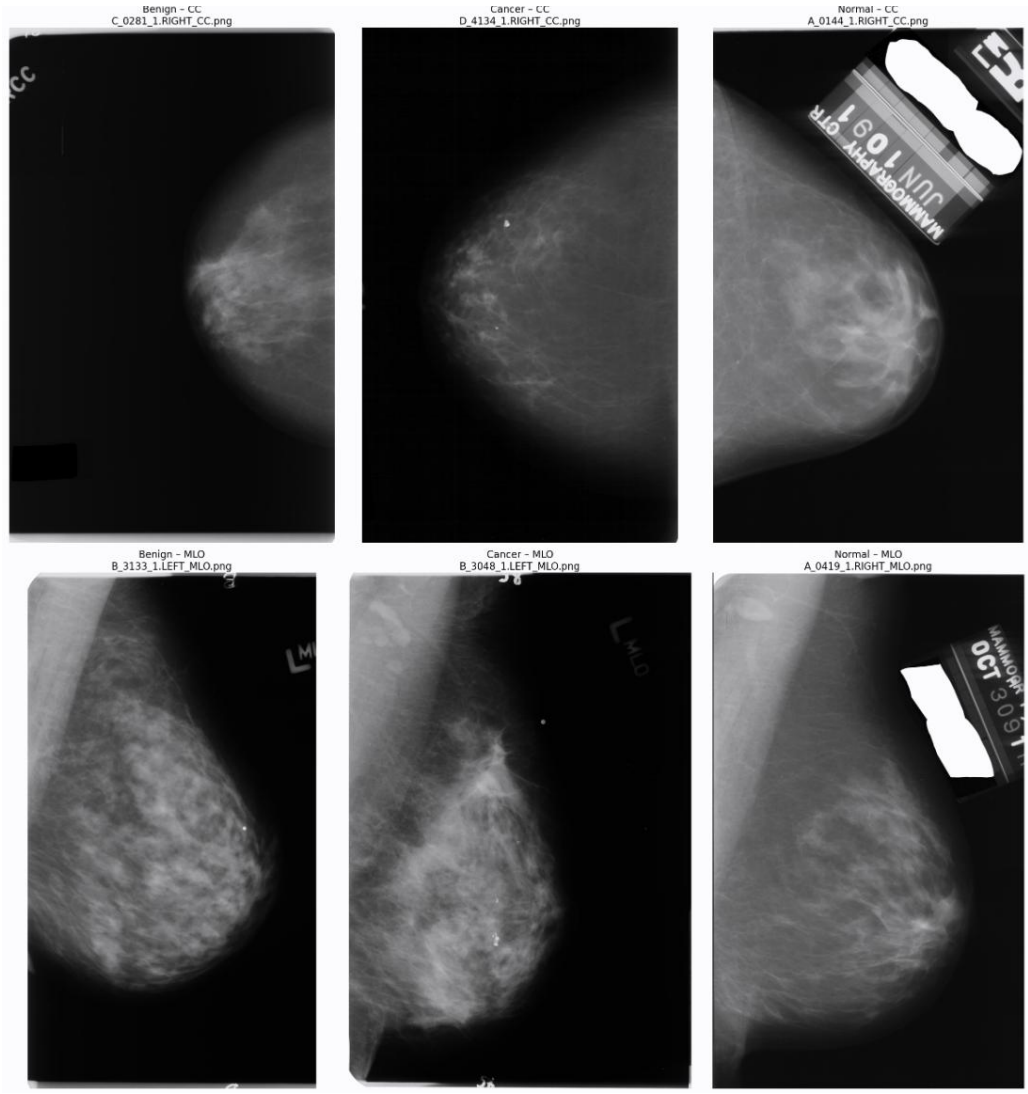


Figure 5.4.1 CC and MLO Breast Cancer Mammography Image Preview

The folders included 2 views for each breast, CC and MLO. The CC view presents a vertical (top down) perspective, displaying a larger portion of the whole breast, while the MLO view offers a lateral (side view) perspective of the breast, generally highlighting more of the upper outer area. (See Figure 5.4.1)

Preprocessing and augmentation are being applied to improve generalization. Images are resized to 224×224 pixels with 32 images being processed per batch during training. Images are normalized to to [0.0, 1.0] and 20% of the data is reserved for validation and test.

Augmentation was not done for validation and test set so that the model can be evaluated on unaltered images.

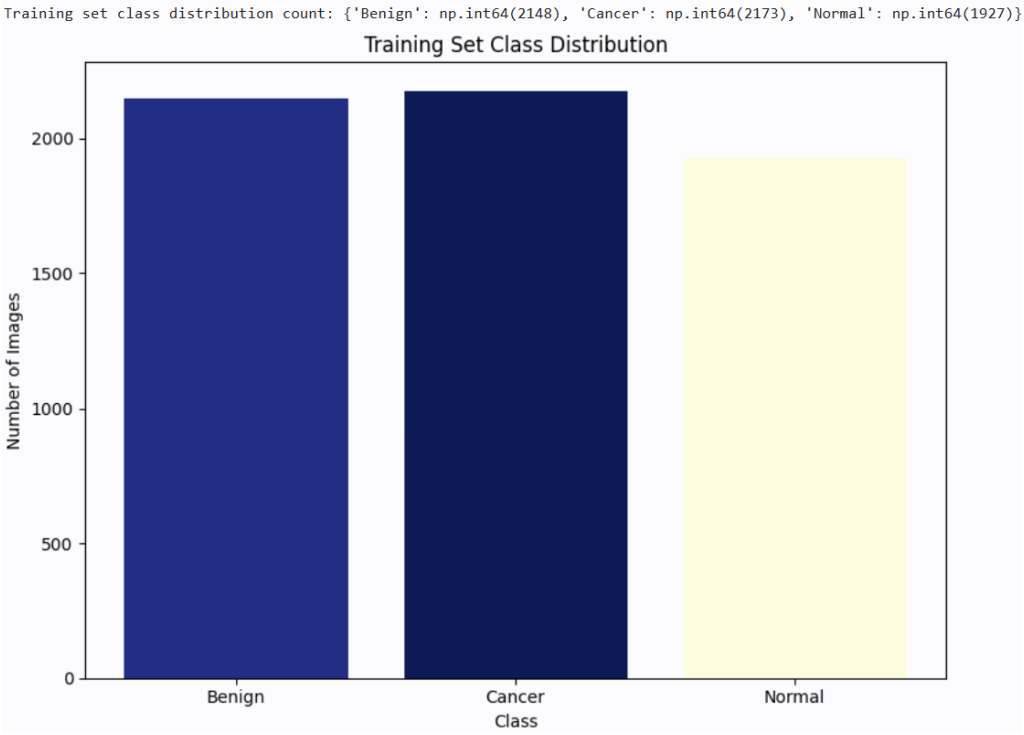


Figure 5.4.2 Baseline Model Training Set Class Distribution

After splitting training (80% of data) and validation (20% of data), class imbalance was addressed by adding class weights to prevent biasness. (See Figure 5.4.2)

5.5 Developing a model that does better than a baseline

Naïve Baseline Model

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Table 5.5.1 Naive Baseline Model Summary Table

The naive baseline model acts as a basic reference for assessing more complex architectures later in the project. It utilizes a simple CNN architecture and incorporates class weighting to tackle class imbalance. Although straightforward, this model sets a baseline expected performance and aids in uncovering initial issues, like misclassification trends or sensitivity to underrepresented classes. (See Table 5.5.1)

Baseline Model

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 224, 224, 32)	320
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_4 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_5 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense_2 (Dense)	(None, 128)	16,512
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

Table 5.5.2 Baseline Model Summary Table

The baseline model builds upon the naïve version by adding further improvements designed to enhance learning ability and generalization. Although still quite simple, the baseline model establishes a more solid and resilient benchmark.

Key modifications comprised incorporating Batch Normalization layers following each convolution to enhance and quicken training, utilizing padding='same' to maintain spatial dimensions, and implementing mild L2 regularization on the last convolutional layer to reduce overfitting. Moreover, Global Average Pooling took the place of Flatten() to lessen overfitting and enhance spatial generalization, and early stopping and model checkpointing were adopted to keep the best model and avoid redundant training.

5.6 Scaling up: developing a model that overfits

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 64)	640
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256
conv2d_1 (Conv2D)	(None, 224, 224, 64)	36,928
batch_normalization_1 (BatchNormalization)	(None, 224, 224, 64)	256
max_pooling2d (MaxPooling2D)	(None, 112, 112, 64)	0
conv2d_2 (Conv2D)	(None, 112, 112, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 112, 112, 128)	512
conv2d_3 (Conv2D)	(None, 112, 112, 128)	147,584
batch_normalization_3 (BatchNormalization)	(None, 112, 112, 128)	512
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 128)	0
conv2d_4 (Conv2D)	(None, 56, 56, 256)	295,168
batch_normalization_4 (BatchNormalization)	(None, 56, 56, 256)	1,024
conv2d_5 (Conv2D)	(None, 56, 56, 256)	590,080
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 256)	1,024
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 512)	131,584
dense_1 (Dense)	(None, 3)	1,539

Table 5.6.1 Overfitting Model Summary Table

Understanding the intricacies required to align the data is vital for determining appropriate model architecture and regularization techniques. The overfitting model assists us in identifying the point when the model becomes overly intricate and starts to overfit, which helps in choosing regularization techniques. Epochs were increased from 30 to 50, and larger filter sizes were employed to achieve overfitting.

5.7 Regularizing model and tuning hyperparameters

Stratified K-fold Cross-Validation was applied pre regularized model implementation to guarantee a thorough and equitable assessment while preserving class distribution among folds, offering insights into the stability and generalization ability of the baseline model before implementing regularization and hyperparameter optimizations.

Result output of each fold:

- Fold 1 results summary: Accuracy: 0.6466, Loss: 0.7774, AUC-ROC: 0.8392
- Fold 2 results summary: Accuracy: 0.6588, Loss: 0.7223, AUC-ROC: 0.8508
- Fold 3 results summary: Accuracy: 0.6428, Loss: 0.7691, AUC-ROC: 0.8382
- Fold 4 results summary: Accuracy: 0.6265, Loss: 0.7838, AUC-ROC: 0.8268
- Fold 5 results summary: Accuracy: 0.6611, Loss: 0.7290, AUC-ROC: 0.8443

```
print("Stratified K-Fold Cross-Validation Regularized Model Result Sumary")
print(f"Mean Accuracy: {np.mean(fold_accuracies):.4f} ± {np.std(fold_accuracies):.4f}")
print(f"Mean Loss: {np.mean(fold_losses):.4f} ± {np.std(fold_losses):.4f}")
print(f"Mean AUC-ROC: {np.mean(fold_aucs):.4f} ± {np.std(fold_aucs):.4f}")
```

```
Stratified K-Fold Cross-Validation Regularized Model Result Sumary
Mean Accuracy: 0.6472 ± 0.0125
Mean Loss: 0.7563 ± 0.0256
Mean AUC-ROC: 0.8399 ± 0.0079
```

Figure 5.7.1 K-fold Cross Validation Results

The Stratified K-fold Cross-Validation indicate significant enhancements compared to earlier single-split assessments, reaching a mean accuracy of 0.6278% ± 0.015%, alongside a mean loss of 0.7892 ± 0.0526 over the folds. Significantly, the model achieved a mean AUC-ROC of 0.8196 ± 0.0177, demonstrating substantial discriminatory capabilities beyond random chances. These findings validate the efficacy of stratified cross-validation in offering a reliable, representative assessment while addressing the overfitting and class imbalance problems. (See Figure 5.7.1)

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 224, 224, 32)	320
batch_normalization_3 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 32)	0
spatial_dropout2d (SpatialDropout2D)	(None, 112, 112, 32)	0
conv2d_4 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0
spatial_dropout2d_1 (SpatialDropout2D)	(None, 56, 56, 64)	0
conv2d_5 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 128)	0
spatial_dropout2d_2 (SpatialDropout2D)	(None, 28, 28, 128)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	0
dense_2 (Dense)	(None, 128)	16,512
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

Table 5.7.1 Regularized Model Summary Table

To address the overfitting model, the regularized model is built upon the baseline model by adding more regularization techniques. L2 regularization was implemented throughout all convolutional and dense layers to discourage large weights and minimize overfitting. SpatialDropout2D was also added to each pooling layer to encourage robustness in spatial feature learning, and dropout rate increased gradually to mitigate overfitting risk from high-capacity layers.

5.8 Wider, Deeper, Narrower Models

Wider, deeper, and narrower model configurations are being tested to help examine the impact of capacity on learning on breast screening mammography images and acts as an extension of tuning hyperparameters. The wider model was developed to enable the network to recognize more complex patterns in the data. On the other hand, deeper model enables the network to grasp more intricate hierarchical representations. Finally, the narrower model, prioritizing simplicity and assist in mitigating the chances of overfitting by reducing the model's complexity.

Wider Model

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 224, 224, 64)	640
batch_normalization_6 (BatchNormalization)	(None, 224, 224, 64)	256
max_pooling2d_6 (MaxPooling2D)	(None, 112, 112, 64)	0
spatial_dropout2d_6 (SpatialDropout2D)	(None, 112, 112, 64)	0
conv2d_7 (Conv2D)	(None, 112, 112, 128)	73,856
batch_normalization_7 (BatchNormalization)	(None, 112, 112, 128)	512
max_pooling2d_7 (MaxPooling2D)	(None, 56, 56, 128)	0
spatial_dropout2d_7 (SpatialDropout2D)	(None, 56, 56, 128)	0
conv2d_8 (Conv2D)	(None, 56, 56, 256)	295,168
batch_normalization_8 (BatchNormalization)	(None, 56, 56, 256)	1,024
max_pooling2d_8 (MaxPooling2D)	(None, 28, 28, 256)	0
spatial_dropout2d_8 (SpatialDropout2D)	(None, 28, 28, 256)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 256)	0
dense_2 (Dense)	(None, 256)	65,792
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 3)	771

Table 5.8.1 Wider Model Summary Table

The wider model was created by doubling the filters per layer (64, 128, 256), allowing the model to explore capacity while maintaining the previous architecture.

Deeper Model

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	320
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
spatial_dropout2d (SpatialDropout2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 32)	128
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 32)	0
spatial_dropout2d_1 (SpatialDropout2D)	(None, 56, 56, 32)	0
conv2d_2 (Conv2D)	(None, 56, 56, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 64)	256
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 64)	0
spatial_dropout2d_2 (SpatialDropout2D)	(None, 28, 28, 64)	0
conv2d_3 (Conv2D)	(None, 28, 28, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	0
spatial_dropout2d_3 (SpatialDropout2D)	(None, 14, 14, 64)	0
conv2d_4 (Conv2D)	(None, 14, 14, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 14, 14, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
spatial_dropout2d_4 (SpatialDropout2D)	(None, 7, 7, 128)	0
conv2d_5 (Conv2D)	(None, 7, 7, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 3, 3, 128)	0
spatial_dropout2d_5 (SpatialDropout2D)	(None, 3, 3, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Table 5.8.2 Deeper Model Summary Table

The deeper model is modelled by adding an additional 32, 64, and 128 filtered layer on top of the regularized model.

Narrower Model

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 224, 224, 16)	160
batch_normalization_9 (BatchNormalization)	(None, 224, 224, 16)	64
max_pooling2d_9 (MaxPooling2D)	(None, 112, 112, 16)	0
spatial_dropout2d_9 (SpatialDropout2D)	(None, 112, 112, 16)	0
conv2d_10 (Conv2D)	(None, 112, 112, 32)	4,640
batch_normalization_10 (BatchNormalization)	(None, 112, 112, 32)	128
max_pooling2d_10 (MaxPooling2D)	(None, 56, 56, 32)	0
spatial_dropout2d_10 (SpatialDropout2D)	(None, 56, 56, 32)	0
conv2d_11 (Conv2D)	(None, 56, 56, 64)	18,496
batch_normalization_11 (BatchNormalization)	(None, 56, 56, 64)	256
max_pooling2d_11 (MaxPooling2D)	(None, 28, 28, 64)	0
spatial_dropout2d_11 (SpatialDropout2D)	(None, 28, 28, 64)	0
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
dropout_2 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 3)	195

Table 5.8.3 Narrower Model Summary Table

As opposed to the wider model, the narrower model was created by reducing the filters per layer by half (16, 32, 64).

Narrower Model Two

Layer (type)	Output Shape	Param #
conv2d_15 (Conv2D)	(None, 2560, 1440, 16)	160
batch_normalization_15 (BatchNormalization)	(None, 2560, 1440, 16)	64
max_pooling2d_15 (MaxPooling2D)	(None, 1280, 720, 16)	0
spatial_dropout2d_15 (SpatialDropout2D)	(None, 1280, 720, 16)	0
conv2d_16 (Conv2D)	(None, 1280, 720, 32)	4,640
batch_normalization_16 (BatchNormalization)	(None, 1280, 720, 32)	128
max_pooling2d_16 (MaxPooling2D)	(None, 640, 360, 32)	0
spatial_dropout2d_16 (SpatialDropout2D)	(None, 640, 360, 32)	0
conv2d_17 (Conv2D)	(None, 640, 360, 64)	18,496
batch_normalization_17 (BatchNormalization)	(None, 640, 360, 64)	256
max_pooling2d_17 (MaxPooling2D)	(None, 320, 180, 64)	0
spatial_dropout2d_17 (SpatialDropout2D)	(None, 320, 180, 64)	0
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 64)	0
dense_8 (Dense)	(None, 64)	4,160
dropout_4 (Dropout)	(None, 64)	0
dense_9 (Dense)	(None, 3)	195

Table 5.8.4 Narrower Model Two Summary Table

A second narrower model was re-implemented with increased image height and width dimensions (2560 x 1440) in hopes of achieving better results and compare outputs with resolution differences.

5.9 Transfer Learning Model

Transfer learning model was created with EfficientNetB0 [53][54], pretrained on ImageNet [56] to improve classification performance. Due to EfficientNetB0's [53][54] requirement for RGB input, grayscale mammograms were transformed into RGB, employing TensorFlow's ImageDataGenerator with color_mode='rgb'. Techniques for data augmentation, such as random rotations, shifts, zooms, and flips, were utilized to enhance generalization.

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None, 1280)	0
dense_10 (Dense)	(None, 128)	163,968
dropout_5 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 3)	387

Table 5.9.1 Transfer learning Model Summary Table

At the start, the layers of the base model were locked to utilize the pretrained feature extraction while the new top layers were trained on the mammogram dataset. The EfficientNetB0 [53][54] base model was imported without its upper layers, and a tailored classification head comprising a GlobalAveragePooling2D layer, a dense layer featuring 128 units with ReLU activation, a dropout layer for regularization, and a concluding dense layer with softmax activation for the three-class classification task was also incorporated.

\

5.10 Exploring Models and Architectures

Narrower and Transfer Learning Multi-Branch Model

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 4)	0	-
conv2d (Conv2D)	(None, 224, 224, 16)	592	input_layer[0][0]
batch_normalization (BatchNormalizatio…	(None, 224, 224, 16)	64	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 112, 112, 16)	0	batch_normalizat…
spatial_dropout2d (SpatialDropout2D)	(None, 112, 112, 16)	0	max_pooling2d[0]…
conv2d_1 (Conv2D)	(None, 112, 112, 32)	4,640	spatial_dropout2…
batch_normalizatio… (BatchNormalizatio…	(None, 112, 112, 32)	128	conv2d_1[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 32)	0	batch_normalizat…
spatial_dropout2d_1 (SpatialDropout2D)	(None, 56, 56, 32)	0	max_pooling2d_1[…
conv2d_2 (Conv2D)	(None, 56, 56, 64)	18,496	spatial_dropout2…
batch_normalizatio… (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2d_2[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 64)	0	batch_normalizat…
slice_rgb (Lambda)	(None, 224, 224, 3)	0	input_layer[0][0]
spatial_dropout2d_2 (SpatialDropout2D)	(None, 28, 28, 64)	0	max_pooling2d_2[…
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571	slice_rgb[0][0]
global_average_poo… (GlobalAveragePool…	(None, 64)	0	spatial_dropout2…
global_average_poo… (GlobalAveragePool…	(None, 1280)	0	efficientnetb0[0…
concatenate (Concatenate)	(None, 1344)	0	global_average_p… global_average_p…
dense (Dense)	(None, 128)	172,160	concatenate[0][0]
dropout (Dropout)	(None, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 3)	387	dropout[0][0]

Table 5.10.1 Narrower and Transfer Learning Multi-Branch Model Summary Table

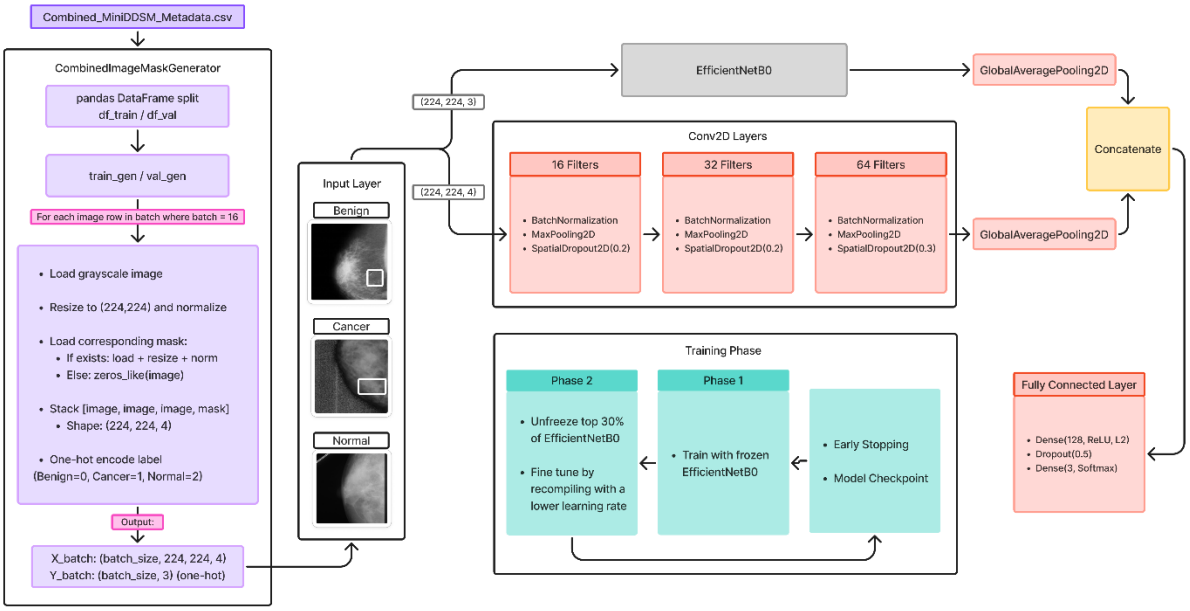


Figure 5.10.1 Narrower and Transfer Learning Multi-Branch Model Architecture

A multi-branch architecture was developed with mask images pinpointing ROIs using narrower model for training Benign and Normal, and transfer learning EfficientNetB0 [53][54] model for training the Cancer class. (See Figure 5.10.1)

Modified U-Net Model

Modified U-Net was also implemented for its performance on medical image segmentation with limited data and can provide a strong foundation for further advanced architectures.

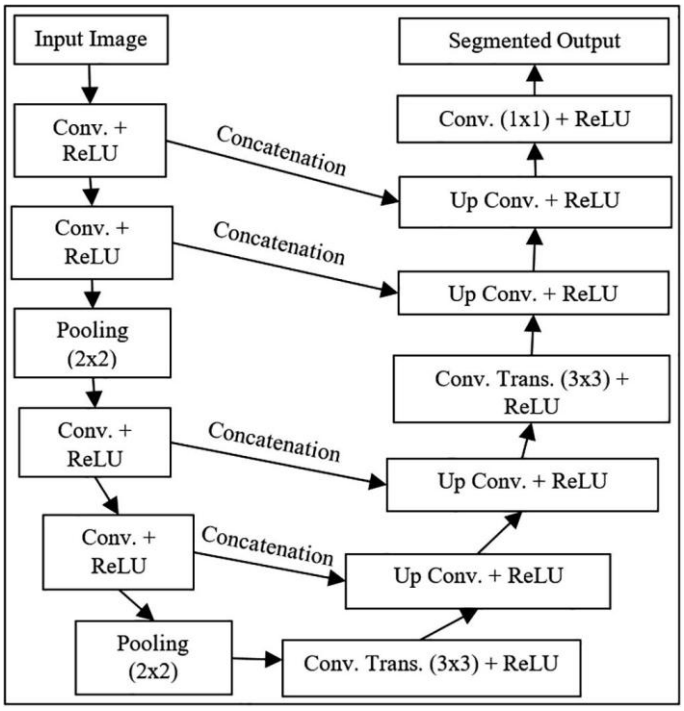


Figure 5.10.2 Modified U-Net Model Architecture by Hossain, M.S. (2022) [62]

The Modified U-Net architecture was developed based on Hossain, M.S. (2022) [62] where Laplacian filter is introduced to enhance the images and images are trained in patches. Quantitative evaluation also follows the work where recall (sensitivity), precision, accuracy, F-measurement, Jaccard index and Dice score are being analysed.

- **Jaccard Index**

  Assesses the intersection between the predicted positive area and the real positive area. A greater Jaccard score signifies improved alignment between the identified abnormality and the actual truth. This is vital as accurate positioning of lesions directly impacts diagnosis and treatment strategies. Jaccard Index Equation:

$$Jaccard\ Index\ = \frac{TP}{TP + FN + FP}$$

- **Dice Score**

  Like Jaccard Index, Dice Score places more importance to accurately predicted positives. It assesses how alike the predicted and actual sets are, rendering it highly responsive to overlap. This is particularly beneficial for segmentation tasks where it is crucial to reduce both over-segmentation (false positives) and under-segmentation (false negatives). Dice Score Equation:

$$Dice\ Score\ = \frac{2TP}{2\ \cdot\ TP + FN + FP}$$

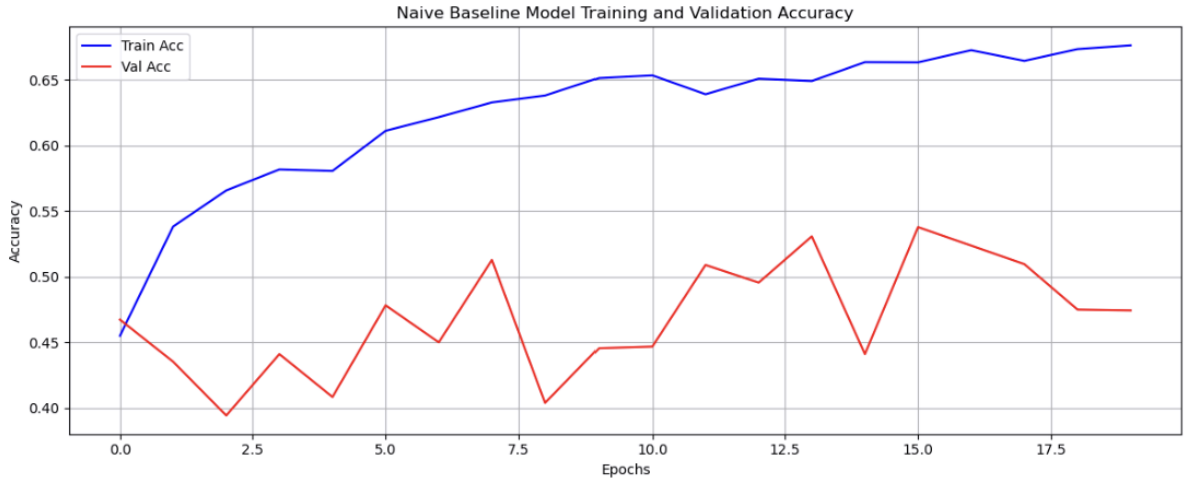
Although somewhat similar, the code and architecture may differ slightly in areas that may not have been explicitly stated by Hossain, M.S. (2022) [62]:

- BatchNormalization and Dropout which are typical for Modified U-Nets are being utilized.
- Final layer Conv2D(1, (1,1), activation='sigmoid') matches Conv.(1x1) + ReLU
- (2×2) Transposed Conv is used for up sampling instead of fixed Up Conv blocks

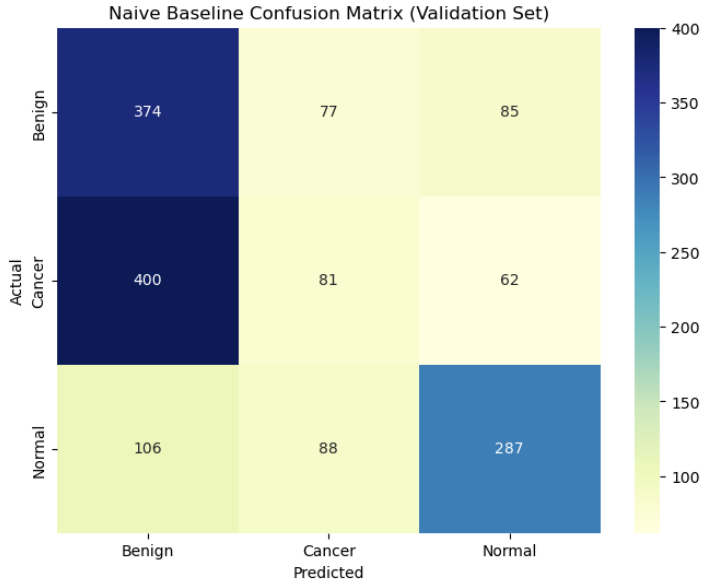
While the implementation may be a slightly modified version, these adjustments are reasonable alterations identified in contemporary literature aimed at enhancing convergence and generalization while maintaining consistent architecture structure.

6. Evaluation

6.1 Naïve Baseline Model Evaluation




```
49/49 ━━━━━━━━━━━━━━━━━━━━ 119s 2s/step
```




```
Naive Baseline Scores
              precision    recall  f1-score   support

      Benign       0.42      0.70      0.53       536
      Cancer       0.33      0.15      0.21       543
      Normal       0.66      0.60      0.63       481

    accuracy                           0.48      1560
   macro avg       0.47      0.48      0.45      1560
weighted avg       0.46      0.48      0.45      1560

Naive Baseline AUC-ROC (macro average): 0.6168780193044541
```

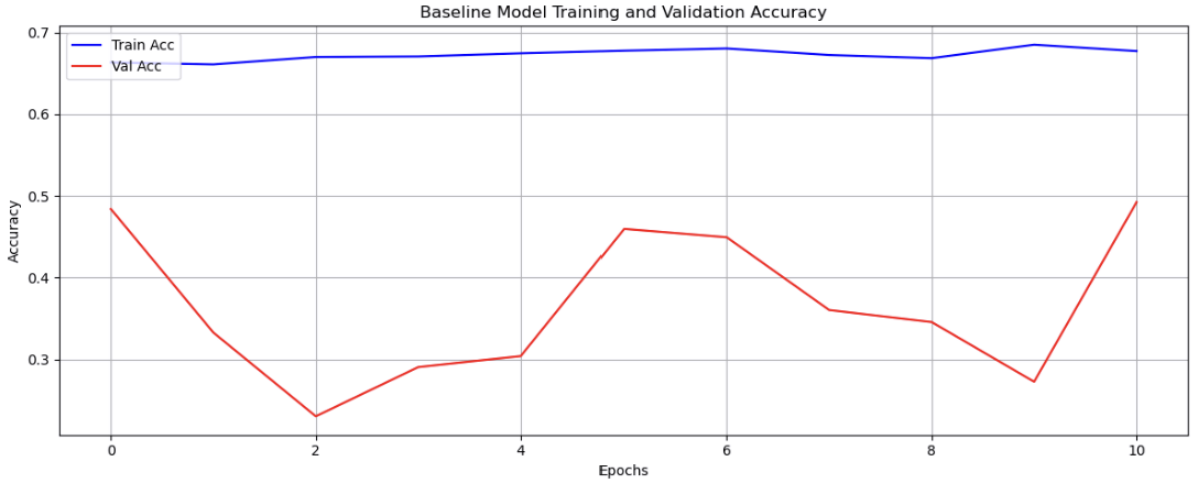
Figure 6.1.1 Naive Baseline Model Results

The training and validation accuracy graphs indicate that the naive baseline model reached a maximum training accuracy of around 65%, while validation accuracy varied between 40% and 54% throughout the epochs. This suggests overfitting and uneven generalization. To tackle class imbalance, weights were used during training, resulting in a more balanced classification. The confusion matrix shows the model could identify all three classes to a certain degree. (See Figure 6.1.1)

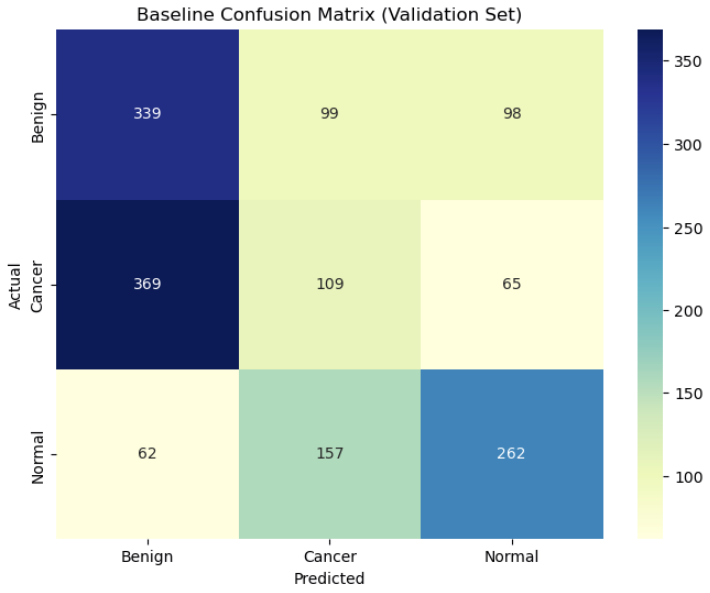
Results show the Cancer class is the weakest, with an F1-score of 0.21 due to elevated misclassification. Overall, the model achieved an accuracy of 48%, a macro-averaged F1-score of 0.45, and a macro AUC-ROC of 0.62, indicating moderate discriminative capability beyond random chance. (See Figure 6.1.1)

Overall, the model faces considerable misclassifications between Benign and Cancer, which are more difficult to distinguish. Consequently, although the naive baseline provides a strong foundation, it still falls short. The next stage involves developing an enhanced baseline model with more sophisticated architectural elements and regularization methods to better capture underlying patterns and reduce overfitting.

6.2 Baseline Model Evaluation




```
49/49 ━━━━━━━━━━━━━━━━━━━━ 123s 2s/step
```




```
Baseline Model Scores
              precision    recall  f1-score   support

      Benign       0.44      0.63      0.52       536
      Cancer       0.30      0.20      0.24       543
      Normal       0.62      0.54      0.58       481

    accuracy                           0.46      1560
   macro avg       0.45      0.46      0.45      1560
weighted avg       0.45      0.46      0.44      1560

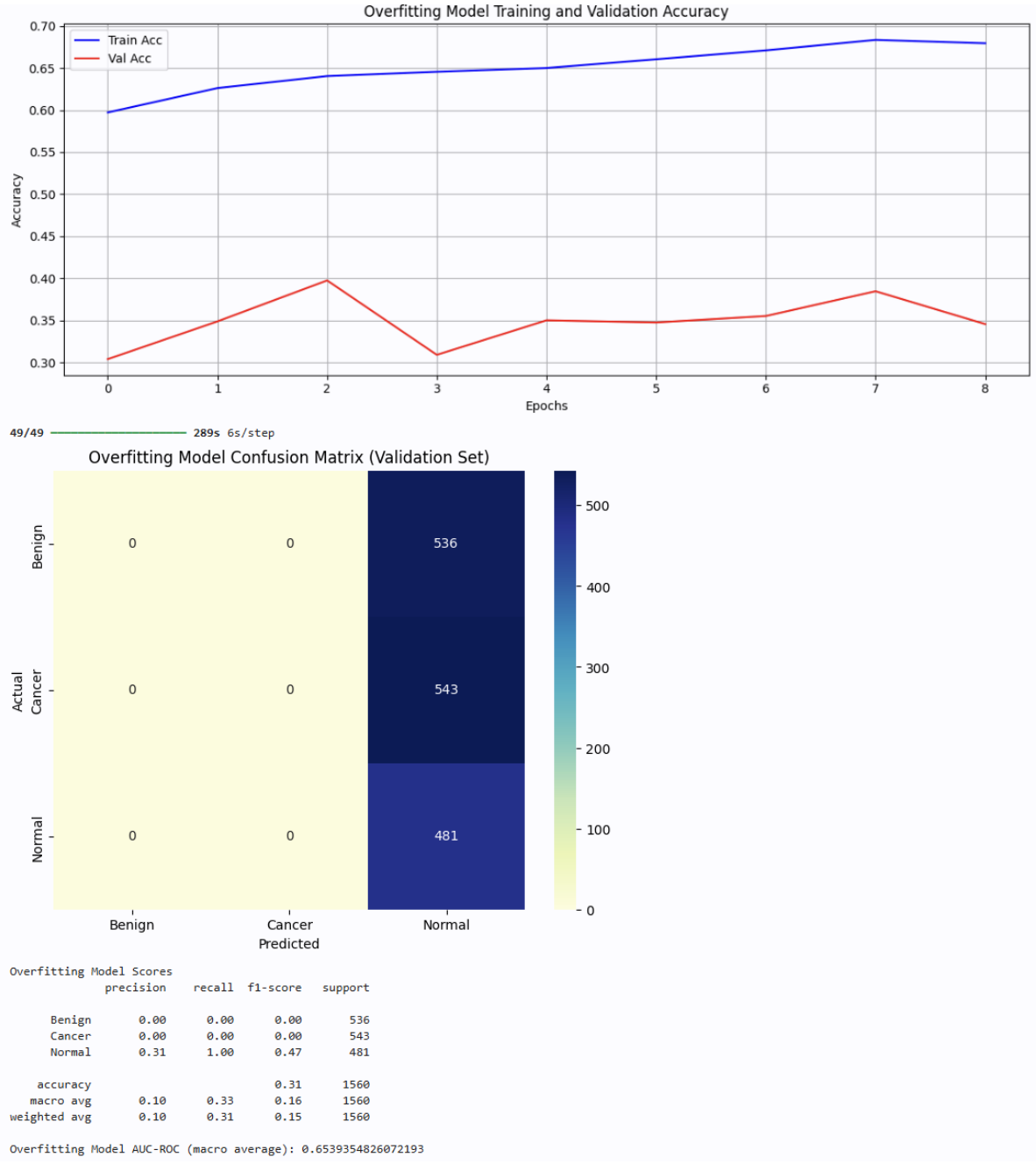
Baseline AUC-ROC (macro average): 0.5815512748174277
```

Figure 6.2.1 Baseline Model Results

The baseline model training and validation accuracy plots illustrate that the model consistently achieved a high and steady training accuracy of approximately 67%, whereas the validation accuracy exhibited considerable fluctuations, varying between 24% and 50%. This significant difference shows that overfitting continues to be a problem, even with modifications to the architecture and early stopping techniques.

The confusion matrix shows that each of the three classes was identified to different degrees. The model recorded its highest recall (0.63) for the Benign class and the top F1-score (0.58) for the Normal class. Nonetheless, it persisted in having difficulty accurately identifying Cancer cases, reaching merely 0.20 recall and 0.24 F1-score. The prevalence of misclassifications between Benign and Cancer highlights the challenge in differentiating between these two visually alike groups. The model recorded an overall accuracy of 46%, a macro-averaged F1-score of 0.45, and a macro AUC-ROC of 0.58, which is somewhat less than the naive baseline's AUC-ROC. These findings indicate that although the model has improved in recognizing benign and normal patterns, its capacity to identify cancer continues to be inadequate. (See Figure 6.2.1)

6.3 Overfitting Model Evaluation




```
Overfitting Model Scores
              precision    recall  f1-score   support

      Benign       0.00      0.00      0.00       536
      Cancer       0.00      0.00      0.00       543
      Normal       0.31      1.00      0.47       481

    accuracy                           0.31      1560
   macro avg       0.10      0.33      0.16      1560
weighted avg       0.10      0.31      0.15      1560

Overfitting Model AUC-ROC (macro average): 0.6539354826072193
```

Figure 6.3.1 Overfitting Model Results

The overfitting model reached a notable training accuracy of around 68%, but validation accuracy remained low, ranging from 31% to 39% throughout. This significant difference suggests extreme overfitting, as the model retains the training data yet does not apply to unfamiliar validation data. (See Figure 6.3.1)

Class weights were implemented, but the model did not utilize them effectively, as indicated by the confusion matrix, which showed that all validation samples were categorized as Normal regardless of their true class. Consequently, both the Benign and Cancer categories achieved zero recall and F1-scores, whereas the Normal category reached a recall of 1.00 and an F1-

score of 0.47, solely because of the classifier's inclination towards dominant predictions. The model attained AUC-ROC macro average 0.65, indicating the model's effectiveness in ranking predictions accurately, even with misclassifications. (See Figure 6.3.1)

These results highlight the limitations of the current configuration, emphasizing the need for stricter regularization and architectural adjustments. The findings reaffirm the importance of addressing overfitting explicitly before progressing to further enhancements.

6.4 Regularized Model Evaluation



Figure 6.4.1 Regularized Model Results

The regularized model accuracy curves for training and validation show a smaller gap, with training accuracy levelling off around 63% and validation accuracy varying between 40% and 55% across epochs, suggesting better generalization than previous models. (See Figure 10.7.3)

The confusion matrix shows that the model successfully differentiates between the Benign and Normal classes, yet it still has difficulty detecting Cancer, where most Cancer cases seem to be incorrectly classified as Benign. The Benign class reached a notable recall of 0.89 and an F1-score of 0.61, whereas the Normal class secured the highest F1-score of 0.73, indicating the model's effectiveness in detecting and differentiating normal images. Nonetheless, the Cancer category achieved a low F1-score of 0.03, emphasizing ongoing challenges in accurately classifying Cancer cases. (See Figure 6.4.1)

6.5 Wider, Deeper, Narrower Models Evaluation

Wider, Deeper, Narrower Models Comparison

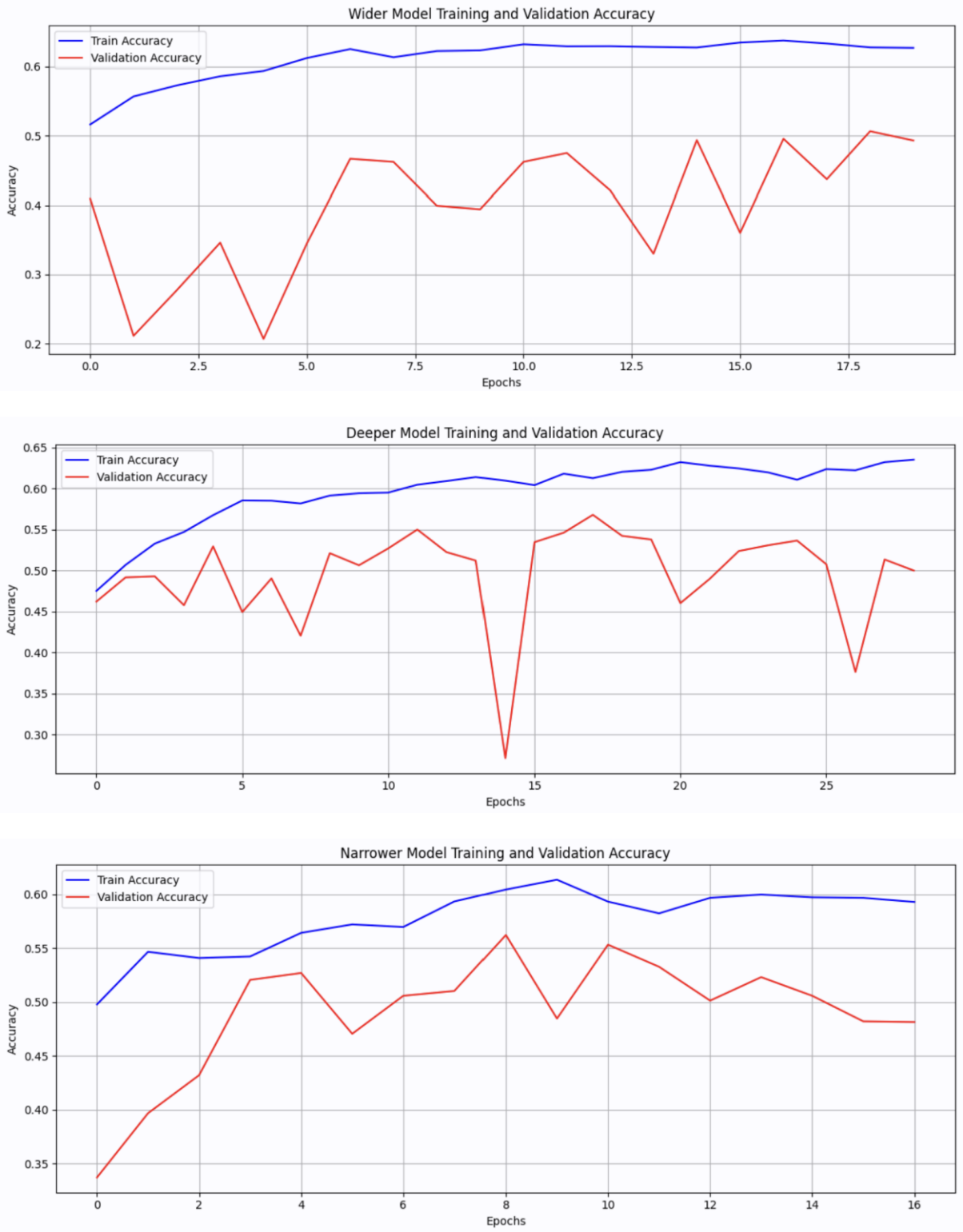


Figure 6.5.1 Wider, Deeper, Narrower Models Accuracy Plot Comparison

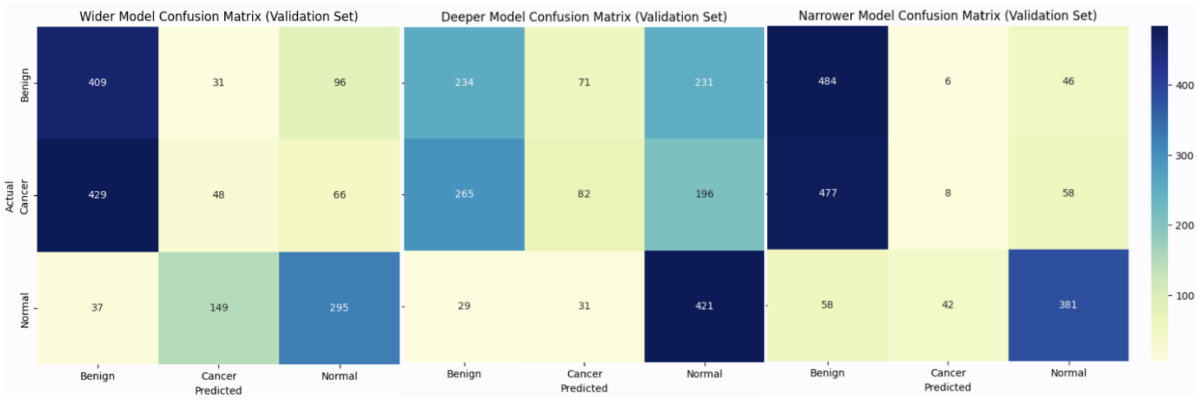


Figure 6.5.2 Wider, Deeper, Narrower Models Confusion Matrix Comparison

Wider Model Scores

	precision	recall	f1-score	support
Benign	0.47	0.76	0.58	536
Cancer	0.21	0.09	0.12	543
Normal	0.65	0.61	0.63	481
accuracy			0.48	1560
macro avg	0.44	0.49	0.44	1560
weighted avg	0.43	0.48	0.44	1560

Wider Model AUC-ROC (macro average): 0.6430

Deeper Model Scores

	precision	recall	f1-score	support
Benign	0.44	0.44	0.44	536
Cancer	0.45	0.15	0.23	543
Normal	0.50	0.88	0.63	481
accuracy			0.47	1560
macro avg	0.46	0.49	0.43	1560
weighted avg	0.46	0.47	0.42	1560

Deeper Model AUC-ROC (macro average): 0.7444

Narrower Model Scores

	precision	recall	f1-score	support
Benign	0.47	0.90	0.62	536
Cancer	0.14	0.01	0.03	543
Normal	0.79	0.79	0.79	481
accuracy			0.56	1560
macro avg	0.47	0.57	0.48	1560
weighted avg	0.46	0.56	0.47	1560

Narrower Model AUC-ROC (macro average): 0.7017

Figure 6.5.3 Wider, Deeper, Narrower Models Scores Summary Comparison

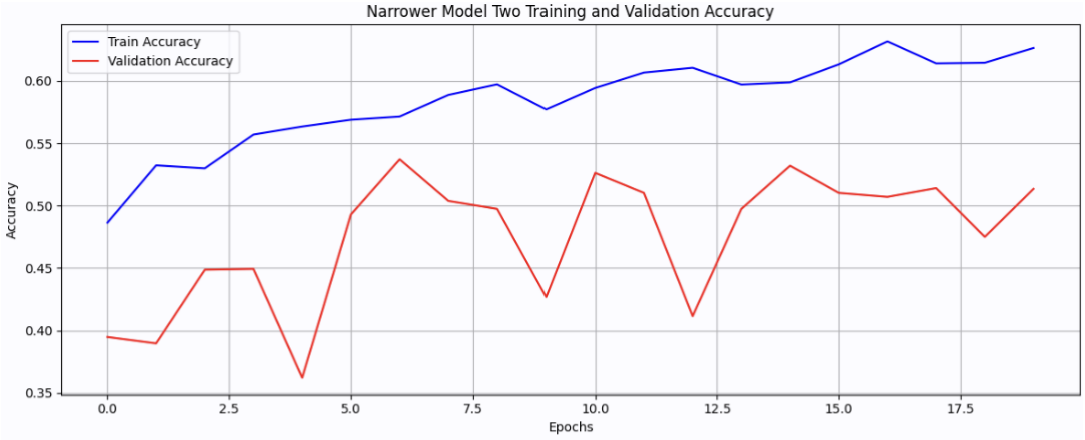
The Deeper model showed a balance between generalization and the feature extraction depth, reaching an accuracy of 47% and a macro AUC-ROC of 0.74, which was the highest of the three. Normal class sensitivity enhanced (recall = 0.88).

The Wider model attained an accuracy of 48% and a macro AUC-ROC of 0.64, indicating better classification of Benign and Normal cases but minimal improvement in Cancer detection (recall = 0.09). Its increased number of parameters enhanced feature variety, but continues to misclassify Cancer cases, possibly due to the image width and height declared.

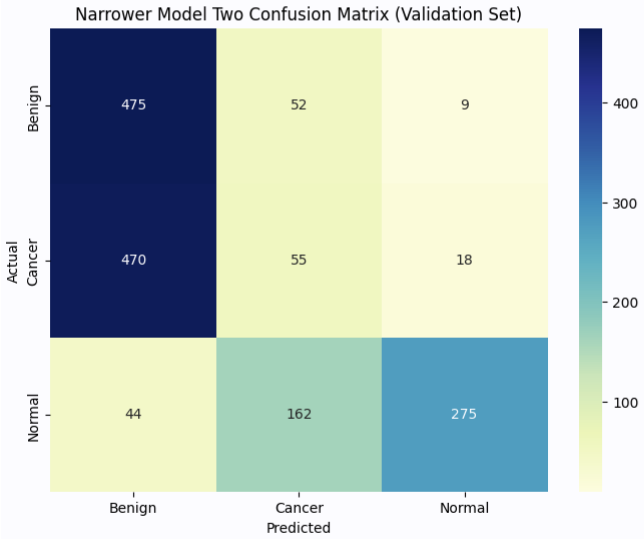
The Narrower model reached the top accuracy of 56% alongside a macro AUC-ROC of 0.70. It demonstrated robust Normal (recall = 0.79) and Benign class performance (recall = 0.90), suggesting efficient class distinction despite a reduced model width.

The greater accuracy of the narrower model indicates that a more streamlined architecture may generalize more effectively on the dataset within the existing limitations; however, all models emphasize the necessity for specific enhancements, especially in cancer detection.

Narrower Model Two Evaluation




```
49/49 ━━━━━━━━━━━━━━━━━━━━ 187s 4s/step
```




```
Narrower Model Two Scores
              precision    recall  f1-score   support

      Benign       0.48      0.89      0.62       536
      Cancer       0.20      0.10      0.14       543
      Normal       0.91      0.57      0.70       481

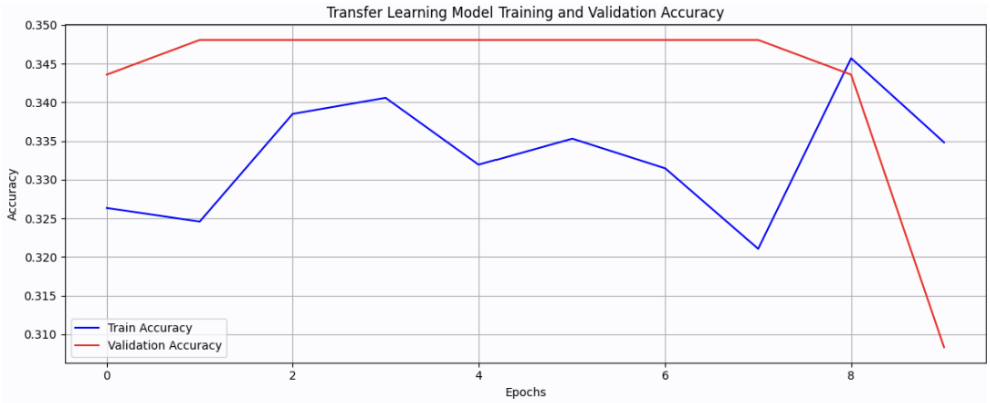
    accuracy                           0.52      1560
   macro avg       0.53      0.52      0.49      1560
weighted avg       0.52      0.52      0.48      1560

Narrower Model Two AUC-ROC (macro average): 0.5960
```

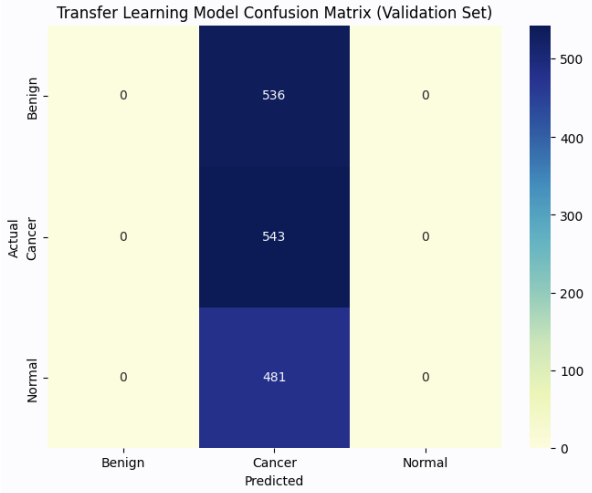
Figure 6.5.4 Narrower Model Two Results

Despite the increased resolution (2560 x 1440), the second narrower model did not yield substantial improvements in detecting Cancer cases. Possible implications might be due to the high-resolution requiring additional and better architecture.

6.6 Transfer Learning Model Evaluation




```
49/49 ━━━━━━━━━━━━━━━━━━━━ 242s 5s/step
```




```
Transfer Learning Model Scores
              precision    recall  f1-score   support

      Benign       0.00      0.00      0.00       536
      Cancer       0.35      1.00      0.52       543
      Normal       0.00      0.00      0.00       481

    accuracy                           0.35      1560
   macro avg       0.12      0.33      0.17      1560
weighted avg       0.12      0.35      0.18      1560

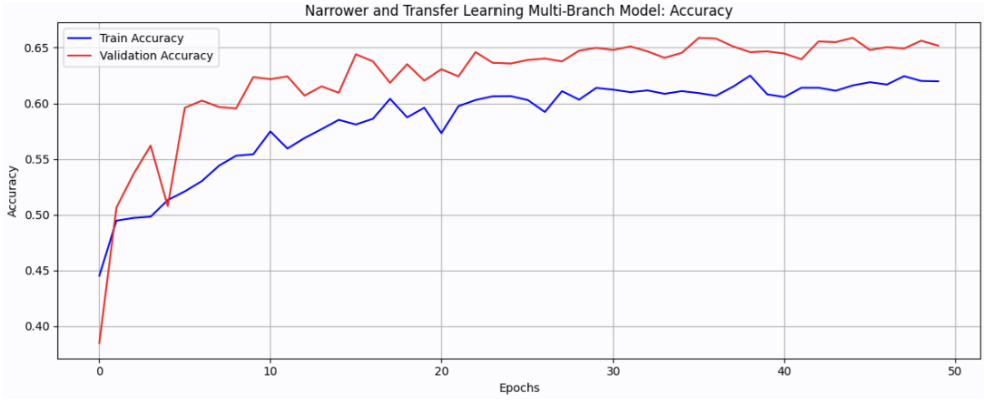
Transfer Learning Model AUC-ROC (macro average): 0.5000
```

Figure 6.6.1 Transfer Learning Model Results

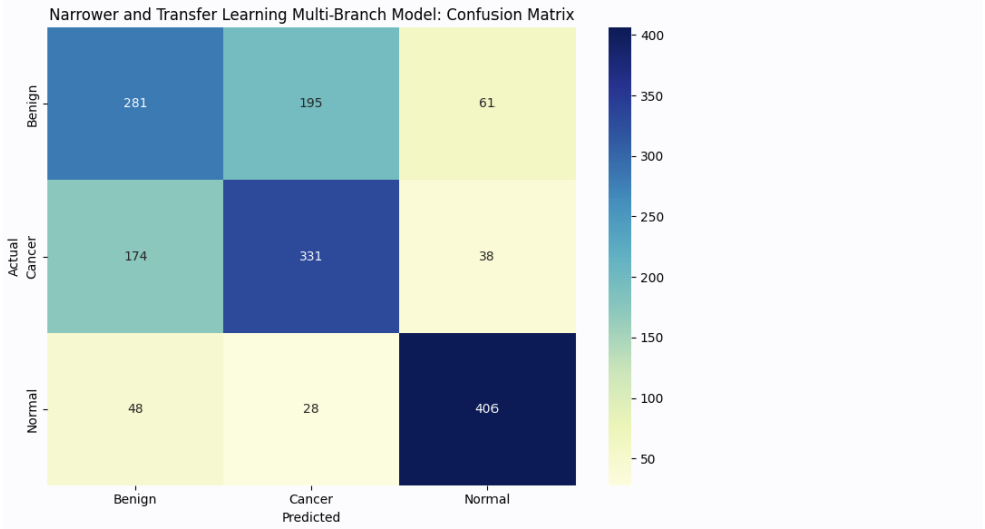
The transfer learning model successfully extracted features to recognize the Cancer class, obtaining perfect recall for this class throughout the validation set. Nonetheless, the model output failures for between Benign and Normal cases, leading to no recall or precision for these categories. The Cancer class, which, although achieved high recall, led to low overall accuracy and a macro AUC-ROC of 0.50, reflecting random performance. Validation accuracy levelled off throughout the epochs, indicating little learning advancement and affirming that the model underperformed in the mammography classification task.

Possible reasons for this result is that utilizing frozen base layers in EfficientNet [53][54] hindered the model's ability to adapt and learn features particular to mammography images, resulting in overly simplified outputs. Moreover, although pretraining on ImageNet [56] provides advantages in feature extraction, the characteristics of the images might not effectively translate from the grayscale specific attributes found in medical imaging tasks without further fine-tuning.

6.7 Narrower and Transfer Learning Multi-Branch Model Evaluation




```
98/98 ━━━━━━━━━━━━━━━━━━━━ 188s 2s/step
```




```
Narrower and Transfer Learning Multi-Branch Model Scores:
              precision    recall  f1-score   support

      Benign       0.56      0.52      0.54       537
      Cancer       0.60      0.61      0.60       543
      Normal       0.80      0.84      0.82       482

    accuracy                           0.65      1562
   macro avg       0.65      0.66      0.66      1562
weighted avg       0.65      0.65      0.65      1562

Narrower and Transfer Learning Multi-Branch Model AUC-ROC (macro average): 0.8563
```

Figure 6.7.1 Narrower and Transfer Learning Multi-Branch Model Results

The multi-branch model showcased a balanced performance across classes, with the Cancer class achieving a recall of 61% and an F1-score of 60%, indicating a significant enhancement compared to earlier models that had difficulties with this class. The Normal class reached a recall of 84% and an F1-score of 82%, whereas the Benign class showed moderate performance, achieving an F1-score of 54%. (See Figure 6.7.1)

This architecture improved the model's sensitivity towards Cancer cases and boosted its overall robustness and class balance, tackling previous limitations found in single-branch and transfer learning methods.

6.8 Modified U-Net Model Evaluation

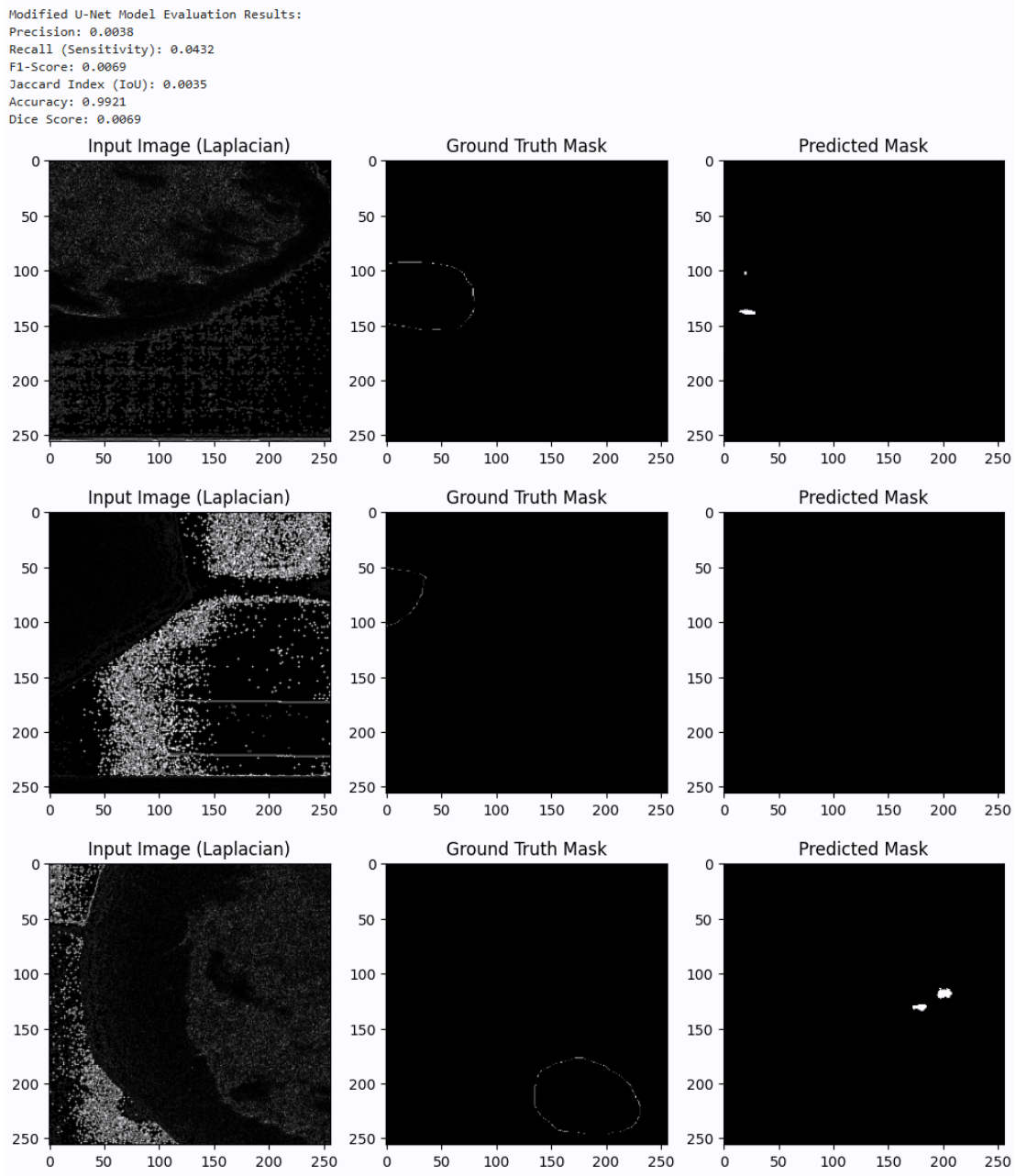


Figure 6.8.1 Modified U-Net Model Results

The Modified U-Net resulted in low Dice and IoU scores, even with high accuracy, indicating the class imbalance issue where lesion pixels represent a minor part of the image. The Laplacian-based preprocessing accentuated high-frequency edges, yet it did not significantly enhance the visibility of lesion areas in every instance. The model often forecasted background because of inadequate lesion representation in training, worsened by the small size of lesions compared to patch dimensions. Future research should investigate advanced data balancing techniques, including focal loss, weighted sampling, or synthetic augmentation, to tackle these issues.

6.9 Model Comparison

Model	Metrics	Precision	recall	f1-score	support	Confusion Matrix Numerical Labels		
Naïve Baseline Model	Benign	0.42	0.70	0.53	536	374	77	85
	Cancer	0.33	0.15	0.21	543	400	81	62
	Normal	0.66	0.60	0.63	481	106	88	287
	accuracy			0.48	1560			
	macro avg	0.47	0.48	0.45	1560			
	weighted avg	0.46	0.48	0.45	1560			
Baseline Model	Benign	0.44	0.63	0.52	536	339	99	98
	Cancer	0.30	0.20	0.24	543	369	109	65
	Normal	0.62	0.54	0.58	481	62	157	262
	accuracy			0.46	1560			
	macro avg	0.45	0.46	0.45	1560			
	weighted avg	0.45	0.46	0.44	1560			
Overfitting Model	Benign	0.00	0.00	0.00	536	0	0	536
	Cancer	0.00	0.00	0.00	543	0	0	543
	Normal	0.31	1.00	0.47	481	0	0	481
	accuracy			0.31	1560			
	macro avg	0.10	0.33	0.16	1560			
	weighted avg	0.10	0.31	0.15	1560			
Regularized Model	Benign	0.46	0.89	0.61	536	478	5	52
	Cancer	0.13	0.02	0.03	543	479	10	54
	Normal	0.76	0.70	0.73	481	85	60	336
	accuracy			0.53	1560			
	macro avg	0.45	0.54	0.46	1560			
	weighted avg	0.44	0.53	0.44	1560			
Wider Model	Benign	0.47	0.76	0.58	536	409	31	96
	Cancer	0.21	0.09	0.12	543	429	48	66
	Normal	0.65	0.61	0.63	481	37	149	295
	accuracy			0.48	1560			
	macro avg	0.44	0.49	0.44	1560			
	weighted avg	0.43	0.48	0.44	1560			
Deeper Model	Benign	0.44	0.44	0.44	536	234	71	231
	Cancer	0.45	0.15	0.23	543	265	82	198
	Normal	0.50	0.88	0.63	481	29	31	421
	accuracy			0.47	1560			
	macro avg	0.46	0.49	0.43	1560			
	weighted avg	0.46	0.47	0.42	1560			
Narrower Model	Benign	0.47	0.90	0.62	536	484	6	46
	Cancer	0.14	0.01	0.03	543	477	8	58
	Normal	0.79	0.79	0.79	481	58	42	381
	accuracy			0.56	1560			
	macro avg	0.47	0.57	0.48	1560			
	weighted avg	0.46	0.56	0.47	1560			
Narrower Model Two	Benign	0.48	0.89	0.62	536	475	52	9
	Cancer	0.20	0.10	0.14	543	470	55	18
	Normal	0.91	0.57	0.70	481	44	162	275
	accuracy			0.52	1560			
	macro avg	0.53	0.52	0.49	1560			
	weighted avg	0.52	0.52	0.48	1560			
Transfer Learning Model	Benign	0.00	0.00	0.00	536	0	536	0
	Cancer	0.35	1.00	0.52	543	0	543	0
	Normal	0.00	0.00	0.00	481	0	481	0
	accuracy			0.35	1560			
	macro avg	0.12	0.33	0.17	1560			
	weighted avg	0.12	0.35	0.18	1560			
Narrower and Transfer Learning Multi-Branch Model	Benign	0.56	0.52	0.54	536	281	195	61
	Cancer	0.60	0.61	0.6	543	174	331	38
	Normal	0.80	0.84	0.82	481	48	28	406
	accuracy			0.65	1560			
	macro avg	0.65	0.66	0.66	1560			
	weighted avg	0.65	0.65	0.65	1560			
Model	Metrics	Score						
Modified U-Net Model	Precision	0.0038						
	recall	0.0432						
	f1-score	0.0069						
	Jaccard Index	0.0035						
	accuracy	0.9921						
	Dice Score	0.0069						

Table 6.9.1 Model Evaluation Results Comparison

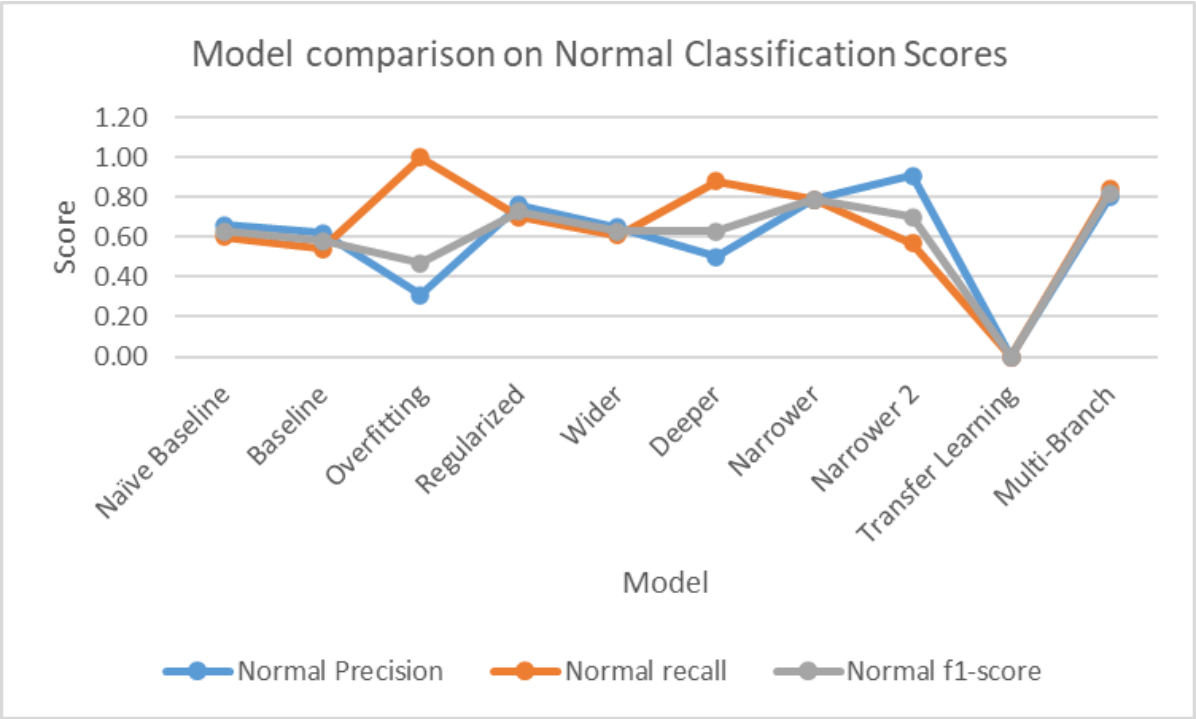


Figure 6.9.1 Model Comparison on Normal Classification Scores Chart

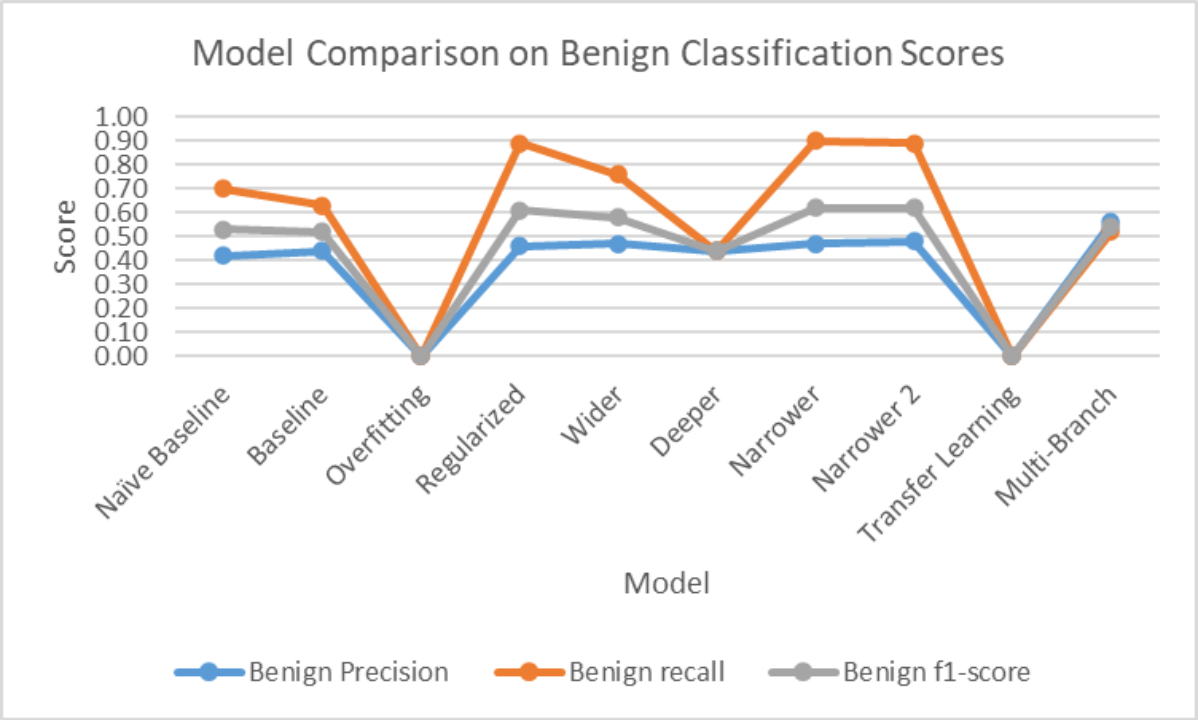


Figure 6.9.2 Model Comparison on Benign Classification Scores Chart

Overall, the models were stable in identifying Normal and Benign cases, least overfitting and transfer learning. (See Figure 6.9.1 and 6.9.2)

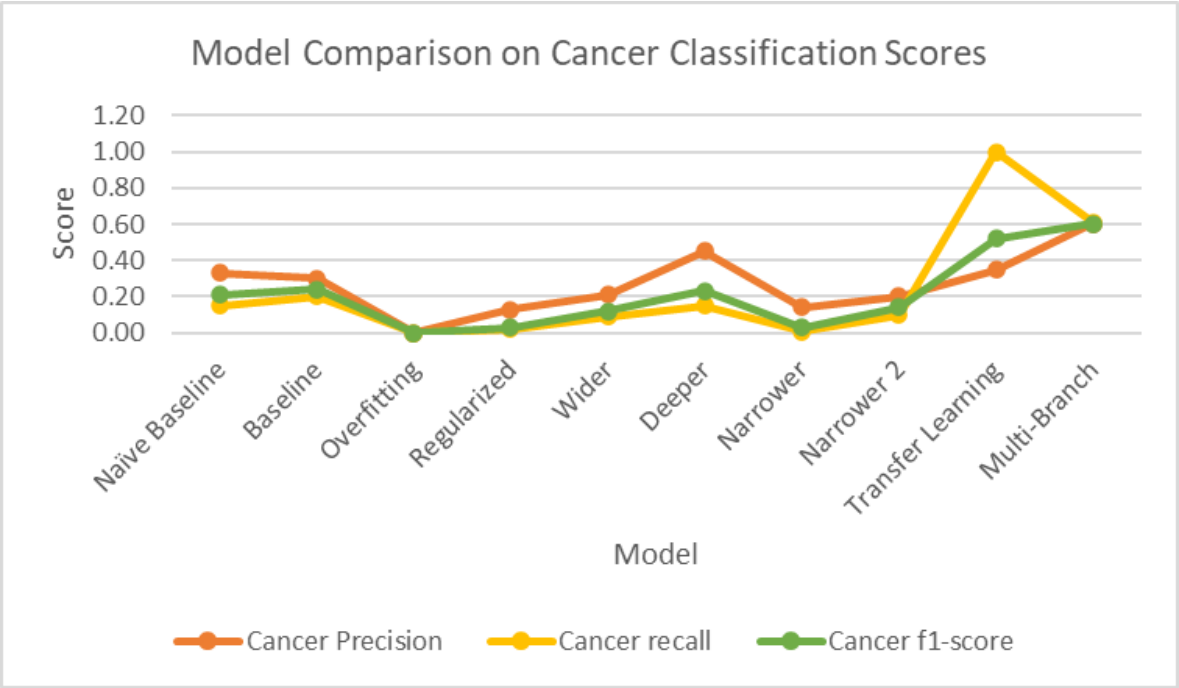


Figure 6.9.3 Model Comparison on Cancer Classification Scores Chart

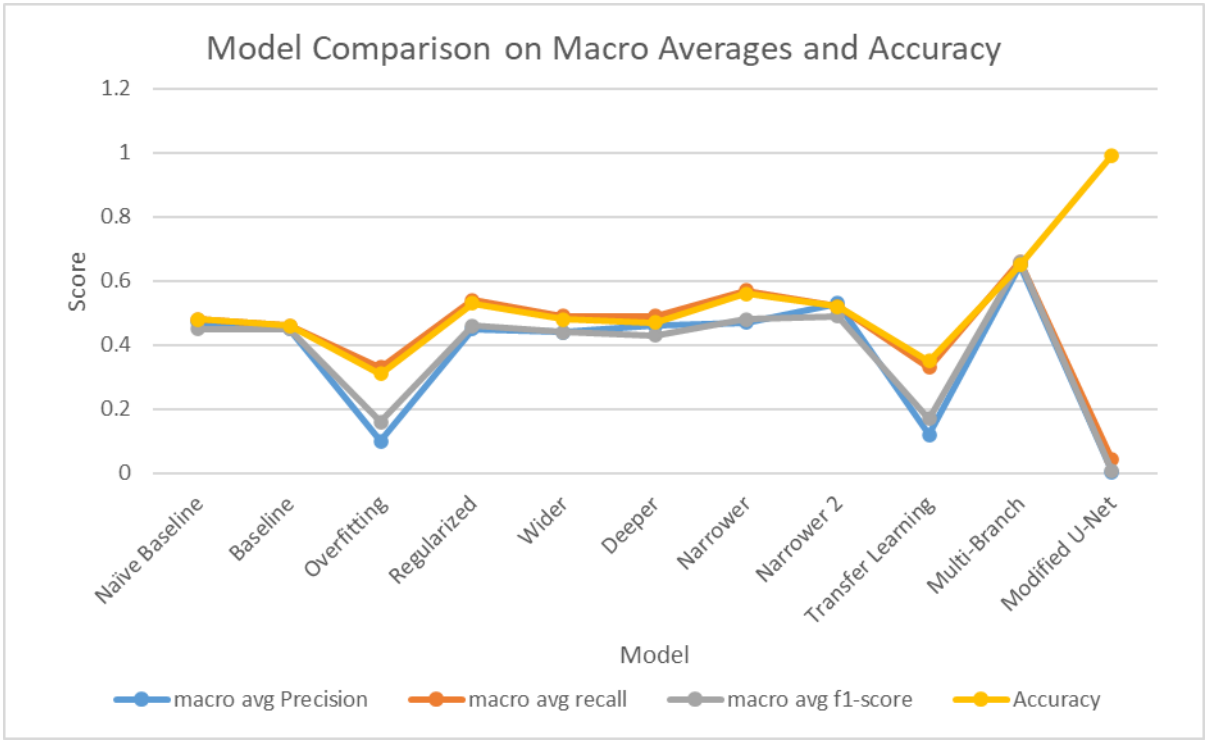


Figure 6.9.4 Model Comparison on Macro Averages and Accuracy Chart

Model comparisons indicated that merely expanding or deepening architectures led to minimal improvements, with accuracy stabilizing around 0.47–0.48 and subpar cancer detection. Regularization slightly enhanced recall but did not affect precision. The narrower models attained greater accuracy (0.56) yet continued to face challenges with cancer class precision. Transfer learning by itself could not generalize, predicting solely the cancer category.

Nevertheless, the multi-branch design that integrated the narrower model and transfer learning model, greatly enhanced performance, attaining 0.65 accuracy along with balanced macro and weighted F1-scores of 0.65, signifying improved management of class imbalance and feature extraction.

The Modified U-Net attained excellent per-pixel accuracy yet exhibited very low Dice and IoU scores. The main reason for the low scores is due toa lack of segmentation procedures prior to model training. This emphasizes difficulties in segmenting tumour boundaries even with background predominance in mammograms.

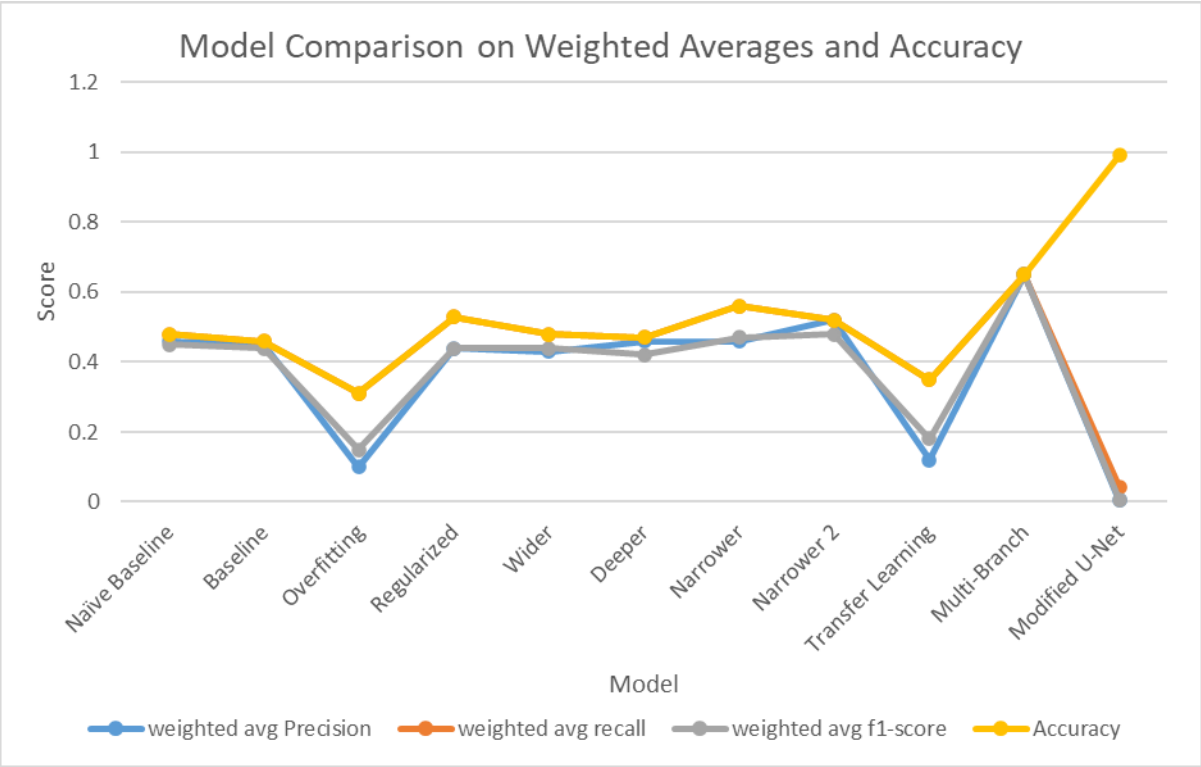


Figure 6.9.5 Model Comparison on Weighted Averages and Accuracy Chart

These findings collectively highlight that although changes in architecture and regularization provide small enhancements, tackling the fundamental issues of data imbalance and feature representation presents the most advantageous approach for enhancing mammogram classification and segmentation for the Mini-DDSM dataset. (See Table 6.9.1)

7. Conclusion

7.1 Implications

One of the biggest limitations by far is the large time consumption training the models due to lack of computational resources and high-resolution mammography images. Even though the images have been shrunk to 224×224 pixels and early stopping as well as checkpoint are being integrated, the time taken to train a model is still substantial.

While the Modified U-Net model was implemented according to Hossain, M.S. (2022) [62], the output results show huge discrepancy compared to the article. Although there may be some differences in architectural approach, parameters, and computational environment, the implemented model concept remains similar. Main possible reason for the output disparity is largely due to the absence of the Skin and air boundary segmentation, Pectoral region segmentation, Suspicious regions selection, and manual observation to mark positive and negative patches after the automatic selection of suspicious and non-suspicious regions. Solutions would involve thresholding, Hough Transform, and MSER (Maximally Stable Extremal Regions).

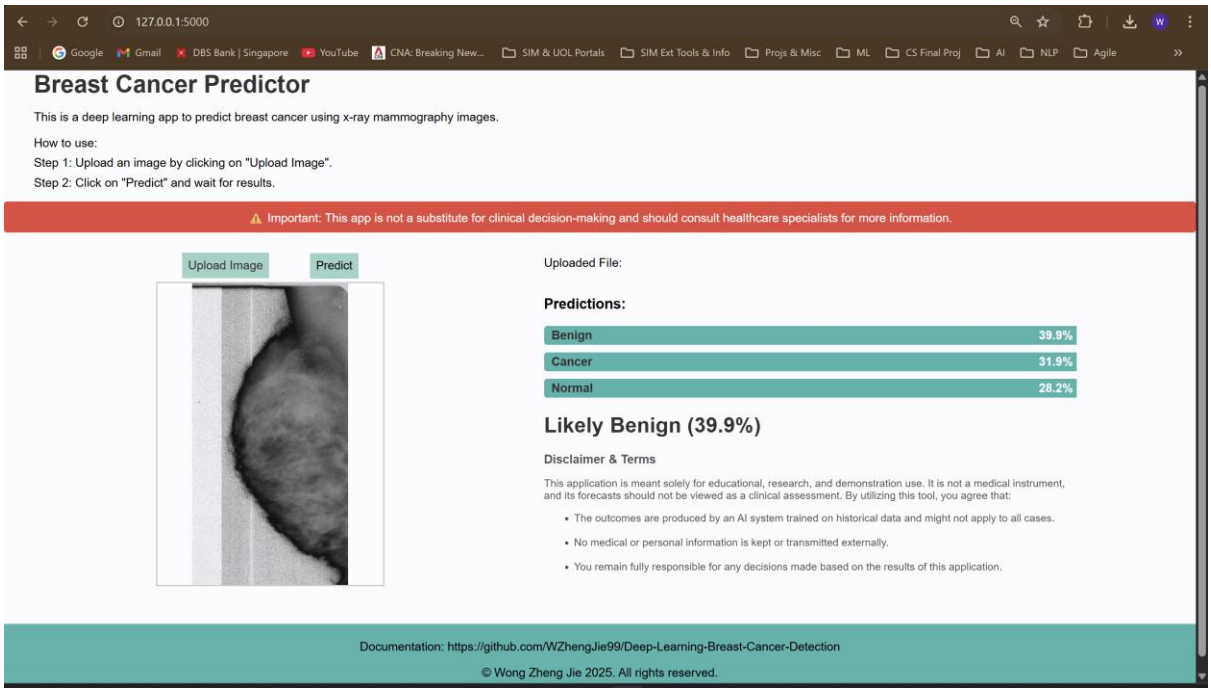


Figure 7.1.1 UI App localhost Implementation

A simple UI was created using the narrower_and_transfer_learning_model.keras checkpoint in python running locally using Flask [63] with HTML and CSS for styling and presentation. The application starts up via a batch file that checks for dependencies for the python script before execution. (See Figure 7.1.1 and 16. Appendix E – User Interface for further details)

Project overall under delivers model results due to Cancer misclassification but highlights directions for improvements such as hybrid models in multi class classification for mammography images and segmentation importance for micro calcification detection. Solutions moving forward would involve heavy segmentation procedures in the hybrid models and especially for identification between Benign and Cancer cases.

Current code implementation uses downloaded Mini-DDSM files locally and may cause strains for computers with memory and computational challenges. Although GitHub allows Large File Storage (LFS) [54], it is not optimal for large datasets to be uploaded as part of the repository especially since they are accessible via its origin, Kaggle [3][4]. While not of current concern, future developments could include initializing the Mini-DDSM dataset from its Kaggle source for data manipulation to minimize or eliminate dataset related local storage.

7.2 Future Directions

While future directions includes improvements that build upon and strengthen the works of this project, it also includes areas that can assist radiologists in hopes of progressing to gather feed back from the industry and eventually clinical trials.

Firstly, future research could investigate the use of more tailored models for each class within the multi-branch framework, aiming to enhance overall classification precision. For instance, lighter models like MobileNet [59] or EfficientNet-Lite [64] could be ideal for the Benign class, since these architectures efficiently capture texture and shape details without incurring too much computational cost. Conversely, intricate models like DenseNet [65], EfficientNet [53][54], or transformer-based structures such as the Swin Transformer [66] may be utilized for the Cancer class, where identifying subtle and detailed features is essential. Likewise, the Normal class could gain from encoders trained through self-supervised learning methods, which can grasp overall breast structures and contextual details. Customizing architectures to fit the distinct features of each class would enable the branches to specialize better, and their combined outputs might improve the overall robustness and precision of the multi-branch model.

Secondly, advanced segmentation methods should be applied and examined particularly for the Cancer class and its related masks to enhance the detection of cancer calcifications. Techniques like pyramid pooling [67] or Feature Pyramid Networks (FPNs) [68] can be employed to detect calcifications at various scales, due to their considerable diversity in size and appearance. Additionally, frameworks like Attention U-Net [69], Swin-UNet [70], or nnU-Net [71] could be evaluated for their proven effectiveness in medical image segmentation. These methods would enable the model to more clearly define suspicious areas of interest, thereby increasing interpretability and boosting the dependability of cancer detection in clinical environments.

Thirdly, future works can aim to emphasize the locations of cancer when it is identified. This can be accomplished by integrating weakly supervised localization or object detection methods that can generate interpretable visual results. For example, object detection models like YOLOv8 [72] or Faster R-CNN [73] could be trained to identify and locate suspicious lesions,

generating bounding boxes and heatmaps that highlight cancer predictions. Alternatively, techniques for explainability like Grad-CAM [74] or Score-CAM [75] can be utilized to emphasize the important regions of mammograms that affect the model's choices. Integrating visual indicators with classification results would enhance confidence in the system and assist in connecting automated predictions with medical decision making.

Finally, the framework can be aimed to forecast not just the existence of cancer but also address its stage and related biomarkers. This can be accomplished via multi-task learning [76], utilizing distinct output heads for training in both binary cancer detection and stage classification. This method would offer more clinically relevant insights, advancing from mere detection to prognostic evaluations. Additionally, employing explainable AI models may help pinpoint the particular image characteristics like calcification density, spiculated edges, or abnormal growth patterns that relate to cancer stages or established biomarkers.

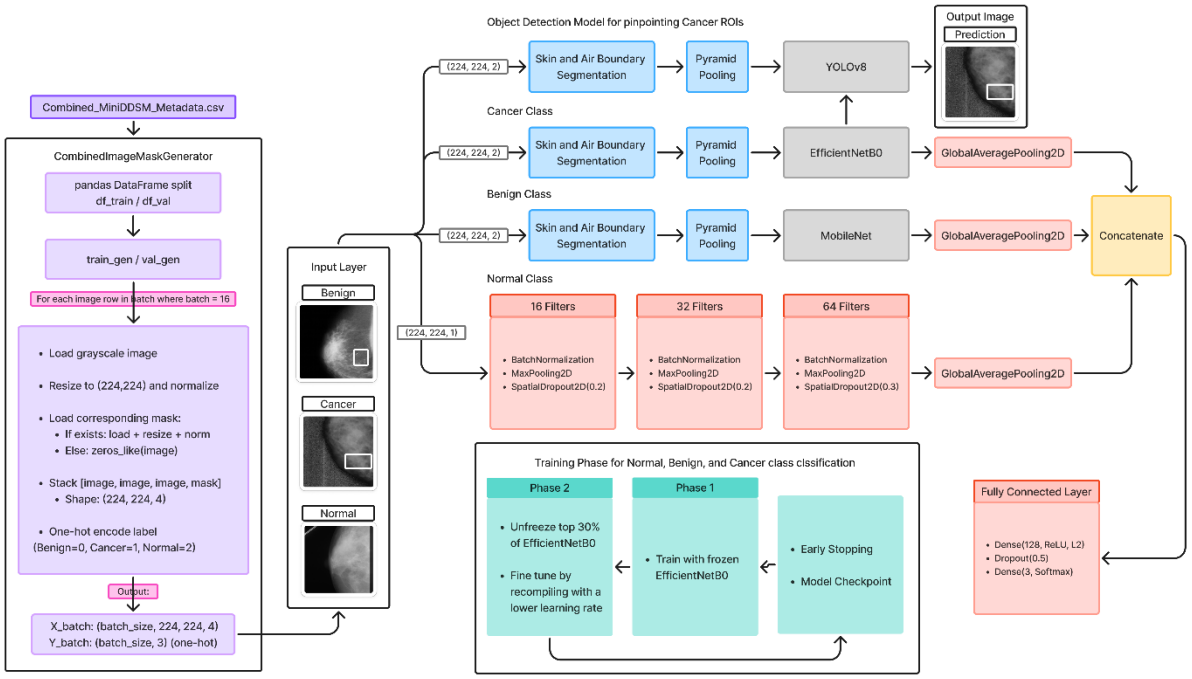


Figure 7.2.1 4-Branch Model Architecture for Detecting Cancer Presence and ouput Predicted Cancer ROIs

All in all, despite the appearance of multiple challenges, including overfitting, class imbalance, and complications in accurately identifying cancerous instances, the existing outcomes, especially the Narrower and Transfer Learning Multi-Branch Model, shows steady improvements. These future research directions suggested can enhance and expand upon current metric scores, offering a more comprehensive perspective on cancer diagnosis and progression. (See Figure 7.2.1)

8. Artefacts

The artefacts listed below include the developments and results of this project and serve as a basis for evaluation, reproduction, and future advancements. These artifacts are made available to and include the implementation required to reproduce the results outlined in this report.

GitHub Project Overview: View Project Overview

GitHub Repository: View Code Repository

Dataset – Mini-DDSM [3]: Kaggle Dataset

These artifacts together enhance the project's transparency, reproducibility, and accessibility for academic, research, and educational objectives.

9. Authors' Declarations

9.1 Conflict of Interest

The author declares that there are no commercial or financial relationships that could be interpreted as a potential conflict of interest in this research.

9.2 Consent

This study makes use of publicly available secondary data (Mini-DDSM dataset [3]) and does not involve any direct interaction with human participants or animals by the author.

9.3 Authors' Notes

Given the significant rapidly expanding and evolving volume of related literature, some pertinent studies may not have been cited. Nonetheless, attempts were undertaken to reference several publications and ensure an academic foundation for the project.

The author also wishes to note deadlines set, unforeseen circumstances, and time constraints that may have resulted in a lack and or lapse of certain actions, outcomes, and insights that could have potentially contributed towards the project.

Correspondence: Wong Zheng Jie, wzhengjie99@gmail.com; zjwong004@mymail.sim.edu.sg

10. References

[1] NHS Digital (2025) AI knowledge repository. [Online] Available at: https://digital.nhs.uk/services/ai-knowledge-repository (Accessed: 2025).

[2] UK Government (2025) World-leading AI trial to tackle breast cancer launched. [Online] Available at: https://www.gov.uk/government/news/world-leading-ai-trial-to-tackle-breast-cancer-launched (Accessed: 2025).

[3] Cheddad, A. (2021) The Complete Mini-DDSM [Data set]. Kaggle. [Online] Available at: https://www.kaggle.com/datasets/cheddad/miniddsm2 (Accessed: 2025).

[4] Lekamlage, C.D., Afzal, F., Westerberg, E. and Cheddad, A. (2020) 'Mini-DDSM: Mammography-based Automatic Age Estimation', in Proceedings of the 3rd International Conference on Digital Medicine and Image Processing (DMIP 2020), Kyoto, Japan, 6–9 November. New York: ACM. [Online] Available at: https://arxiv.org/pdf/2010.00494v1 (Accessed: 2025).

[5] Heath, M., Bowyer, K., Kopans, D., Moore, R. and Kegelmeyer, W.P. (2001) 'The Digital Database for Screening Mammography', in Yaffe, M.J. (ed.) Proceedings of the Fifth International Workshop on Digital Mammography. Madison, WI: Medical Physics Publishing, pp. 212–218. [Online] Available at: http://www.eng.usf.edu/cvprg/Mammography/software/HeathEtAlIWDM_2000.pdf (Accessed: 2025).

[6] Wang, L. (2024) 'Mammography with deep learning for breast cancer detection', Frontiers in Oncology, 14, 1281922. doi: 10.3389/fonc.2024.1281922. [Online] Available at: https://www.frontiersin.org/journals/oncology/articles/10.3389/fonc.2024.1281922/full (Accessed: 2025).

[7] Sawyer-Lee, R., Gimenez, F., Hoogi, A., & Rubin, D. (2016). Curated Breast Imaging Subset of Digital Database for Screening Mammography (CBIS-DDSM) [Data set]. The Cancer Imaging Archive. doi: 10.7937/K9/TCIA.2016.7O02S9CY. Available at: https://doi.org/10.7937/K9/TCIA.2016.7O02S9CY (Accessed: 2025).

[8] World Health Organization (WHO) and International Agency for Research on Cancer (IARC) (2024) Global cancer burden growing, amidst mounting need for services. Lyon, France; Geneva, Switzerland: WHO. [Online] Available at: https://www.who.int/news/item/01-02-2024-global-cancer-burden-growing--amidst-mounting-need-for-services (Accessed: 2025).

[9] OHSU Knight Cancer Institute (2025) Understanding breast cancer. Oregon Health & Science University. [Online] Available at: https://www.ohsu.edu/knight-cancer-institute/understanding-breast-cancer (Accessed: 2025).

[10] DePolo, J. and Fox, K. (2025) Understanding your pathology report – breast cancer stages. Breastcancer.org. [Online] Available at: https://www.breastcancer.org/pathology-report/breast-cancer-stages (Accessed: 2025).

[11] Malhotra, P., El-Ashry, D., Rubio, M., Schlamadinger, D. and Sile, E. (2025) Mammograms: everything you need to know. Breast Cancer Research Foundation. [Online] Available at: https://www.bcrf.org/about-breast-cancer/mammogram-breast-cancer-screening/ (Accessed: 2025).

[12] Cancer Research UK (2023) Getting diagnosed with breast cancer – breast screening. [Online] Available at: https://www.cancerresearchuk.org/about-cancer/breast-cancer/getting-diagnosed/screening-breast (Accessed: 2025).

[13] Cerdas, M.G., Farhat, J., Elshafie, S.I., Mariyam, F., James, L., Qureshi, A.K., Potru, M., Paliwei, P., Joshi, M.R., Abraham, G. and Siddiqui, H.F. (2025) 'Exploring the evolution of breast cancer imaging: a review of conventional and emerging modalities', Cureus. doi: 10.7759/cureus.82762. [Online] Available at: https://acsjournals.onlinelibrary.wiley.com/doi/full/10.3322/caac.21863 (Accessed: 2025).

[14] Srivastava, A., Ahmad, R., Yadav, K., Siddiqui, S., Trivedi, A., Misra, A., Mehrotra, S., Ahmad, B.A., and Khan, M.A. (2024) 'An update on existing therapeutic options and status of novel anti-metastatic agents in breast cancer: elucidating the molecular mechanisms underlying the pleiotropic action of Withania somnifera (Indian ginseng) in breast cancer attenuation', International Immunopharmacology, 136, 112232. doi: 10.1016/j.intimp.2024.112232. [Online] Available at: https://onlinelibrary.wiley.com/doi/full/10.1155/2024/9003572 (Accessed: 2025).

[15] UK Government (2024) Independent investigation of the NHS in England: Lord Darzi's report. Department of Health and Social Care. [Online] Available at: https://www.gov.uk/government/publications/independent-investigation-of-the-nhs-in-england (Accessed: 2025).

[16] zohyan (2019) François Chollet's Deep Learning with Python: The universal workflow of machine learning. GitHub. [Online] Available at: https://github.com/zohyan/Deep-Learning-with-python/blob/master/4_5_The_universal_workflow_of_machine_learning.ipynb (Accessed: 2025).

[17] Chollet, F. (2017) Deep learning with Python. Shelter Island, NY: Manning Publications. ISBN: 9781617294433. [Online] Available at: https://www.manning.com/books/deep-learning-with-python (Accessed: 2025).

[18] Figma (n.d.) Figma – collaborative interface design tool. Available at: https://www.figma.com (Accessed: 2025).

[19] GitHub (n.d.) GitHub: Hosting service for software projects using Git – Main Page. [Online] Available at: https://github.com/ (Accessed: 2025).

[20] GitHub, Inc. (2025) GitHub Docs – Projects: About Projects. [Online] Available at: https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects (Accessed: 2025).

[21] Beremauro, S. and Girio-Fragkoulakis, C. (2022) 'Imaging techniques in breast cancer', Surgery (Oxford), 40(2), pp. 94–103. doi: https://doi.org/10.1016/j.mpsur.2021.11.014. Available at: https://www.sciencedirect.com/science/article/pii/S0263931921002593 (Accessed: 2025).

[22] Santeramo, R., Damiani, C., Wei, J. et al. (2024) 'Are better AI algorithms for breast cancer detection also better at predicting risk? A paired case–control study', Breast Cancer Research, 26, 25. doi: https://doi.org/10.1186/s13058-024-01775-z. Available at: https://breast-cancer-research.biomedcentral.com/articles/10.1186/s13058-024-01775-z (Accessed: 2025).

[23] Wu, N. et al. (2020) 'Deep Neural Networks Improve Radiologists' Performance in Breast Cancer Screening', IEEE Transactions on Medical Imaging, 39(4), pp. 1184–1194. doi: https://doi.org/10.1109/TMI.2019.2945514. Available at: https://ieeexplore.ieee.org/abstract/document/8861376 (Accessed: 2025).

[24] Yala, A., Mikhael, P.G., Strand, F., Lin, G., Smith, K., Wan, Y.-L., Lamb, L., Hughes, K., Lehman, C. and Barzilay, R. (2021) 'Toward robust mammography-based models for breast cancer risk', Science Translational Medicine, 13(578), eaba4373. doi: https://doi.org/10.1126/scitranslmed.aba4373. Available at: https://www.science.org/doi/abs/10.1126/scitranslmed.aba4373 (Accessed: 2025).

[25] Morales, J. (2024) 'AI X-ray Scans Help NHS Doctors Reduce Missed Fractures for Just £1', CCN. Published 22 Oct 2024. Available at: https://www.ccn.com/news/technology/ai-x-ray-scans-to-help-nhs-doctors-reduce-missed-fractures/ (Accessed: 2025).

[26] National Institute for Health and Care Excellence (NICE) (2024) Evidence generation plan - Artificial intelligence technologies to help detect fractures on X-rays in urgent care. Available at: https://www.nice.org.uk/guidance/hte20/documents/accessible-version-2 (Accessed: 2025).

[27] National Institute for Health and Care Excellence (NICE) (2025) Health technology evaluation - Artificial intelligence technologies to help detect fractures on X-rays in urgent care: early value assessment. Published: 14 January 2025. Available at: https://www.nice.org.uk/guidance/hte20/resources/artificial-intelligence-technologies-to-help-detect-fractures-on-xrays-in-urgent-care-early-value-assessment-pdf-50262025025221 (Accessed: 2025).

[28] Kumar, Y., Shrivastav, S., Garg, K. et al. (2024) 'Automating cancer diagnosis using advanced deep learning techniques for multi-cancer image classification', Scientific Reports, 14, 25006. doi: https://doi.org/10.1038/s41598-024-75876-2 (Accessed: 2025).

[29] Kim, C., Shin, S. and Jeong, S. (2023) 'Convolution Neural Network Approaches for Cancer Cell Image Classification', Biotechnology and Bioprocess Engineering, 28(5), pp. 707–719. doi: https://doi.org/10.1007/s12257-023-0164-7 (Accessed: 2025).

[30] Albawi, S., Mohammed, T.A. and Al-Zawi, S. (2017) 'Understanding of a convolutional neural network', 2017 International Conference on Engineering and Technology (ICET), Antalya, Turkey, pp. 1–6. doi: 10.1109/ICEngTechnol.2017.8308186. Available at: https://ieeexplore.ieee.org/document/8308186 (Accessed: 2025).

[31] Pan, S.J. and Yang, Q. (2010) 'A Survey on Transfer Learning', IEEE Transactions on Knowledge and Data Engineering, 22(10), pp. 1345–1359. doi: https://doi.org/10.1109/TKDE.2009.191. Available at: https://ieeexplore.ieee.org/document/5288526 (Accessed: 2025).

[32] Hospedales, T., Antoniou, A., Micaelli, P. and Storkey, A. (2021) 'Meta-learning in neural networks: A survey', IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(9), pp. 5149–5169. doi: https://doi.org/10.48550/arXiv.2004.05439 (Accessed: 2025).

[33] Triantafillou, E. et al. (2021) 'Learning a universal template for few-shot dataset generalization', International Conference on Machine Learning (ICML), PMLR. doi: https://doi.org/10.48550/arXiv.2105.07029 (Accessed: 2025).

[34] Archana, R. and Jeevaraj, P.S.E. (2024) 'Deep learning models for digital image processing: A review', Artificial Intelligence Review, 57(1), p.11. doi: https://doi.org/10.1007/s10462-023-10631-z (Accessed: 2025).

[35] Ahmad, S. et al. (2022) 'A novel hybrid deep learning model for metastatic cancer detection', Computational Intelligence and Neuroscience, Article ID 8141530. doi: https://doi.org/10.1155/2022/8141530 (Accessed: 2025).

[36] Reddit user (Excellent_Attempt353) 'Balancing Under-fitting, Overfitting, and Robustness', *reddit.com*, posted on r/learnmachinelearning. Available at: https://www.reddit.com/r/learnmachinelearning/comments/1fwieep/evaluating_machine_learning_models_balancing/ (Accessed: 2025).

[37] Myllyaho, L., Raatikainen, M., Männistö, T., Mikkonen, T. & Nurminen, J.K. (2021). *Systematic literature review of validation methods for AI systems*. arXiv preprint arXiv:2107.12190 [cs.SE]. Submitted 26 July 2021. Available at: https://arxiv.org/abs/2107.12190 (Accessed: 2025).

[38] Bishop, C.M. and Nasrabadi, N.M., 2006. Pattern recognition and machine learning (Vol. 4, No. 4, p. 738). New York: springer. Available at: https://link.springer.com/9780387310732 (Accessed: 2025).

[39] Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. In: Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence (IJCAI-95), Montreal, Canada, 20–25 August. Morgan Kaufmann, pp. 1137–1145. Available at: https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf (Accessed: 2025)

[40] Philip, A. R. (2024). Optimizing Food101 Classification with Transfer Learning: A Fine-Tuning Approach Using EfficientNetB0. International Journal of Intelligent Information Systems, 13(4), pp. 59–77. doi: 10.11648/j.ijiis.20241304.11. Available at: https://www.sciencepublishinggroup.com/article/10.11648/j.ijiis.20241304.11 (Accessed: 2025).

[41] Mader, K. (2018). Food 101 datasets. Kaggle. [Online] Available at: https://www.kaggle.com/datasets/kmader/food41 (Accessed: 2025).

[42] Zhou, L., Wang, M. and Zhou, N., 2024. Distributed federated learning-based deep learning model for privacy mri brain tumor detection. arXiv preprint arXiv:2404.10026. Available at: https://arxiv.org/abs/2404.10026 (Accessed: 2025).

[43] Alam, I.N., Kartowisastro, I.H. & Wicaksono, P. (2022). Transfer Learning Technique with EfficientNet for Facial Expression Recognition System. Revue d'Intelligence

Artificielle, 36(4), pp. 543–552. doi:10.18280/ria.360405. Available at: https://www.iieta.org/journals/ria/paper/10.18280/ria.360405 (Accessed: 2025).

[44] Lucey, P., Cohn, J.F., Kanade, T., Saragih, J., Ambadar, Z., Matthews, L. (2010). The extended cohn-kanade dataset (CK+): A complete dataset for action unit and expression-specified expression. 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition - Workshops, San Francisco, CA, USA. Available at: https://doi.org/10.1109/CVPRW.2010.5543262 (Accessed: 2025).

[45] Wang, P.S.P. (2008). Performance comparisons of facial expression recognition in jaffe database. International Journal of Pattern Recognition and Artificial Intelligence, 22(3): 445-459. https://doi.org/10.1142/S0218001408006284 (Accessed: 2025).

[46] Liu, S., Wang, L. & Yue, W. (2024). An efficient medical image classification network based on multi-branch CNN, token grouping Transformer and mixer MLP. Applied Soft Computing, 153, Article 111323. doi:10.1016/j.asoc.2024.111323. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1568494624000978 (Accessed: 2025).

[47] Ahmed, T., Nitu, A. M., Ibn Afjal, M., Al Mamun, M. A. & Uddin, M. P. (2025). Segmentation-based deep 2D-3D multibranch learning approach for effective hyperspectral image classification. PLOS ONE, 20(5): e0321559. doi:10.1371/journal.pone.0321559. Available at: https://doi.org/10.1371/journal.pone.0321559 (Accessed: 2025).

[48] Zhang, Y., Liu, H. and Hu, Q., 2021, September. Transfuse: Fusing transformers and cnns for medical image segmentation. In International conference on medical image computing and computer-assisted intervention (pp. 14-24). Cham: Springer International Publishing. doi:10.48550/arXiv.2102.08005 Available at: https://arxiv.org/abs/2102.08005 (Accessed: 2025).

[49] Montavon, G., Samek, W. and Müller, K.-R. (2017) 'Methods for interpreting and understanding deep neural networks', Digital Signal Processing, [online] Available at: https://doi.org/10.1016/j.dsp.2017.10.011 (Accessed: 2025).

[50] O'Shea, K. and Nash, R. (2015) 'An introduction to convolutional neural networks', arXiv preprint arXiv:1511.08458. doi: https://doi.org/10.48550/arXiv.1511.08458 (Accessed: 2025).

[51] Thrun, S. (1995). Is learning the n-th thing any easier than learning the first?. Touretzky, D., Mozer, M. C. & Hasselmo, M. (eds.), Advances in Neural Information Processing Systems 8 (NIPS 1995). Morgan Kaufmann, pp. 641–647. Available at:

https://proceedings.neurips.cc/paper_files/paper/1995/file/bdb106a0560c4e46ccc488ef010af787-Paper.pdf (Accessed: 2025).

[52] He, K., Zhang, X., Ren, S. & Sun, J. (2015). Deep residual learning for image recognition. arXiv preprint arXiv:1512.03385. Available at: https://arxiv.org/pdf/1512.03385 (Accessed: 2025).

[53] Keras Applications: EfficientNet. Available at: https://keras.io/api/applications/efficientnet/ (Accessed: 2025).

[54] Tan, M. and Le, Q., 2019, May. Efficientnet: Rethinking model scaling for convolutional neural networks. In International conference on machine learning (pp. 6105-6114). PMLR. Available at: https://doi.org/10.48550/arXiv.1905.11946 (Accessed: 2025).

[55] Simonyan, K. & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 [cs.CV]. Submitted 4 September 2014; last revised 10 April 2015. Available at: https://arxiv.org/abs/1409.1556 (Accessed: 2025).

[56] ImageNet (n.d.) ImageNet. Available at: https://www.image-net.org/ (Accessed: 2025).

[57] PyTorch (n.d.) torchvision.models.resnet50. Available at: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html (Accessed: 2025).

[58] Keras (n.d.) InceptionV3. Available at: https://keras.io/api/applications/inceptionv3/ (Accessed: 2025).

[59] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*. Available at: https://arxiv.org/abs/1704.04861 (Accessed: 2025).

[60] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. & Rabinovich, A. (2014). *Going Deeper with Convolutions*. arXiv preprint arXiv:1409.4842 [cs.CV]. Available at: https://arxiv.org/pdf/1409.4842 (Accessed: 2025).

[61] Hossin, M. and Sulaiman, M.N., (2015). A review on evaluation metrics for data classification evaluations. International journal of data mining & knowledge management process, 5(2), p.1. Available at: https://doi.org/10.5121/ijdkp.2015.5201 (Accessed: 2025).

[62] Hossain, M.S., (2022). Microc alcification segmentation using modified u-net segmentation network from mammogram images. Journal of King Saud University-Computer and Information Sciences, 34(2), pp.86-94. Available at: https://doi.org/10.1016/j.jksuci.2019.10.014 (Accessed: 2025).

[63] Pallets Project (n.d.). Flask documentation. Available at: https://flask.palletsprojects.com/en/stable/ (Accessed: 2025).

[64] TensorFlow (2020) EfficientNet-Lite models. Available at: https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet/lite (Accessed: 2025).

[65] Keras (n.d.) DenseNet. Available at: https://keras.io/api/applications/densenet/ (Accessed: 2025).

[66] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S. and Guo, B., 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 10012-10022). Available at: https://arxiv.org/abs/2103.14030 (Accessed: 2025).

[67] He, K., Zhang, X., Ren, S. and Sun, J., 2015. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, *37*(9), pp.1904-1916. Available at: https://arxiv.org/abs/1406.4729 (Accessed: 2025).

[68] Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B. and Belongie, S., 2017. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2117-2125). Available at: https://arxiv.org/abs/1612.03144 (Accessed: 2025).

[69] Oktay, O., Schlemper, J., Folgoc, L.L., Lee, M., Heinrich, M., Misawa, K., Mori, K., McDonagh, S., Hammerla, N.Y., Kainz, B. and Glocker, B., 2018. Attention u-net: Learning where to look for the pancreas. *arXiv preprint arXiv:1804.03999*. Available at: https://arxiv.org/abs/1804.03999 (Accessed: 2025).

[70] Cao, H., Wang, Y., Chen, J., Jiang, D., Zhang, X., Tian, Q. and Wang, M., 2022, October. Swin-unet: Unet-like pure transformer for medical image segmentation. In *European conference on computer vision* (pp. 205-218). Cham: Springer Nature Switzerland. Available at: https://arxiv.org/abs/2105.05537 (Accessed: 2025).

[71] Isensee, F., Petersen, J., Klein, A., Zimmerer, D., Jaeger, P.F., Kohl, S., Wasserthal, J., Koehler, G., Norajitra, T., Wirkert, S. and Maier-Hein, K.H., 2018. nnu-net: Self-

adapting framework for u-net-based medical image segmentation. *arXiv preprint arXiv:1809.10486*. Available at: https://arxiv.org/abs/1809.10486 (Accessed: 2025).

[72] Ultralytics (2025) YOLOv8: State-of-the-Art Computer Vision Model. Available at: https://yolov8.com/ (Accessed: 2025).

[73] Ren, S., He, K., Girshick, R. and Sun, J., 2015. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, *28*. Available at: https://arxiv.org/abs/1506.01497 (Accessed: 2025).

[74] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626). Available at: https://arxiv.org/abs/1610.02391 (Accessed: 2025).

[75] Wang, H., Wang, Z., Du, M., Yang, F., Zhang, Z., Ding, S., Mardziel, P. and Hu, X., 2020. Score-CAM: Score-weighted visual explanations for convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops* (pp. 24-25). Available at: https://arxiv.org/abs/1910.01279 (Accessed: 2025).

[76] Wikipedia contributors (2025) 'Multi-task learning', *Wikipedia: The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Multi-task_learning (Accessed: 2025).

11. Bibliography

[77] Alderwick, H. and Dixon, J. (2019) 'The NHS long term plan', BMJ, 364, l84. doi: https://doi.org/10.1136/bmj.l84. Available at: https://www.bmj.com/content/364/bmj.l84 (Accessed: 2025).

[78] Godley, K.C., Gladwell, C., Murray, P.J. and Denton, E. (2017) 'The UK breast screening program – what you need to know', Climacteric, 20(4), pp. 313–320. doi: https://doi.org/10.1080/13697137.2017.1342619 (Accessed: 2025).

[79] Kaul, V., Enslin, S. and Gross, S.A. (2020) 'History of artificial intelligence in medicine', Gastrointestinal Endoscopy, 92(4), pp. 807–812. doi: https://doi.org/10.1016/j.gie.2020.06.040. Available at: https://www.sciencedirect.com/science/article/pii/S0016510720344667 (Accessed: 2025).

[80] Lee, R., Gimenez, F., Hoogi, A., Miyake, K.K., Gorovoy, M. and Rubin, D.L. (2017) 'A curated mammography dataset for use in computer-aided detection and diagnosis research', Scientific Data, 4, 170177. doi: https://doi.org/10.1038/sdata.2017.177 (Accessed: 2025).

[81] Mascara, M. and Constantinou, C. (2021) 'Global perceptions of women on breast cancer and barriers to screening', Current Oncology Reports, 23, 74. doi: https://doi.org/10.1007/s11912-021-01069-z (Accessed: 2025).

[82] Massat, N.J., et al. (2016) 'Impact of screening on breast cancer mortality: the UK program 20 years on', Cancer Epidemiology, Biomarkers & Prevention, 25(3), pp. 455–462. doi: https://doi.org/10.1158/1055-9965.EPI-15-0803 (Accessed: 2025).

[83] Rodriguez-Ruiz, A. et al. (2019) 'Stand-alone artificial intelligence for breast cancer detection in mammography: Comparison with 101 radiologists', Journal of the National Cancer Institute, 111(9), pp. 916–922. doi: https://doi.org/10.1093/jnci/djy222 (Accessed: 2025).

[84] Shen, L., Margolies, L.R., Rothstein, J.H., Fluder, E., McBride, R. and Sieh, W. (2019) 'Deep learning to improve breast cancer detection on screening mammography', Scientific Reports, 9, 12495. doi: https://doi.org/10.1038/s41598-019-48995-4 (Accessed: 2025).

Appendices

12. Appendix A – Supplementary Planning Content

12.1 Kanban System

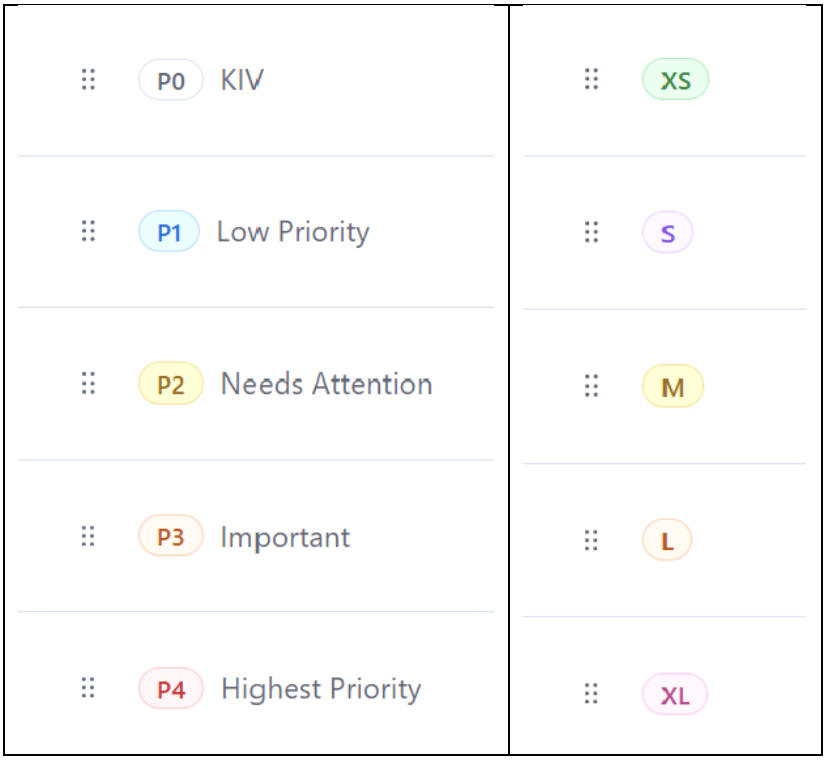


Appendix Figure 12.1.1 A Kanban Board [85]

Kanban [85] is an agile management approach centred on continuous improvement. The "board" features tasks displayed as cards while the project progresses. Stages are displayed in columns, providing flexibility since modifications and updates can be made at any moment during development. The Kanban approach was used in GitHub [19][20] to facilitate task tracking, resource management, and version control together. (See Appendix Figure 12.1.1)

The project board consists of 5 stages:

1. Backlog: Includes tasks that are planned to do and item cards in consideration.
2. Ready: tasks that are scheduled to begin.
3. In progress: tasks that are currently being worked on.
4. In review: tasks that are being reviewed.
5. Done: completed tasks that have passed review.



Appendix Figure 12.1.2 Kanban Priority (Left) and Size (Right) Options [19][20]

Item cards are also issued with a priority tag so that critical tasks take precedence over others and are completed first before moving on. Additionally, sizing is also allocated to task cards that denotes the estimated amount of time, effort, resources, required to complete the task. By utilizing the Kanban method [85], this system facilitates enhanced workflow visibility and task prioritization, allowing the project to handle tasks more efficiently and finish them with greater effectiveness. (See Appendix Figure 12.1.2)

12.2 Gantt Chart



Appendix Figure 12.2.1 Gantt Chart

Gantt Chart was created using Figma [18] as well to aid in the projects planning process. Below is a rough breakdown of the Gantt chart planning (See Appendix Figure 12.2.1 and Appendix Figure 19.1.1):

- Choosing Project (7 days)
- Project Proposal Research (14 days)
    - Project Research
    - Proposal Video
- Introduction (14 days)
    - Project Background
    - Data Source and Relevancy
    - About Breast Cancer
- Aim and Objectives (7 days)
- Motivations (7 days)
- Stakeholders (14 days)
- Methodological Approach (14 days)
- Planning (14 days)
- Literature Review (14 days)
    - Social, Economic, and Technological Factors
    - Existing Processes
- Key Technologies and Methods (7 days)
- Scope (7 days)
- Defining the problem and assembling a dataset (7 days)
- Choosing a measure of success (7 days)
- Deciding on an evaluation protocol (7 days)
- Preparing your data (14 days)
- Developing a model that does better than a baseline (14 days)

    - Baseline Model
    - Better Baseline Model

- Preliminary Outcomes and Discussion (7 days)

- Safety Net 1 (7 days)

- Scaling up: developing a model that overfits (7 days)

- Regularizing your model and tuning your hyperparameters (7 days)
    - Stratified K-fold Cross Validation
    - Regularized Model

- Wider, Deeper, Narrower Models (14 days)
    - Wider Model
    - Deeper Model
    - Narrower Model

- Application of transfer learning (14 days)

- Exploring Models and Architectures (21 days)

- Evaluation (7 days)

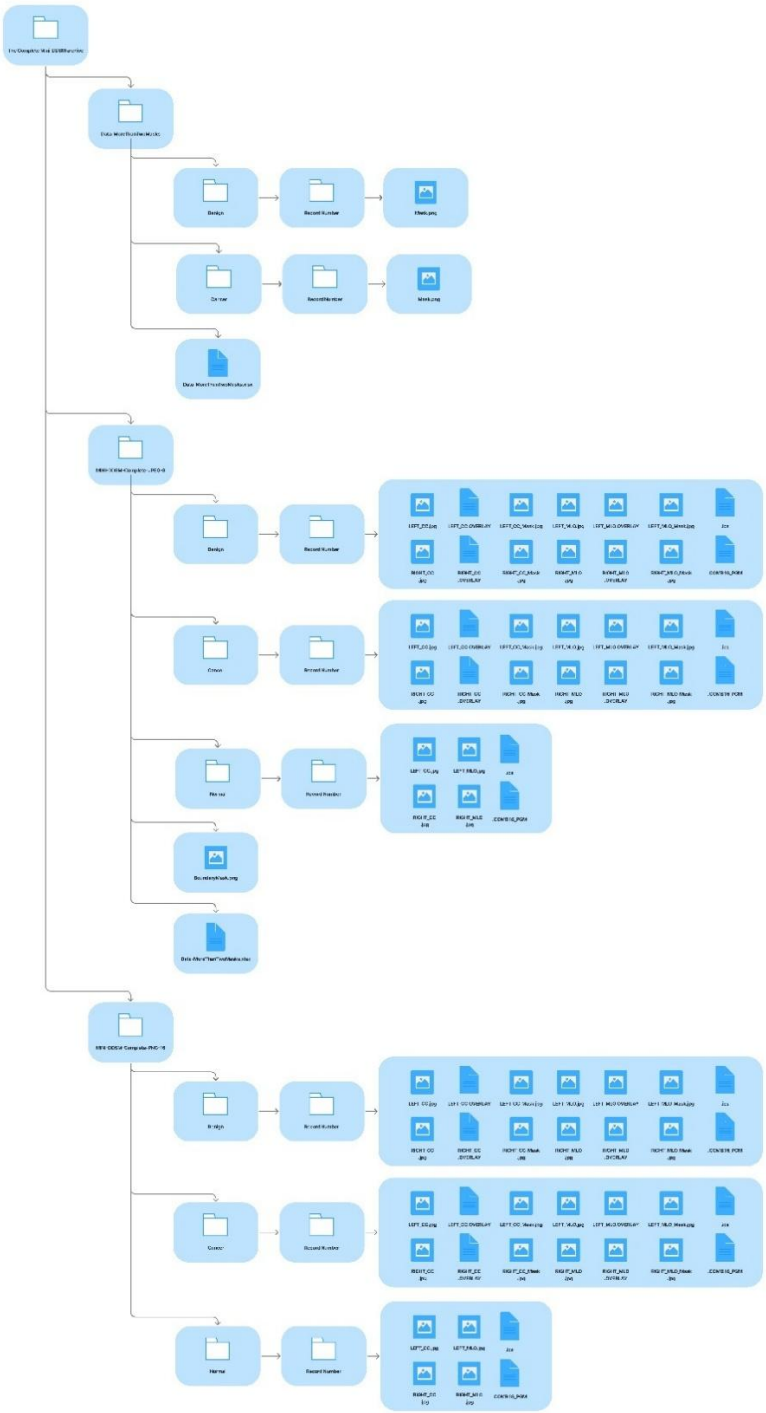
- UI Implementation (7 days)

- Conclusion (7 days)

- Safety Net 2 (7 days)

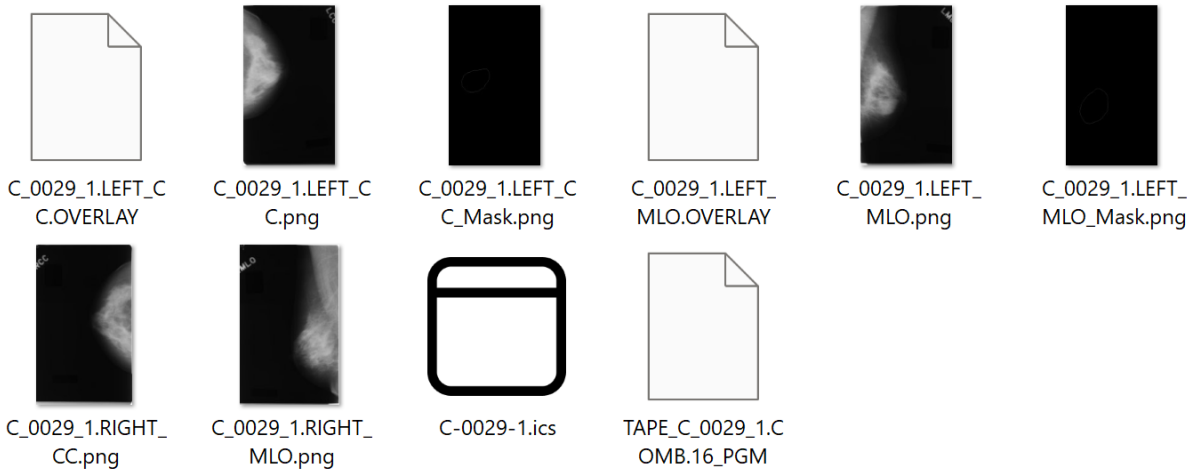
Two sections namely “Safety Net 1” and “Safety Net 2” were added to cater to unforeseen circumstances and difficulties in project tasks. These sections also act as buffers that gives the project time for analysis before moving forward.

13. Appendix B – The Complete Mini-DDSM archive Folder Structure



Appendix Figure 13.1 The Complete Mini-DDSM archive Folder Structure

The Mini-DDSM comprises of a folder with cases that have more than two masks (3-4-5-6 ROIs), and 2 main folders, MINI-DDSM-Complete-JPEG-8 and MINI-DDSM-Complete-PNG-16. (See Appendix Figure 13.1 and Appendix Figure 19.1.2)



Appendix Figure 13.2 MINI-DDSM-Complete-PNG-16 Internal Cancer Folder Structure

The folders storing normal cases has CC (top down view) and MLO (side view) images of the left and right breasts, while the benign and cancer folders include additional masks and overlays for breasts with abnormalities and cancer locations. (See Appendix Figure 13.2)

File Type	File Information
CC.png, MLO.png	Main Mammography Images
_Mask.png	Localization of Cancer Cells
.ics	Metadata
.PGM	Lagacy Images
.OVERLAY	Text based Annotations

Appendix Table 13.1 MINI-DDSM-Complete-PNG-16 Internal Cancer Folder File Information

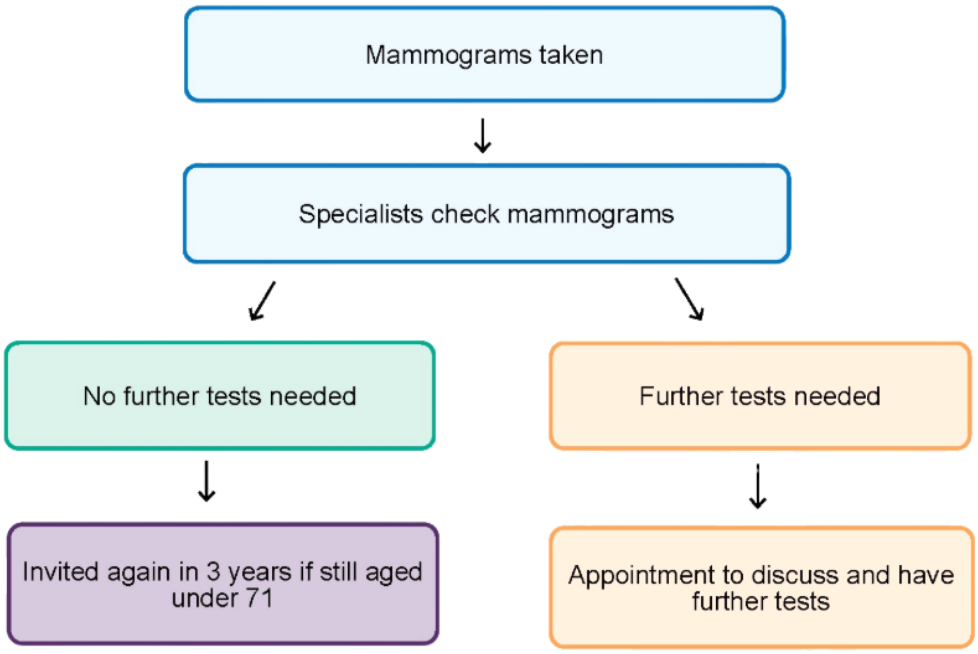
All case folders also contain iCalendar (.ics) and Portable Gray Map (.PGM) files that holds metadata and the legacy images. (See Appendix Table 13.1)

The MINI-DDSM-Complete-JPEG-8 dataset, 8-bit depth lossy compression with lower file size, was developed to facilitate public access with restricted bandwidth. This project, however, employs the MINI-DDSM-Complete-PNG-16 version, which maintains 16-bit depth lossless compression and possibly keeps considerably more pixel intensity details, crucial for identifying subtle variations in tissue.

14. Appendix C – Supplementary Literature Review

This section acts as an extension of literature review and provides additional background by examining current diagnostic processes and the technological, social, and economic aspects that impact the progression on AI DL in breast cancer screening.

14.1 Breast Cancer Diagnosis



Appendix Figure 14.1.1 Breast Screening Routine Steps [86]

In the UK, screening services usually takes place in hospitals or mobile screening units. Mammograms are being taken to identify any cancer bio markers in the breast and are being reviewed by specialists. Most people often do not require further tests due to absence of cancer; However, signs of breast cancer require further appointments and tests [86]. (See Appendix Figure 14.1.1)

A paper by Lulu Wang (2017) [87] reviewed notable available screening and biomarkers along with biosensor techniques for diagnosing early-stage breast cancer. Mentioned in the article includes diagnostic methods such as mammography, MRI, ultrasonography, PET, breast Microwave imaging (MI) and biopsy.

The paper concluded that while MI had the potential to rival or complement conventional Mammography for breast cancer diagnosis, its techniques are still immature and require obstacles to be solved before clinical trials implementation.

14.2 Social Factors

Societal perception and confidence in AI technologies hold significant importance. Research in England conducted by Lauren Gatting et al. (2024) [88] shows that although there is careful optimism regarding AI's capabilities, patients still express concerns over data privacy and the transparency of algorithms.

The research [88] surveyed varied demographic groups across geographical regions in England eligible for breast screening for public understanding and attitudes towards AI and its possible use in the NHS Breast Screening Programme [89]. Participants involved felt uncertain about AI capabilities and desired strong supporting evidence to prove its effectiveness in breast screening. Additionally, consistent with a survey study in Italy [90], a belief of total AI decision cannot be relied upon and requires human interactions.

14.3 Economic Factors

The incorporation of AI into breast cancer screening offers both advantages and challenges from an economic standpoint. A paper by Ya-Chen Tina Shih et al. (2022) [91] denoted that implementation requires substantial investment in technology infrastructure and training. Moreover, disparities in resource allocation could exacerbate existing inequalities in healthcare access, proposing connections between crucial data sources both within and across stakeholders to enhance research possibilities while safeguarding patient privacy and confidentiality.

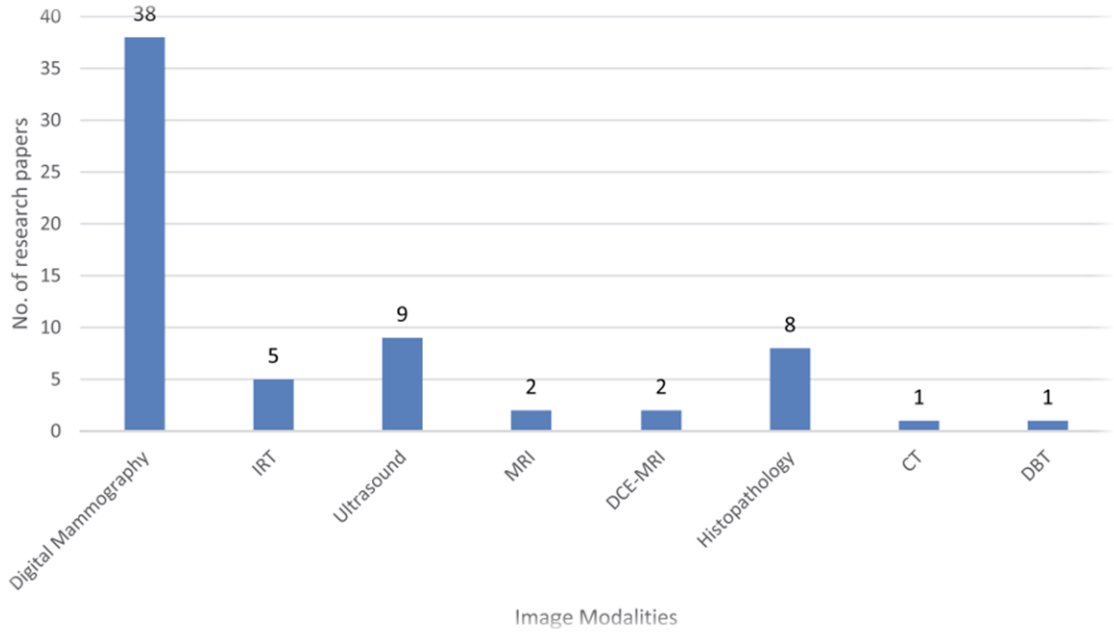
A cost-effectiveness study of incorporating risk-stratified screening using a breast cancer AI model into the United Kingdom (UK) National Breast Cancer Screening Program [89] by Johnson et al. (2024) [92] indicates that AI-assisted screening has the potential to lower long-term healthcare expenses by facilitating earlier diagnosis and intervention. The study also noted that specific risk-stratified breast cancer screening regimens being implemented [93] could also allocate resources to tackle screening delays and minimize wait times where issues arise in the UK [94], possibly enhancing breast cancer results.

14.4 Technological Factors

Technological progress has brought AI and DL to the leading edge of breast cancer detection with research showing the effectiveness of CNNs in examining mammographic images with great precision. A paper by Sharafaddini et al. (2024) [95] examined 68 studies and discovered that DL models consistently surpassed traditional approaches across multiple imaging modalities, such as mammography and ultrasound. Leveraging on CNNs to enhance image

accuracy and consistency across multiple imaging modes, the paper also addresses limited and imbalanced datasets that hamper DL model performances.

Furthermore, the fusion of radiomics and AI has improved the forecasting abilities of screening instruments, allowing for more tailored risk evaluations. Paper by Reza Elahi and Mahdis Nazari (2024) [96] discusses novel quantitative computational methods, such as radiomics, developed to improve the sensitivity and specificity of early breast cancer diagnosis and classification.



Appendix Figure 14.4.1 This Number of Publications Vs Modalities [97]

A study by Thakur, Neha et al. (2024) [97] conducted a systematic review of machine and deep learning techniques for the identification and classification of breast cancer through medical image modalities. The paper observed different classification models being used and recorded the statistics of medical imaging modalities per classifier as well. The paper also states that while mammographic images are widely used for early-stage detection, they are not used to multi-classify breast lesions (i.e. regular, benign, malignant, and their further sub-categories). This presents opportunities in DL model developments and improvements towards breast lesions multiclassification. (See Appendix Figure 14.4.1)

15. Appendix D – CNN Layers

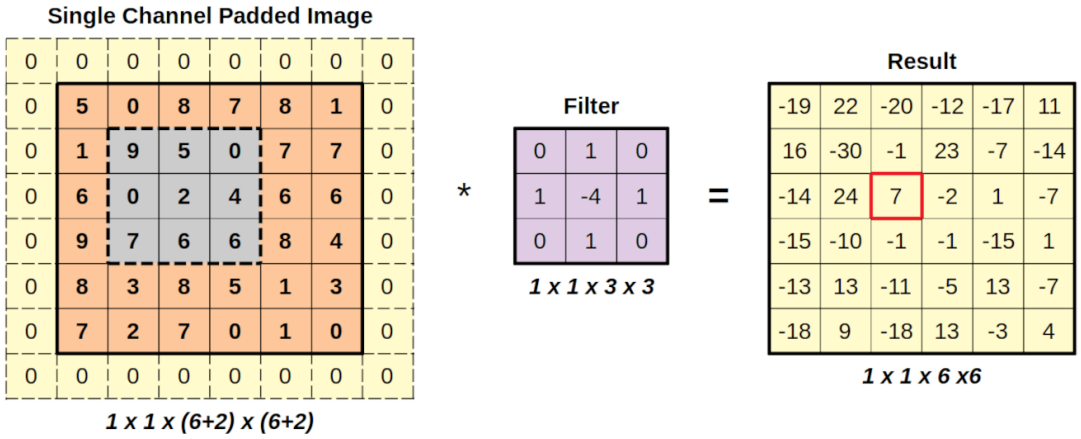
15.1 Convolutional Layers

Convolution Operation [98]:

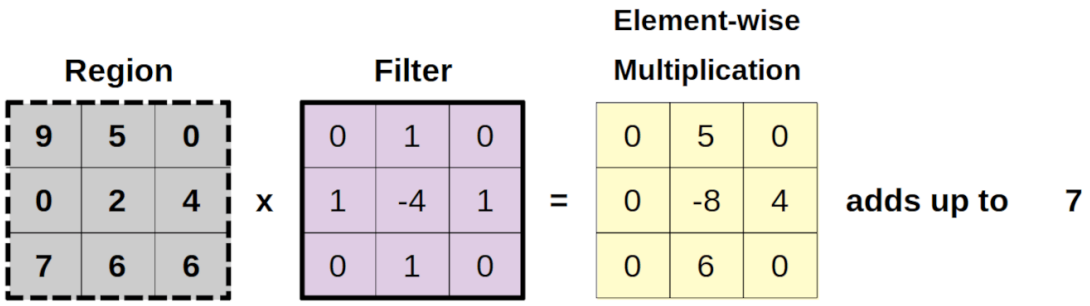
$$(f * g)(i,j) = \sum_m \sum_n f(i+m, j+n) \cdot g(m,n)$$

Where f is the input image and g represents the filter.

Convolutional layers serve as the core of CNN models. Their main goal is to derive significant features from the input data, usually an image, while maintaining the spatial connections among pixels. In contrast to standard fully connected layers where each neuron connects to every input, convolutional layers have local connections. This signifies that every neuron is linked solely to a limited area of the input, referred to as the receptive field.



Appendix Figure 15.1.1 Convolution on a Padded Single-Channel Image [101]

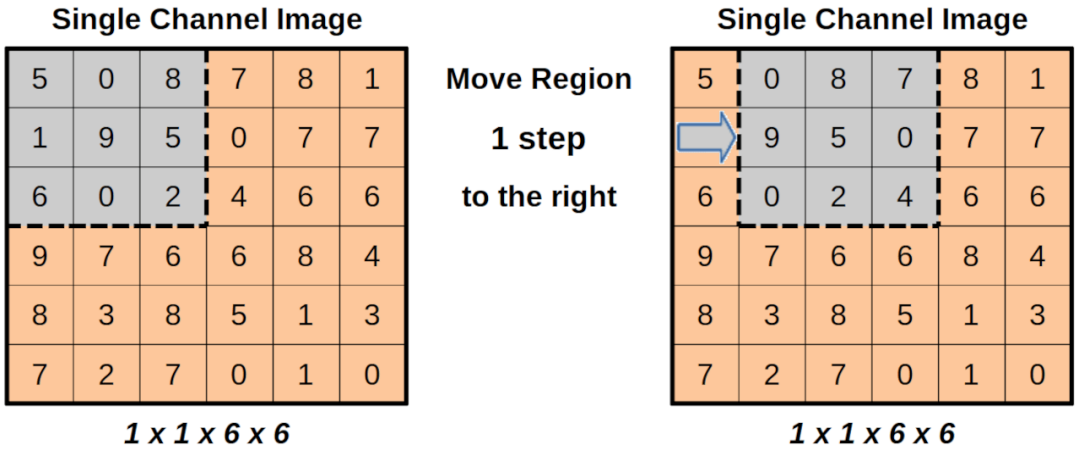


Appendix Figure 15.1.2 Single Convolution Step Element-wise Computation [101]

The layer employs adjustable filters (known as kernels), which are tiny matrices that move across the input image. At each stage, the filter executes an element-wise multiplication with

the segment of the image it encompasses and subsequently adds the outcome. This process is referred to as convolution, and the result is a feature map that shows where the filter's pattern appears in the image. A solitary convolutional layer typically contains various filters, with each one trained to identify distinct patterns like edges, corners, or textures. (See Appendix Figure 15.1.1 and Appendix Figure 15.1.2)

By layering numerous convolutional segments, CNNs can acquire hierarchical feature representations, starting from basic low-level features in the initial layers to more intricate, abstract patterns in the later layers. For example, in medical imaging like mammograms, initial layers might recognize outlines and patterns, whereas deeper layers might learn to detect lesions or other abnormalities indicative of cancer.

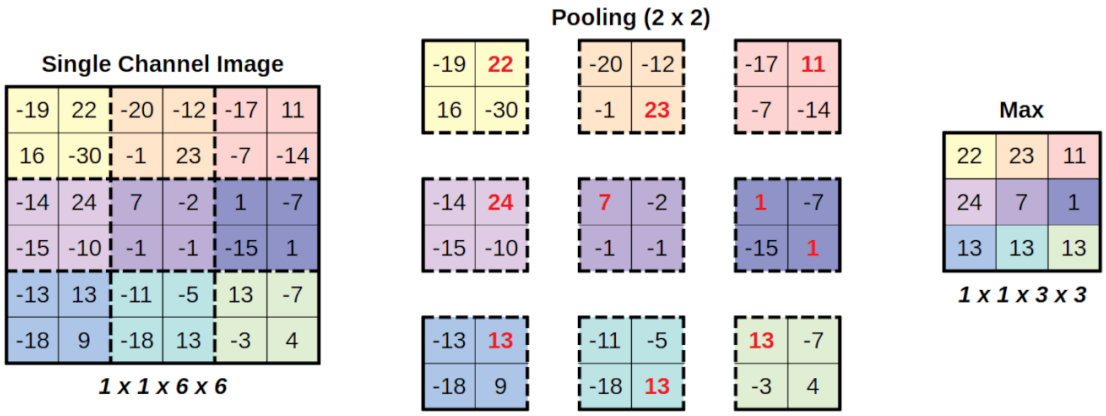


Appendix Figure 15.1.3 This Convolutional Filter Moving Across a Single Channel Image with a Stride of 1 [101]

Another important element of convolutional layers is the implementation of padding and stride. Padding aids in maintaining spatial dimensions by inserting additional pixels (frequently zeros) around the edges of the image, whereas stride dictates the distance the filter shifts each time it glides across the input. These factors affect the dimensions of the resulting feature maps and manage the balance between spatial accuracy and computational efficiency. (See Appendix Figure 15.1.1 and Appendix Figure 15.1.3)

15.2 Pooling Layers

Pooling layers are frequently employed in CNNs to decrease the spatial dimensions of the feature maps produced by convolutional layers. This down-sampling process has two primary objectives: it reduces the number of parameters and calculations in the network, enhancing efficiency, and it creates a type of translation invariance, enabling the network to identify features despite minor positional variations in the input.



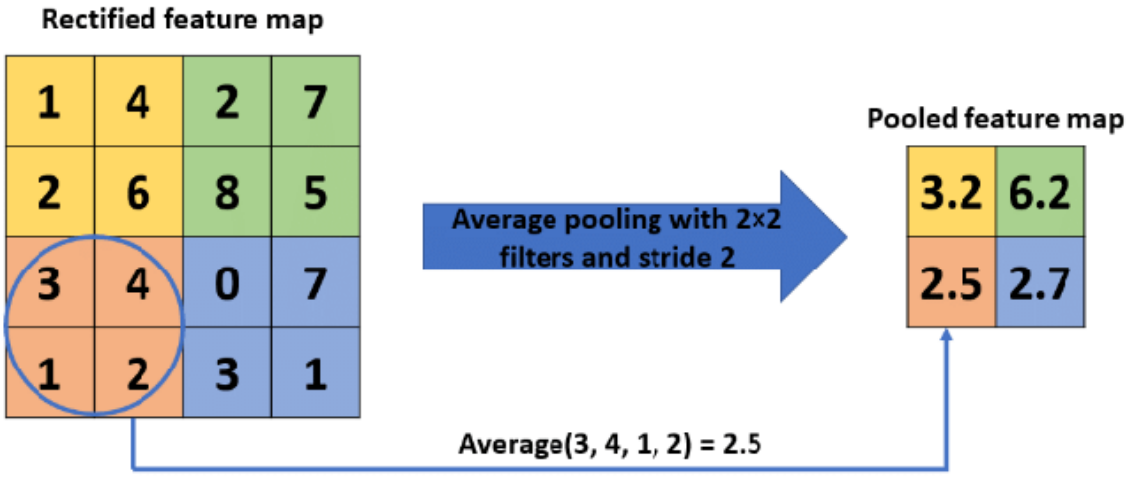
Appendix Figure 15.2.1 2x2 Max Pooling Operation [101]

Max Pooling Operation [98][99][100]:

$$y^{\{(l+1)\}}_{\{i',j',d\}} = max_{\{0 \le m < H, 0 \le n < W\}} \left(x^{\{(l)\}}_{\{i' \cdot H + m, j' \cdot W + n, d\}} \right)$$

Where $0 \le i' < H^{\{(l+1)\}}, 0 \le j' < W^{\{(l+1)\}}, 0 \le d < D^{\{(l+1)\}} = D^{\{(l)\}}$

Various types of pooling exist, but max pooling is the most utilized. In max pooling, a small window (for instance, 2x2) moves across the feature map, and solely the maximum value within every window is kept. This method retains the most notable characteristics while eliminating less important details. (See Appendix Figure 15.2.1)



Appendix Figure 15.2.2 Average Pooling Operation [102]

Average pooling is an alternative method that calculates the average value inside the window rather than the maximum. Although average pooling can soften the features, it can also diminish sharpness and distinguishing ability, which is why max pooling is typically favoured

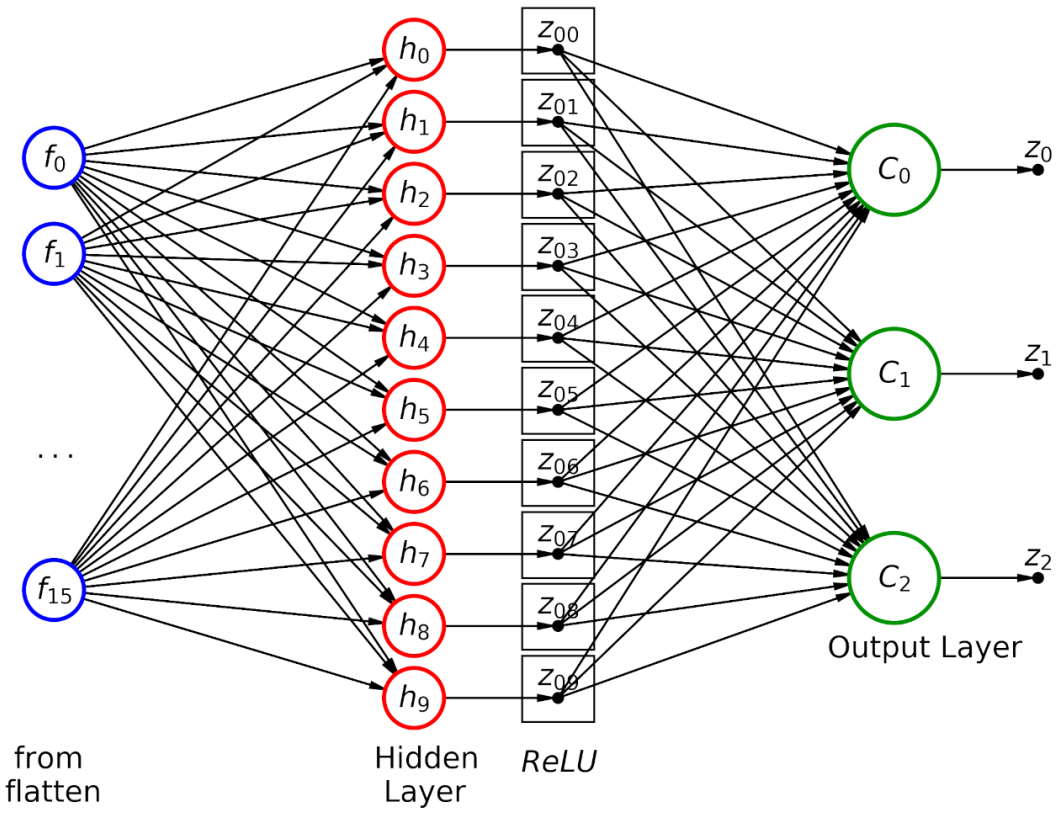
in practice. Other pooling methods in DNNs includes but is not limited to, mixed pooling, stochastic pooling, spatial pooling, spectral pooling [102]. (See Appendix Figure 15.2.2)

Pooling layers usually come after convolutional layers in the structure. They play an essential role in progressively diminishing the spatial dimensions of the representations, which aids in mitigating overfitting and enhancing generalization. In breast cancer detection, pooling enables the network to preserve only the key features like tumour boundaries or mass density while disregarding unnecessary variations in the image background.

Although pooling is an uncomplicated operation, its impact on the model's capacity to concentrate on essential spatial hierarchies cannot be underestimated. Researchers occasionally investigate options such as strided convolutions or adaptive pooling to enhance this process, based on the dataset and the sensitivity of the task

15.3 Fully Connected / Dense Layers

Fully connected layers, often referred to as dense layers, typically appear near the end of most CNN architectures and function as the final step where acquired features are converted into class scores or regression results. In contrast to convolutional layers that preserve spatial structure, Fully Connected layers flatten the input, transforming 2D feature maps into a 1D vector, linking every neuron in one layer to all neurons in the subsequent layer.



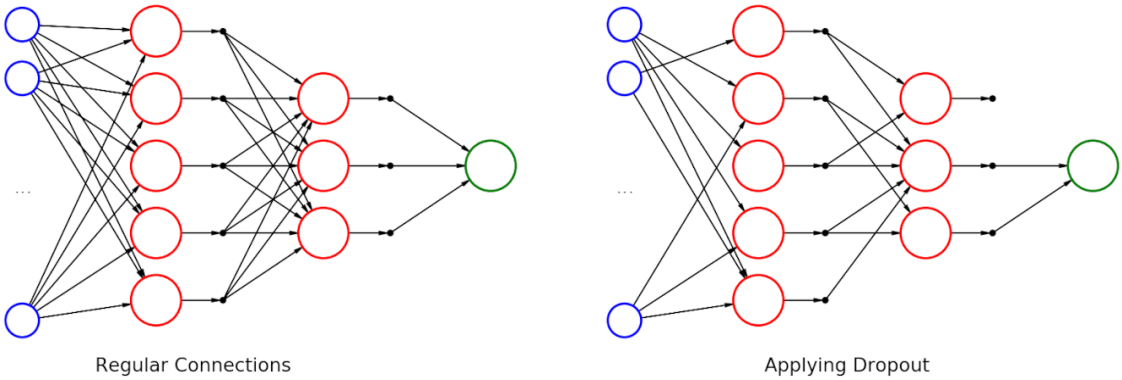
Appendix Figure 15.3.1 Softmax Activation [101]

ReLU Activation Function [98][99][100]:

$$f(x) = \max(0, x)$$

These layers integrate all the localized features acquired during the convolution and pooling phases and generate a final prediction based on this information. In a fully connected layer, every connection has a weight that is modified during training to reduce the prediction error. Typically, the result from each neuron is processed through a non-linear activation function, like the ReLU, to add non-linearity and allow the network to capture intricate relationships.

In classification tasks, the last fully connected layer typically utilizes a softmax activation function to transform raw class scores into probability distributions across the target classes. For simple binary classifications like deciding if a mammogram indicates breast cancer, a sigmoid activation function can be employed instead, producing a value ranging from 0 to 1. (See Appendix Figure 15.3.1)



Appendix Figure 15.3.2 Regular Connections and Dropout Applied Comparison [101]

Although fully connected layers are common in deep learning frameworks, they can be susceptible to overfitting, particularly when dealing with a large input feature vector. Methods like dropout which involves disregarding a random selection of neurons during training, are frequently applied in these layers to enhance generalization. In certain contemporary architectures, fully connected layers are either substituted or diminished to decrease model size and enhance interpretability, particularly in fields such as medical imaging where transparency is vital. (See Appendix Figure 15.3.2)

16. Appendix E – User Interface

16.1 Ethics on Application Interaction

The ethical implications of utilizing artificial intelligence (AI) in healthcare are crucial, particularly in situations dealing with sensitive medical information like breast cancer mammography images. Although the potential application planned in this project would demonstrate capabilities of deep learning models in aiding the analysis of mammogram images, its implementation brings forth ethical concerns regarding privacy, safety, and the proper use of automated forecasts.

Data Privacy

This application is meant to operate locally or in a browser (e.g., using GitHub Pages) without sending data to outside servers. Users are strongly recommended against uploading actual patient data or identifiable medical records in an online setting. Ensuring patient confidentiality is a core ethical obligation as the tool is designed solely for use with anonymity.

Risks of Application Misinterpretation

While the application model aims to categorize images into types like cancer, benign, or normal, it does not possess the thorough contextual insight and clinical expertise of human radiologists. Misunderstanding the model's outcomes could result in unnecessary worry, false comfort, and or inappropriate self-evaluation. To mitigate these risks, the application should highlight disclaimers that warrant users about the generated outcomes.

User Responsibility

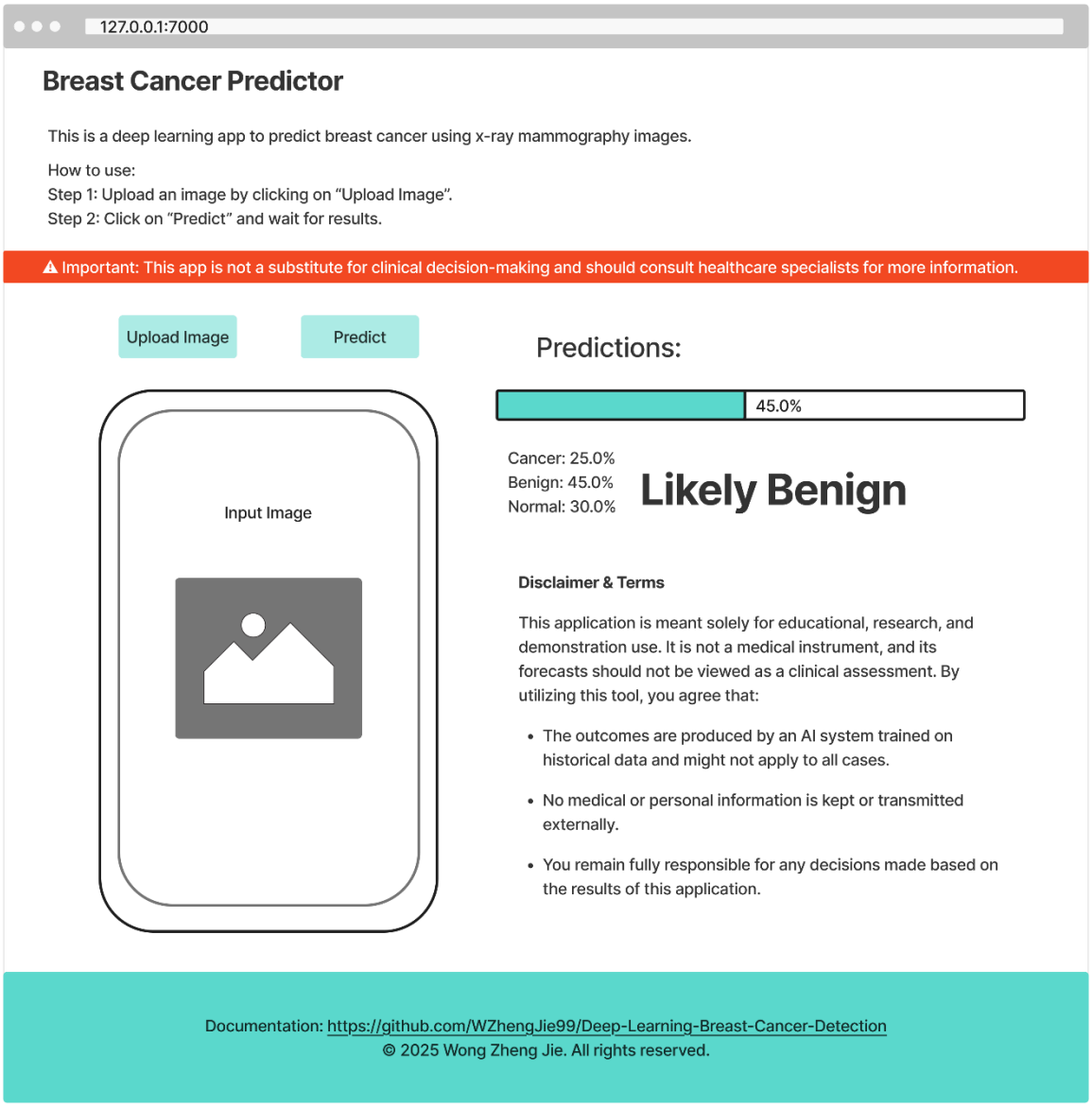
Engagement with the application should involves a clear understanding and educated use by the user. The app offers feedback by displaying probability distributions and prediction labels to assist users in understanding results carefully. Users should be advised to consider the model’s predictions as indicative rather than definitive, and to acknowledge the probabilistic aspect of the output results.

Usage in Healthcare Settings

The application should function under human supervision in healthcare situations and particularly in clinical settings. Although automation can enhance diagnostic processes, ultimate decisions should rest with qualified healthcare professionals and specialists.

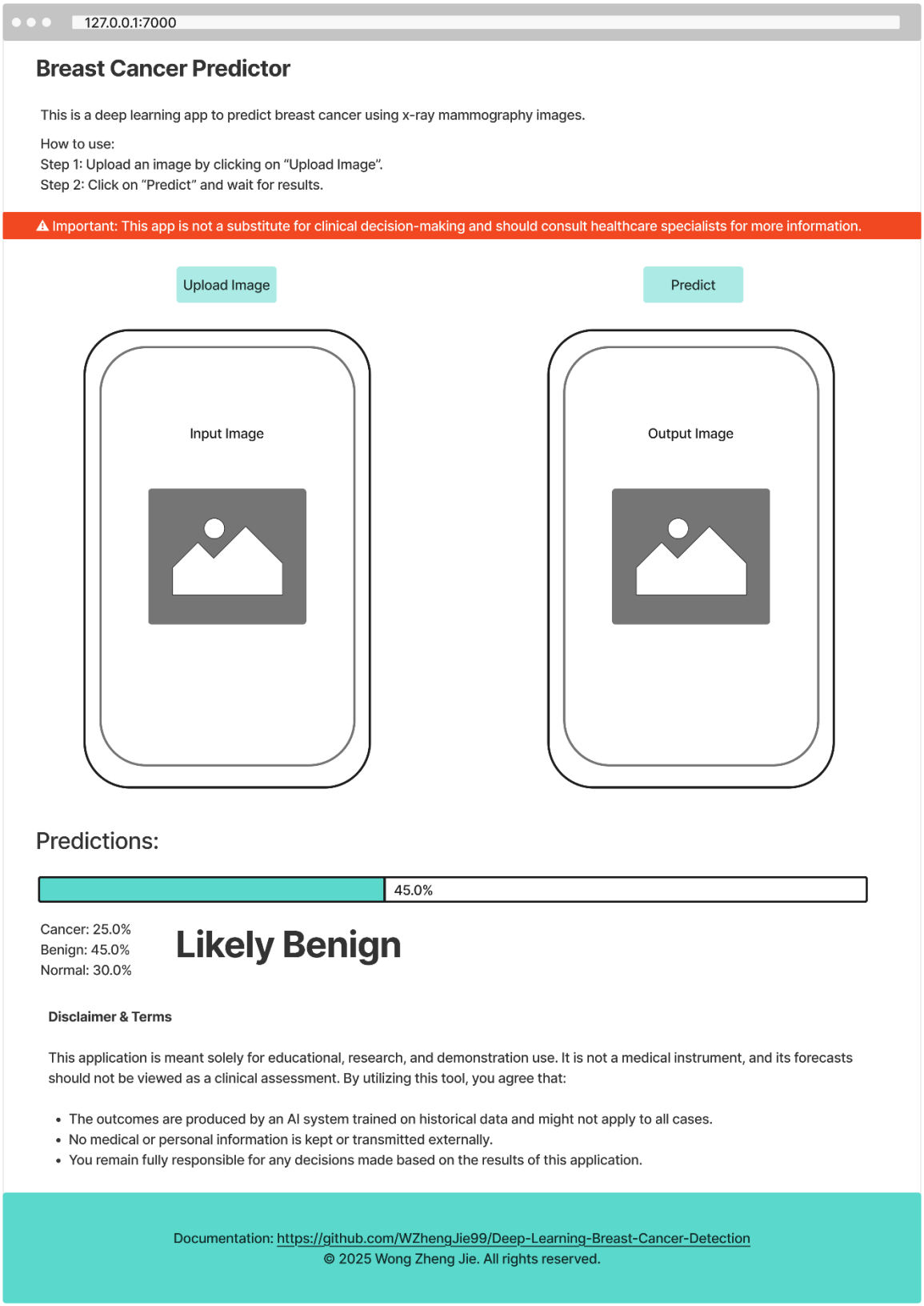
16.2 UI Design

The application is planned to be developed as an executable (.exe) file or batch file (.bat) for clinical settings as a restrain and prevent input data from being exposed to others as a privacy concern. However, it was also thought to be deployed in a browser (e.g., using GitHub Pages) for academic purposes. Therefore, the application design revolves around webpage deployment as a static webpage for online and local use (127.0.0.1).



Appendix Figure 16.2.1 Simple UI Design Version 1

The initial concept mainly encompasses uploading the image and its preview as well as output predictions and disclaimers involved. (See Appendix Figure 16.2.1)



Appendix Figure 16.2.2 Simple UI Design Version 2

A second design was created to show an output image that would indicate cancer localization. While the designs show aspects for clinical use, both don't cover predictions expressively.

Further designs and application inclusions would cover and is not limited to, other evaluation metrics, about breast cancer, model selection, and explanation of results (cancer stages, masses, calcifications, architectural distortions, treatment. etc). (See Appendix Figure 16.2.2)

Simple UI Highlighted Important Warning

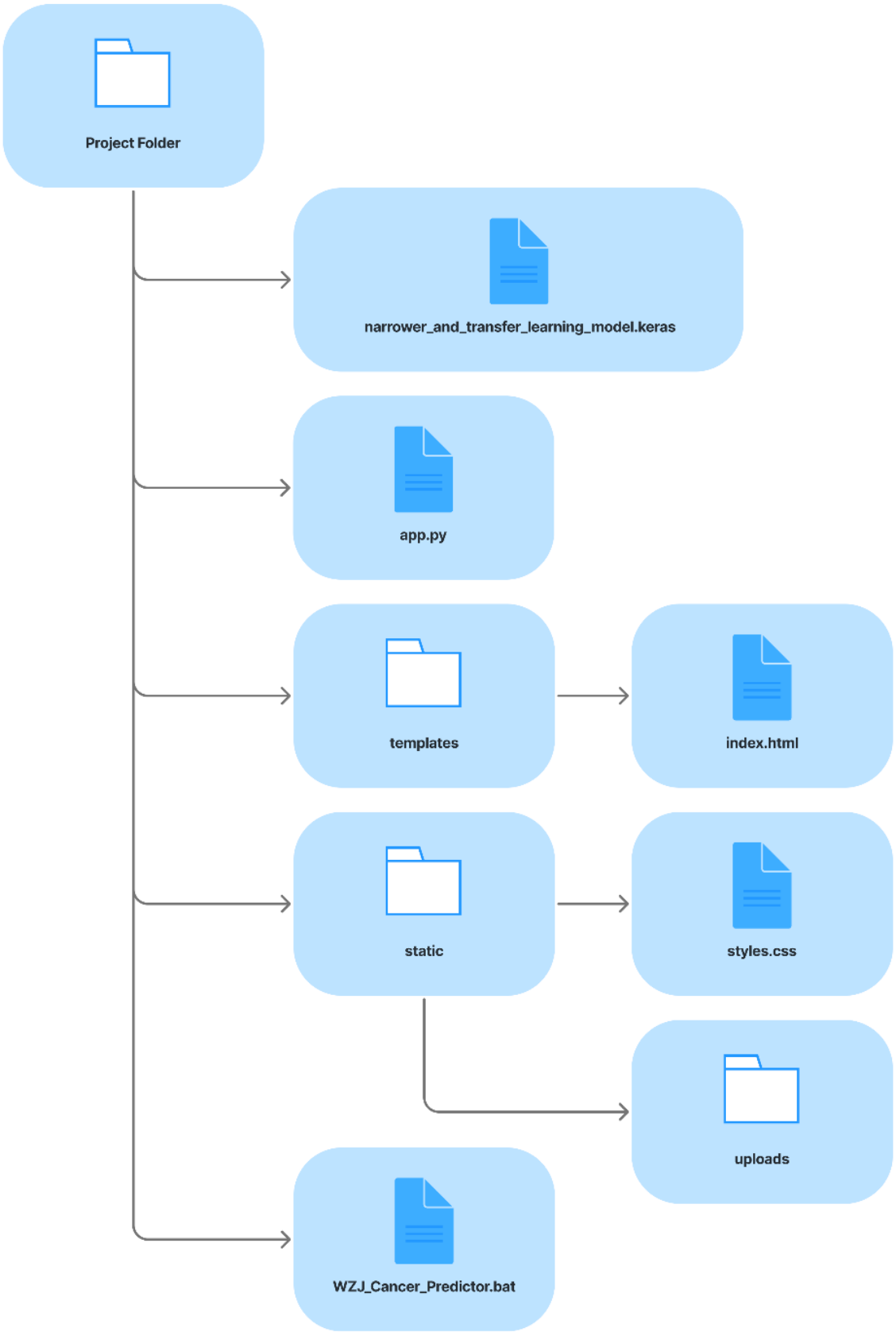
⚠️ Important: This app is not a substitute for clinical decision-making and should consult healthcare specialists for more information.

Simple UI Design Disclaimer & Terms

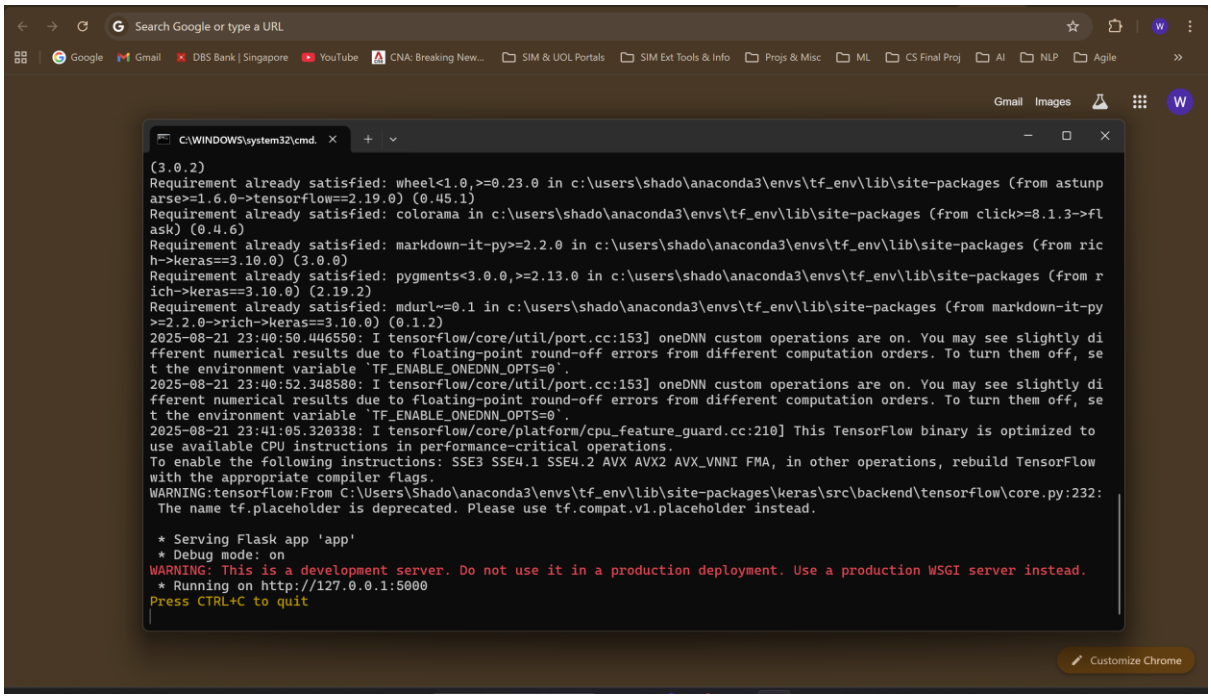
This application is meant solely for educational, research, and demonstration use. It is not a medical instrument, and its forecasts should not be viewed as a clinical assessment. By utilizing this tool, you agree that:

- The outcomes are produced by an AI system trained on historical data and might not apply to all cases.
- No medical or personal information is kept or transmitted externally.
- You remain fully responsible for any decisions made based on the results of this application.

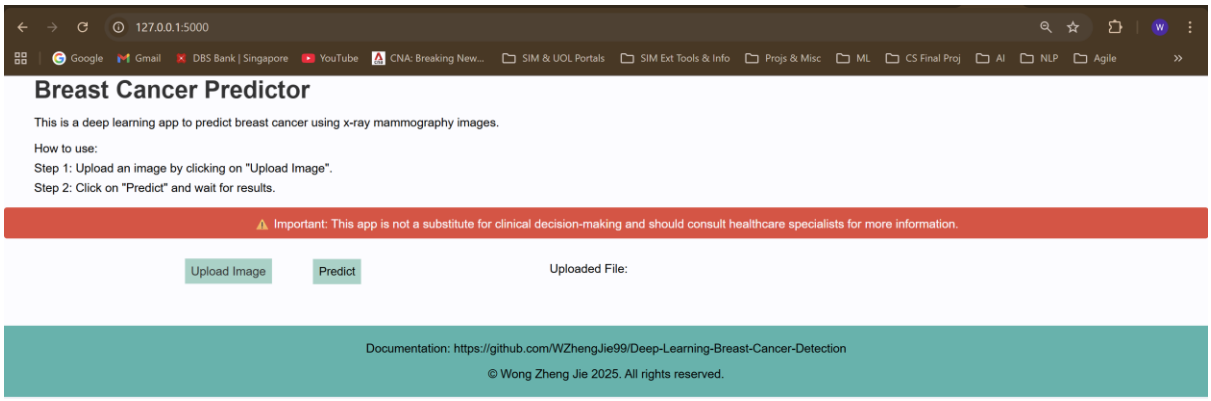
16.3 UI Implementation



Appendix Figure 16.3.1 UI Implementation Folder Structure



Appendix Figure 16.3.2 UI Implementation Batch File Start Up



Appendix Figure 16.3.3 UI Implementation Initial Load Up

The UI uses python to utilize the narrower_and_transfer_learning_model.keras checkpoint trained in an IPYNB and Flask [63] web application framework to run the application locally. Some HTML and CSS was coded for stylization and presentation. The python script was made to start using a batch file which includes mainly checking 3 important dependencies, Python version (v3.10.18), TensorFlow [103] version (v2.19.0), Keras [104] version (3.10.0). (See Figure 7.1.1 and Appendix Figure 16.3.2)

The main concern for the 3 above mentioned dependencies is due to the library and environment the keras checkpoint file trained in. Usage of HDF5 conversion and implementation in the script was considered for legacy keras and general usage without specific version control but requires switching / reinstalling matching dependency versions.

16.4 UI Output Results

A small-scale test was performed to assess the effectiveness of the created user interface (UI) by uploading six images, with two from each category (Benign, Cancer, Normal). The UI effectively showed the estimated probabilities for each category after running the images through the trained model.

Original Image Class	Image Name	Benign	Cancer	Normal
Normal	A_0003_1.RIGHT_MLO.png	28.1%	34.0%	**37.9%**
Normal	A_0005_1.LEFT_CC.png	6.6%	4.3%	**89.1%**
Benign	A_1124_1.RIGHT_MLO.png	**35.1%**	30.3%	34.7%
Benign	A_1123_1.LEFT_CC.png	34.0%	**34.3%**	31.7%
Cancer	A_1000_1.LEFT_MLO.png	**39.9%**	31.9%	28.2%
Cancer	A_1400_1.LEFT_CC.png	38.3%	**47.3%**	14.4%

Appendix Table 16.4.1 UI Output Results on 6 Mammography Images

Table 16.4.1 below shows the original image category, the name of the image file, and the prediction outcomes produced by the model. The percentages represents the likelihood distribution among the three categories with bold and yellow shaded cells representing the highest and likely prediction for the images. (See Table 6.4.1)

In general, the UI accurately showed class probabilities for every uploaded image. Benign and Cancer class shows missclassifications, highlighting the challenges of identifying minor distinctions. Although not every prediction aligned with the true class confidently, the test shows that the UI operates correctly and delivers an understandable output of the model's findings.

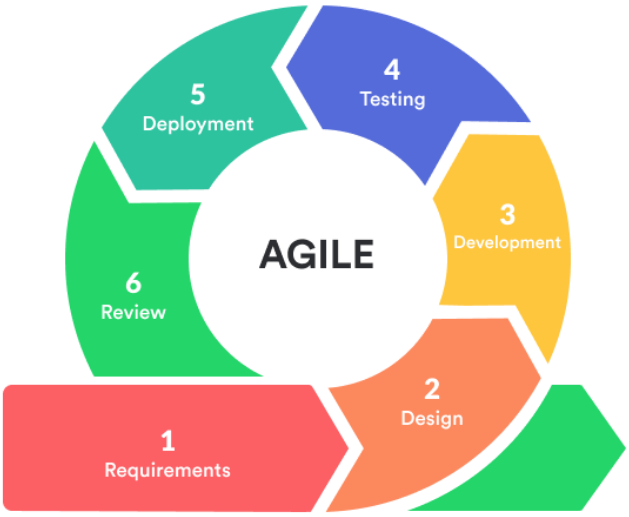
16.5 UI Further Advancement Strategy

While the main aim of this project does not focus on UI implementation, this section highlights the key steps to progress its developments. Market research would be conducted with SWOT analysis, searching for existing cancer predictors utilizing mammography images. User-Centred Design (UCD) and Agile Methodology would be the main approaches for its continuous developments with emphasis on pinpointing areas of microcalcification and Cancer stage evaluation explanation.

User-Centered Design (UCD)

UCD was chosen to ensure that the UI accurately corresponds to user requirements while upholding a straightforward and user-friendly interface. Engaging users, mainly stakeholders, during the development process allows the system to adapt based on genuine feedback and evaluations of usability. This strategy reduces the chances of developing features that do not fulfil user expectations, thereby enhancing the likelihood of user satisfaction and acceptance.

Agile Methodology



Appendix Figure 15.4.1 Agile Methodology Cycle [105]

The Agile methodology was embraced for its flexibility and its emphasis on providing working UI in gradual increments. This structure enables the project to swiftly adapt to changes whether driven by user input or changing demands. Agile fosters a cooperative atmosphere between the project developments and stakeholders, guaranteeing transparency and mutual understanding during the entire project lifecycle. The iterative aspect of Agile also facilitates testing and validation, guaranteeing that every cycle contributes meaningful value and gradually propels the project forward.

17. Appendix F – Ethics in Data Collection

Due to a lack of publicly available data in medical and breast cancer mammography images, with the approval of user data collection, continuous collection of these data can help to robust the dataset, improving AI models that utilize them. This section addresses some of the ethical actions and reasons for future data collection.

Anonymity

Collecting medical data can be very ethically critical, especially when it comes to anonymity as it can reveal a person's medical information. Institutions and individuals collecting these data should omit identities before its manipulation.

Transparency

When collecting and handling medical data, institutions and individuals should explicitly address what they will be collecting, using, and state the reasons for its collection. This allows data providers to know the information they will be providing and understand the reasons and impacts of doing so.

Accountability

Even if data providers have acknowledged the collation of their data, institutions and individuals should hold themselves accountable as they manage these data assets. Holding regular audits and reviews of data practices being conducted ensures compliance with ethical standards and regulatory requirements. Additionally, any changes to data policies should be promptly communicated.

Summary

In summary, as the advancement and application of AI in healthcare expands, prioritizing ethical data practices should remain essential. Institutions and individuals should strive to uphold these practices whether due to ethical reasons or legal obligations.

18. Appendix G – Codes

Below contains the codes for the project's developments. While the Jupyter Notebook (.ipynb) file contains markdown cells, they reflect the same contents outlined in this report and hence are being omitted.

Although written codes should display and function as intended by their names (i.e. data_dir for data directory, def apply_laplacian() for applying Laplacian filter), the comments in these codes act as an annotative note, serving as a documentation precaution, a quick explanation, and addresses oversight defensive coding practices such as 'magic numbers' declared.

WZJ_CM3070_FYP.ipynb

Importing Libraries

```
import os
import shutil
import pandas as pd
import numpy as np
import random
import seaborn as sns
import cv2
import glob
from tqdm import tqdm

import matplotlib.pyplot as plt
from matplotlib.colors import Normalize

import tensorflow as tf
from keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models, regularizers, callbacks, losses, optimizers, backend as K
from tensorflow.keras.utils import to_categorical, Sequence
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import Model, Input
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D, Conv2D, MaxPooling2D, BatchNormalization, Concatenate, SpatialDropout2D, Lambda

from tensorflow.keras.optimizers import Adam

from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score, precision_score, recall_score, f1_score, jaccard_score
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import StratifiedKFold, train_test_split
```

Loading and Preparing Dataset

```
# directories
# Original Source Folder
source_dir = 'The Complete Mini-DDSM archive/MINI-DDSM-Complete-PNG-16'
# Folder to store images
combined_img_dir = 'Combined_MiniDDSM_imgs'
# Folder to store Mask images
combined_img_mask_dir = 'Combined_MiniDDSM_img_Masks'

# Create output directories for images and masks
for root in [combined_img_dir, combined_img_mask_dir]:
    os.makedirs(root, exist_ok=True)
    for label in ['Benign', 'Cancer', 'Normal']:
        os.makedirs(os.path.join(root, label), exist_ok=True)

# copy images and masks
for label in ['Benign', 'Cancer', 'Normal']:
    label_path = os.path.join(source_dir, label)
    target_label_path = os.path.join(combined_img_dir, label)
    target_mask_label_path = os.path.join(combined_img_mask_dir, label)

    for case_id in os.listdir(label_path):
        case_path = os.path.join(label_path, case_id)

        if os.path.isdir(case_path):
            for file in os.listdir(case_path):
                if file.endswith(('CC.png', 'MLO.png')) and 'Mask' not in file:

                    # Copy image file to the new image folder
                    src_img = os.path.join(case_path, file)
                    dst_img = os.path.join(target_label_path, file)
                    shutil.copyfile(src_img, dst_img)

                    # Copy mask image file to the new mask image folder
                    mask_name = file.replace('.png', '_Mask.png')
                    src_mask = os.path.join(case_path, mask_name)
                    if os.path.exists(src_mask):
                        dst_mask = os.path.join(target_mask_label_path, mask_name)
                        shutil.copyfile(src_mask, dst_mask)

print(f"Copy Complete")
```

```
# Empty list to hold metadata for images and mask images
data = []

# Iterates over each directory inside the combined image folder
for label in os.listdir(combined_img_dir):
```

```
    label_combined_img_dir = os.path.join(combined_img_dir, label)
    label_combined_img_mask_dir = os.path.join(combined_img_mask_dir, label)

    if os.path.isdir(label_combined_img_dir):
        for fname in os.listdir(label_combined_img_dir):
            if fname.lower().endswith('.png'):
                img_path = os.path.join(label_combined_img_dir, fname)
                mask_name = fname.replace('.png', '_Mask.png')
                mask_path = os.path.join(label_combined_img_mask_dir, mask_name)

                # If mask exists then add it to the metadata else label as None
                if os.path.exists(mask_path):
                    data.append({'image_path': img_path, 'mask_path': mask_path, 'label': label})
                else:
                    data.append({'image_path': img_path, 'mask_path': None, 'label': label})

# saving metadata csv file
df = pd.DataFrame(data)
df.to_csv("Combined_MiniDDSM_Metadata.csv", index=False)
print("Saved Combined_MiniDDSM_Metadata.csv")
```

```
# Classification labels
classes = ['Benign', 'Cancer', 'Normal']

# Print number of .png images per class
for label in os.listdir(combined_img_dir):
    class_path = os.path.join(combined_img_dir, label)
    if os.path.isdir(class_path):
        image_list = [f for f in os.listdir(class_path) if f.lower().endswith('.png')]
        print(f"{label}: {len(image_list)} images")

# Creating figure for viewing
# 1 rows for viewing 'Benign', 'Cancer', 'Normal' CC images and 1 row for viewing 'Benign',
'Cancer', 'Normal' MLO images.
fig, axs = plt.subplots(2, len(classes), figsize=(24, 25))

for col, label in enumerate(classes):
    class_path = os.path.join(combined_img_dir, label)
    all_images = [f for f in os.listdir(class_path) if f.lower().endswith('.png')]

    # randomly picks one CC image
    cc_images = [f for f in all_images if '_CC.png' in f]
    if cc_images:
        cc_file = random.choice(cc_images)
        cc_path = os.path.join(class_path, cc_file)
        cc_img = cv2.imread(cc_path, cv2.IMREAD_GRAYSCALE)
        if cc_img is not None:
            axs[0, col].imshow(cc_img, cmap='gray')
            axs[0, col].set_title(f"{label} – CC\n{cc_file}", fontsize=16)
```

```
        else:
            axs[0, col].set_title(f"{label} – CC\n(Load failed)", fontsize=16)
    axs[0, col].axis('off')

    # randomly picks one MLO image
    mlo_images = [f for f in all_images if '_MLO.png' in f]
    if mlo_images:
        mlo_file = random.choice(mlo_images)
        mlo_path = os.path.join(class_path, mlo_file)
        mlo_img = cv2.imread(mlo_path, cv2.IMREAD_GRAYSCALE)
        if mlo_img is not None:
            axs[1, col].imshow(mlo_img, cmap='gray')
            axs[1, col].set_title(f"{label} – MLO\n{mlo_file}", fontsize=16)
        else:
            axs[1, col].set_title(f"{label} – MLO\n(Load failed)", fontsize=16)
    axs[1, col].axis('off')

plt.tight_layout()
plt.show()
```

```
# Get images from 3 classes
cancer_imgs = glob.glob('Combined_MiniDDSM_imgs/Cancer/*.png')
benign_imgs = glob.glob('Combined_MiniDDSM_imgs/Benign/*.png')
normal_imgs = glob.glob('Combined_MiniDDSM_imgs/Normal/*.png')

# Sample 18 from each
some_can = np.random.choice(cancer_imgs, 18, replace=False)
some_ben = np.random.choice(benign_imgs, 18, replace=False)
some_nor = np.random.choice(normal_imgs, 18, replace=False)

# Create 6x6 grid showing the images
fig, axs = plt.subplots(6, 6, figsize=(18, 18))
classes = [('Cancer', some_can), ('Benign', some_ben), ('Normal', some_nor)]

for i, (label, images) in enumerate(classes):
    for j in range(18):
        row = (i * 2) + (j // 6 % 2)
        col = j % 6
        img = image.load_img(images[j], color_mode='grayscale', target_size=(100, 100))
        img_array = image.img_to_array(img).squeeze()

        axs[row, col].imshow(img_array, cmap='plasma')
        axs[row, col].axis('off')
        axs[row, col].set_title(label, fontsize=10)

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()
```

Preprocessing and Augmentation

```
# 224×224 pixels input size
img_height, img_width = 224, 224
# Number of samples per batch
batch_size = 32

# preprocessing and augmentation for training set
train_data_generator = ImageDataGenerator(
    # Normalizing pixel values to [0, 1]
    rescale=1./255,
    # Randomly rotates images up to 15 degs
    rotation_range=15,
    # Randomly shifts images up to 10% of width horizontally
    width_shift_range=0.1,
    # Randomly shifts images up to 10% of height vertically
    height_shift_range=0.1,
    # Randomly zooms images up to 10%
    zoom_range=0.1,
    # False to retain left and right original views; no point flipping images horizontally or vertically
    horizontal_flip=False,
    vertical_flip=False,
    # This reserves 20% of data for validation
    validation_split=0.2
)

# preprocessing for validation and test set
# Normalizing pixel values to [0, 1]
test_data_generator = ImageDataGenerator(rescale=1./255)
```

Train, Validation, Test Splitting

```
# training (80% of data)
training_set = train_data_generator.flow_from_directory(
    combined_img_dir,
    target_size=(img_height, img_width),
    color_mode='grayscale',
    batch_size=batch_size,
    class_mode='categorical',
    subset='training',
    shuffle=True
)

# validation (20% of data)
validation_set = train_data_generator.flow_from_directory(
    combined_img_dir,
    target_size=(img_height, img_width),
    color_mode='grayscale',
```

```
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)
```

Class Imbalance

```
# class labels
labels = training_set.classes

# gets number of samples per class
unique, counts = np.unique(labels, return_counts=True)
print("Training set class distribution count:", dict(zip(training_set.class_indices, counts)))

# class names in order
class_names = list(training_set.class_indices.keys())

# color map and bar chart plotting
cmap = plt.get_cmap('YlGnBu')
norm = Normalize(vmin=min(counts), vmax=max(counts))
colors = [cmap(norm(count)) for count in counts]

plt.figure(figsize=(8, 6))
plt.bar(class_names, counts, color=colors)
plt.title("Training Set Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of Images")
plt.tight_layout()
plt.show()
```

```
class_weights = compute_class_weight(
    # Assigns weights inversely proportional to class frequencies
    class_weight='balanced',
    # Unique class labels
    classes=np.unique(labels),
    # The list of class labels
    y=labels
)

# Converts class weights to dictionary mapping class index to weight
class_weights_dict = dict(enumerate(class_weights))
print("Class weights:", class_weights_dict)
```

Naive Baseline Model

```
naive_baseline_model = models.Sequential([
    # Models input size
```

```
    layers.Input(shape=(img_height, img_width, 1)),

    # 32 filters with 3×3 kernel and ReLU activation
    layers.Conv2D(32, (3, 3), activation='relu'),
    # MaxPooling2D(2, 2) downsamples the image by a factor of 2×2
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 128
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    layers.Flatten(),
    # Fully connected layer with 128 neurons
    layers.Dense(128, activation='relu'),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="naive_baseline_model")

naive_baseline_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy'])

naive_baseline_model.summary()
```

```
naive_baseline_model_history = naive_baseline_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # Number of Epochs
    epochs=20,
    # Class weights to address class imbalance
    class_weight=class_weights_dict
)
```

Naive Baseline Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
```

```
plt.plot(naive_baseline_model_history.history['accuracy'], 'b-', label='Train Acc')
plt.plot(naive_baseline_model_history.history['val_accuracy'], 'r-', label='Val Acc')
plt.title("Naive Baseline Model Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()

# This predicts labels for validation set
naive_baseline_model_Y_pred = naive_baseline_model.predict(validation_set)
naive_baseline_model_y_pred = np.argmax(naive_baseline_model_Y_pred, axis=1)
naive_baseline_model_y_true = validation_set.classes
naive_baseline_model_class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
naive_baseline_model_confusion_matrix =
confusion_matrix(naive_baseline_model_y_true, naive_baseline_model_y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(naive_baseline_model_confusion_matrix, annot=True, fmt='d',
xticklabels=naive_baseline_model_class_labels,
yticklabels=naive_baseline_model_class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Naive Baseline Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Naive Baseline Scores")
print(classification_report(naive_baseline_model_y_true, naive_baseline_model_y_pred,
target_names=naive_baseline_model_class_labels,zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
naive_baseline_model_y_true_binarized = label_binarize(naive_baseline_model_y_true,
classes=[0, 1, 2])
naive_baseline_model_auc_roc = roc_auc_score(naive_baseline_model_y_true_binarized,
naive_baseline_model_Y_pred, average='macro', multi_class='ovr')
print("Naive Baseline AUC-ROC (macro average):", naive_baseline_model_auc_roc)
```

Baseline Model

```
baseline_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1)),

    # 32 filters with 3×3 kernel and ReLU activation
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
```

```
    # MaxPooling2D(2, 2) downsamples the image by a factor of 2×2
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 128 and adds L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 128 neurons
    layers.Dense(128, activation='relu'),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="baseline_model")

baseline_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

baseline_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=5, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('baseline_model.keras', save_best_only=True)

baseline_model_history = baseline_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # Number of Epochs
    epochs=30,
```

```
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint]
)
```

Baseline Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(baseline_model_history.history['accuracy'], 'b-', label='Train Acc')
plt.plot(baseline_model_history.history['val_accuracy'], 'r-', label='Val Acc')
plt.title("Baseline Model Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()

# This predicts labels for validation set
baseline_Y_pred = baseline_model.predict(validation_set)
baseline_y_pred = np.argmax(baseline_Y_pred, axis=1)
baseline_y_true = validation_set.classes
baseline_class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
baseline_confusion_matrix = confusion_matrix(baseline_y_true, baseline_y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(baseline_confusion_matrix, annot=True, fmt='d',
xticklabels=baseline_class_labels, yticklabels=baseline_class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Baseline Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Baseline Model Scores")
print(classification_report(baseline_y_true, baseline_y_pred,
target_names=baseline_class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
baseline_y_true_binarized = label_binarize(baseline_y_true, classes=[0, 1, 2])
baseline_auc_roc = roc_auc_score(baseline_y_true_binarized, baseline_Y_pred,
average='macro', multi_class='ovr')
print("Baseline AUC-ROC (macro average):", baseline_auc_roc)
```

Overfitting Model

```
# This is the overfitting model without dropout and regularization
```

```
overfitting_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1)),

    # 64 filters with 3×3 kernel and ReLU activation
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    # MaxPooling2D(2, 2) downsamples the image by a factor of 2×2
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 128
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # Filter is being increased to 256
    layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 512 neurons
    layers.Dense(512, activation='relu'),
    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="overfitting_model")

overfitting_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

overfitting_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=5, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('overfitting_model.keras', save_best_only=True)
```

```
overfitting_model_history = overfitting_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs changed fom 30 to 50 for overfitting
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping]
)
```

Overfitting Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(overfitting_model_history.history['accuracy'], 'b-', label='Train Acc')
plt.plot(overfitting_model_history.history['val_accuracy'], 'r-', label='Val Acc')
plt.title("Overfitting Model Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()

# This predicts labels for validation set
overfitting_Y_pred = overfitting_model.predict(validation_set)
overfitting_y_pred = np.argmax(overfitting_Y_pred, axis=1)
overfitting_y_true = validation_set.classes
overfitting_class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
overfitting_confusion_matrix = confusion_matrix(overfitting_y_true, overfitting_y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(overfitting_confusion_matrix, annot=True, fmt='d',
xticklabels=overfitting_class_labels, yticklabels=overfitting_class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Overfitting Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Overfitting Model Scores")
print(classification_report(overfitting_y_true, overfitting_y_pred,
target_names=overfitting_class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
```

```
overfitting_y_true_binarized = label_binarize(overfitting_y_true, classes=[0, 1, 2])
overfitting_auc_roc = roc_auc_score(overfitting_y_true_binarized, overfitting_Y_pred, average='macro', multi_class='ovr')
print("Overfitting Model AUC-ROC (macro average):", overfitting_auc_roc)
```

Stratified K-fold Cross Validation

```
# This is for numpy arrays for images and labels
data_dir = 'Combined_MiniDDSM_imgs'
classes = ['Benign', 'Cancer', 'Normal']
# img_height and img_width remains same, 224, 224, note to self: maybe increase them to cover the breast cancer areas?
img_height, img_width = 224, 224

X = []
y = []

# This loops through the class directory and load grayscale images
for idx, label in enumerate(classes):
    class_dir = os.path.join(data_dir, label)
    for file in tqdm(os.listdir(class_dir), desc=f"Loading {label}"):
        if file.endswith('.png'):
            # load grayscale images then resize them
            img = cv2.imread(os.path.join(class_dir, file), cv2.IMREAD_GRAYSCALE)
            img = cv2.resize(img, (img_width, img_height))

            # This normalizes pixel values to 0,1
            img = img.astype('float32') / 255.0

            # This adds image to feature list than add label index to label list
            X.append(img)
            y.append(idx)

# This adds channel dimension shape (N, 224, 224, 1) then converts the labels to numpy array: shape (N,)
X = np.expand_dims(np.array(X), -1)
y = np.array(y)  # Shape: (N,)
```

Stratified K-fold Cross Validation on Baseline Model

```
# This is the Class-wise stratified K-Fold cross-validator from sklearn
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# This clears the list before running folds, in case of re-running the folds, it will accumulate results across repeated runs if not cleared
fold_accuracies = []
fold_aucs = []
fold_losses = []
```

```
print(f"Cleared")
```

```
# This will start the K-Fold Cross Validation folding
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    print(f"\nFold {fold + 1}")

    # This splits the dataset using the fold indices
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    # This will 1-hot encode labels for the categorical crossentropy
    y_train_cat = to_categorical(y_train, num_classes=3)
    y_val_cat = to_categorical(y_val, num_classes=3)

    # Note that this is from the baseline model
    fold_model = models.Sequential([
        # Models input size
        layers.Input(shape=(img_height, img_width, 1)),

        # 32 filters with 3×3 kernel and ReLU activation
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        # MaxPooling2D(2, 2) downsamples the image by a factor of 2×2
        layers.MaxPooling2D(2, 2),

        # Filter is being increased to 64
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # Filter is being increased to 128 and adds L2 weight decay to penalize overly large weights and reduce overfitting
        layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
        layers.GlobalAveragePooling2D(),

        # Fully connected layer with 128 neurons
        layers.Dense(128, activation='relu'),
        # Randomly drops 50% of the nodes to minimize overfitting
        layers.Dropout(0.5),

        # Final layer for the 3-classes using softmax activation
        layers.Dense(3, activation='softmax')
    ])
```

```
    fold_model.compile(
        optimizer='adam',
        # categorical_crossentrop for multi-class classification with one-hot encoded labels
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Early stopping to stop training if no changes after consecutive epochs
    early_stopping = callbacks.EarlyStopping(patience=5, restore_best_weights=True)

    k_fold_baseline_history = fold_model.fit(
        # Training data
        X_train, y_train_cat,
        # Validation data
        validation_data=(X_val, y_val_cat),
        # Number of Epochs
        epochs=30,
        # Class weights to address class imbalance
        class_weight=class_weights_dict,
        batch_size=32,
        callbacks=[early_stopping],
        verbose=1
    )

    # This is the evaluation section
    val_loss, val_accuracy = fold_model.evaluate(X_val, y_val_cat, verbose=0)
    # This savea accuracy and loss
    fold_accuracies.append(val_accuracy)
    fold_losses.append(val_loss)

    # This predicts the class probabilities for AUC-ROC
    y_val_pred_probs = fold_model.predict(X_val)

    # This binarizes ground truth labels then compute AUC-ROC
    y_val_bin = label_binarize(y_val, classes=[0, 1, 2])
    auc = roc_auc_score(y_val_bin, y_val_pred_probs, average='macro', multi_class='ovr')
    fold_aucs.append(auc)

    # This print out a result summary of each fold
    print(f"Fold {fold + 1} results summary:")
    print(f"Accuracy: {val_accuracy:.4f}, Loss: {val_loss:.4f}, AUC-ROC: {auc:.4f}")
```

Stratified K-fold Cross Validation Baseline Model Result Summary

```
print("Stratified K-Fold Cross-Validation Baseline Result Sumary")
print(f"Mean Accuracy: {np.mean(fold_accuracies):.4f} ± {np.std(fold_accuracies):.4f}")
print(f"Mean Loss: {np.mean(fold_losses):.4f} ± {np.std(fold_losses):.4f}")
print(f"Mean AUC-ROC: {np.mean(fold_aucs):.4f} ± {np.std(fold_aucs):.4f}")
```

Stratified K-fold Cross Validation for Regularized Model

```
# This is the Class-wise stratified K-Fold cross-validator from sklearn
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# This clears the list before running folds, in case of re-running the folds, it will accumulate results across repeated runs if not cleared
fold_accuracies = []
fold_aucs = []
fold_losses = []

print(f"Cleared")
```

```
# This will start the K-Fold Cross Validation folding
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    print(f"\nFold {fold + 1}")

    # This splits the dataset using the fold indices
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    # This will 1-hot encode labels for the categorical crossentropy
    y_train_cat = to_categorical(y_train, num_classes=3)
    y_val_cat = to_categorical(y_val, num_classes=3)

    # regularized model layers
    fold_model = models.Sequential([
        # Models input size
        layers.Input(shape=(img_height, img_width, 1), name='input_layer'),

        # 32 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
        layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),
        # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
        layers.SpatialDropout2D(0.2),

        # Filter is being increased to 64
        layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),
        layers.SpatialDropout2D(0.2),
```

```
        # Filter is being increased to 128
        layers.Conv2D(128, (3, 3), activation='relu', padding='same',
kernel_regularizer=regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),
        layers.SpatialDropout2D(0.3),

        # This replaces Flatten, averaging the spatial dimensions and reducing the number of
parameters
        layers.GlobalAveragePooling2D(),

        # Fully connected layer with 128 neurons
        layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
        # Randomly drops 50% of the nodes to minimize overfitting
        layers.Dropout(0.5),

        # Final layer for the 3-classes using softmax activation
        layers.Dense(3, activation='softmax')
    ])

    fold_model.compile(
        optimizer='adam',
        # categorical_crossentrop for multi-class classification with one-hot encoded labels
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Early stopping to stop training if no changes after consecutive epochs
    early_stopping = callbacks.EarlyStopping(patience=5, restore_best_weights=True)

    k_fold_regularized_history = fold_model.fit(
        # Training data
        X_train, y_train_cat,
        # Validation data
        validation_data=(X_val, y_val_cat),
        # Number of Epochs
        epochs=30,
        # Class weights to address class imbalance
        class_weight=class_weights_dict,
        batch_size=32,
        callbacks=[early_stopping],
        verbose=1
    )

    # This is the evaluation section
    val_loss, val_accuracy = fold_model.evaluate(X_val, y_val_cat, verbose=0)
    # This savea accuracy and loss
    fold_accuracies.append(val_accuracy)
    fold_losses.append(val_loss)
```

```
    # This predicts the class probabilities for AUC-ROC
    y_val_pred_probs = fold_model.predict(X_val)

    # This binarizes ground truth labels then compute AUC-ROC
    y_val_bin = label_binarize(y_val, classes=[0, 1, 2])
    auc = roc_auc_score(y_val_bin, y_val_pred_probs, average='macro', multi_class='ovr')
    fold_aucs.append(auc)

    # This print out a result summary of each fold
    print(f"Fold {fold + 1} results summary:")
    print(f"Accuracy: {val_accuracy:.4f}, Loss: {val_loss:.4f}, AUC-ROC: {auc:.4f}")
```

Stratified K-fold Cross Validation Regularized Model Result Summary

```
print("Stratified K-Fold Cross-Validation Regularized Model Result Sumary")
print(f"Mean Accuracy: {np.mean(fold_accuracies):.4f} ± {np.std(fold_accuracies):.4f}")
print(f"Mean Loss: {np.mean(fold_losses):.4f} ± {np.std(fold_losses):.4f}")
print(f"Mean AUC-ROC: {np.mean(fold_aucs):.4f} ± {np.std(fold_aucs):.4f}")
```

Regularized Model

```
regularized_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1), name='input_layer'),

    # 32 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 128
    layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),
```

```
    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 128 neurons
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="regularized_model")

regularized_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

regularized_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('regularized_model.keras', save_best_only=True)

regularized_model_history = regularized_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs remain at 50 from overfitting model
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Regularized Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(regularized_model_history.history['accuracy'], 'b-', label='Train Accuracy')
plt.plot(regularized_model_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Regularized Model Training and Validation Accuracy')
plt.xlabel('Epochs')
```

```
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
Y_pred_probs = regularized_model.predict(validation_set)
y_pred = np.argmax(Y_pred_probs, axis=1)
y_true = validation_set.classes
class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Regularized Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Regularized Model Scores")
print(classification_report(y_true, y_pred, target_names=class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_bin = label_binarize(y_true, classes=[0, 1, 2])
auc_roc = roc_auc_score(y_true_bin, Y_pred_probs, average='macro', multi_class='ovr')
# changed to .4f for easier reading, too many nums
print(f"Regularized Model AUC-ROC (macro average): {auc_roc:.4f}")
```

Wider Model

```
wider_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1), name='input_layer'),

    # 64 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 128
```

```
    layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 256
    layers.Conv2D(256, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 256 neurons
    layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="wider_model")

wider_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

wider_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('wider_model.keras', save_best_only=True)

wider_model_history = wider_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs remain at 50 from overfitting model
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
```

```
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Wider Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(wider_model_history.history['accuracy'], 'b-', label='Train Accuracy')
plt.plot(wider_model_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Wider Model Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
Y_pred_probs = wider_model.predict(validation_set)
y_pred = np.argmax(Y_pred_probs, axis=1)
y_true = validation_set.classes
class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels,
cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Wider Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Wider Model Scores")
print(classification_report(y_true, y_pred, target_names=class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_bin = label_binarize(y_true, classes=[0, 1, 2])
auc_roc = roc_auc_score(y_true_bin, Y_pred_probs, average='macro', multi_class='ovr')
print(f"Wider Model AUC-ROC (macro average): {auc_roc:.4f}")
```

Deeper Model

```
deeper_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1), name='input_layer'),
```

```
    # 32 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
    layers.SpatialDropout2D(0.2),

    # This is an additional filter layer for increased depth
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),

    # This is an additional filter layer for increased depth
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # Filter is being increased to 128
    layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # This is an additional filter layer for increased depth
    layers.Conv2D(128, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 128 neurons
```

```
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="deeper_model")

deeper_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

deeper_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('deeper_model.keras', save_best_only=True)

deeper_model_history = deeper_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs remain at 50
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Deeper Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(deeper_model_history.history['accuracy'], 'b-', label='Train Accuracy')
plt.plot(deeper_model_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Deeper Model Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# This predicts labels for validation set
Y_pred_probs = deeper_model.predict(validation_set)
y_pred = np.argmax(Y_pred_probs, axis=1)
y_true = validation_set.classes
class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Deeper Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Deeper Model Scores")
print(classification_report(y_true, y_pred, target_names=class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_bin = label_binarize(y_true, classes=[0, 1, 2])
auc_roc = roc_auc_score(y_true_bin, Y_pred_probs, average='macro', multi_class='ovr')
print(f"Deeper Model AUC-ROC (macro average): {auc_roc:.4f}")
```

Narrower Model

```
narrower_model = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1), name='input_layer'),

    # 16 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(16, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 32
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),
```

```
    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 64 neurons
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')
], name="narrower_model")

narrower_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

narrower_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('narrower_model.keras', save_best_only=True)

narrower_model_history = narrower_model.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs remain at 50
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Narrower Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(narrower_model_history.history['accuracy'], 'b-', label='Train Accuracy')
plt.plot(narrower_model_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Narrower Model Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
Y_pred_probs = narrower_model.predict(validation_set)
y_pred = np.argmax(Y_pred_probs, axis=1)
y_true = validation_set.classes
class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels,
cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Narrower Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Narrower Model Scores")
print(classification_report(y_true, y_pred, target_names=class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_bin = label_binarize(y_true, classes=[0, 1, 2])
auc_roc = roc_auc_score(y_true_bin, Y_pred_probs, average='macro', multi_class='ovr')
print(f"Narrower Model AUC-ROC (macro average): {auc_roc:.4f}")
```

Narrower Model Two

```
# 2k (QHD) resolution: 2560 x 1440
img_height, img_width = 2560, 1440
```

```
narrower_model_two = models.Sequential([
    # Models input size
    layers.Input(shape=(img_height, img_width, 1), name='input_layer'),
```

```
    # 16 filters with 3×3 kernel and ReLU activation and L2 weight decay to penalize overly large weights and reduce overfitting
    layers.Conv2D(16, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    # This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 32
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.2),

    # Filter is being increased to 64
    layers.Conv2D(64, (3, 3), activation='relu', padding='same', kernel_regularizer=regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2, 2),
    layers.SpatialDropout2D(0.3),

    # This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
    layers.GlobalAveragePooling2D(),

    # Fully connected layer with 64 neurons
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    # Randomly drops 50% of the nodes to minimize overfitting
    layers.Dropout(0.5),

    # Final layer for the 3-classes using softmax activation
    layers.Dense(3, activation='softmax')  # 3 classes: Benign, Cancer, Normal
], name="narrower_model_two")

narrower_model_two.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

narrower_model_two.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
```

```
checkpoint = callbacks.ModelCheckpoint('narrower_model_two.keras', save_best_only=True)

narrower_model_two_history = narrower_model_two.fit(
    # Training data
    training_set,
    # Validation data
    validation_data=validation_set,
    # epochs remain at 50
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Narrower Model Two Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(narrower_model_two_history.history['accuracy'], 'b-', label='Train Accuracy')
plt.plot(narrower_model_two_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Narrower Model Two Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
Y_pred_probs_two = narrower_model_two.predict(validation_set)
y_pred_two = np.argmax(Y_pred_probs_two, axis=1)
y_true_two = validation_set.classes
class_labels = list(validation_set.class_indices.keys())

# Confusion Matrix
cm_two = confusion_matrix(y_true_two, y_pred_two)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_two, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Narrower Model Two Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Narrower Model Two Scores")
```

```
print(classification_report(y_true_two, y_pred_two, target_names=class_labels, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_two_bin = label_binarize(y_true_two, classes=[0, 1, 2])
auc_roc_two = roc_auc_score(y_true_two_bin, Y_pred_probs_two, average='macro', multi_class='ovr')
print(f"Narrower Model Two AUC-ROC (macro average): {auc_roc_two:.4f}")
```

Transfer Learning Model

```
# Gonna use EfficientNetB0, https://keras.io/api/applications/efficientnet/
transfer_train_datagen = ImageDataGenerator(
    # This rescales the pixel values to 0,1
    rescale=1./255,
    # This applies augmentation
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    vertical_flip=True,
    # This is 80 train 20 validation split
    validation_split=0.2
)

# This is the training generator
# color_mode='rgb' is to convert grayscale to RGB for the EfficientNet
transfer_training_set = transfer_train_datagen.flow_from_directory(
    combined_img_dir, target_size=(224, 224), color_mode='rgb', batch_size=batch_size, class_mode='categorical', subset='training', shuffle=True
)

# This is the validation generator
# color_mode='rgb' is to convert grayscale to RGB for the EfficientNet
transfer_validation_set = transfer_train_datagen.flow_from_directory(
    combined_img_dir, target_size=(224, 224), color_mode='rgb', batch_size=batch_size, class_mode='categorical', subset='validation', shuffle=False
)

# This loads the EfficientNetB0 model with pretrain on 'imagenet' and remove default dense layers with include_top=False
base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# This is to use pre-trained feature extraction
base_model.trainable = False

# Models input size
```

```
inputs = Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
# This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
x = GlobalAveragePooling2D()(x)
# Fully connected layer with 128 neurons
x = Dense(128, activation='relu')(x)
# Randomly drops 50% of the nodes to minimize overfitting
x = Dropout(0.5)(x)
# Final layer for the 3-classes using softmax activation
outputs = Dense(3, activation='softmax')(x)

transfer_learning_model = Model(inputs, outputs, name="transfer_learning_model")

transfer_learning_model.compile(
    optimizer='adam',
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

transfer_learning_model.summary
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('transfer_learning_model.keras', save_best_only=True)

transfer_learning_model_history = transfer_learning_model.fit(
    # Training data
    transfer_training_set,
    # Validation data
    validation_data=transfer_validation_set,
    # epochs remain at 50
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint],
    verbose=1
)
```

Transfer Learning Model Evaluation

```
# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(transfer_learning_model_history.history['accuracy'], 'b-', label='Train Accuracy')
```

```
plt.plot(transfer_learning_model_history.history['val_accuracy'], 'r-', label='Validation Accuracy')
plt.title('Transfer Learning Model Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
Y_pred_probs_transfer = transfer_learning_model.predict(transfer_validation_set)
y_pred_transfer = np.argmax(Y_pred_probs_transfer, axis=1)
y_true_transfer = transfer_validation_set.classes
class_labels_transfer = list(transfer_validation_set.class_indices.keys())

# Confusion Matrix
cm_transfer = confusion_matrix(y_true_transfer, y_pred_transfer)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_transfer, annot=True, fmt='d', xticklabels=class_labels_transfer, yticklabels=class_labels_transfer, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Transfer Learning Model Confusion Matrix (Validation Set)')
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Transfer Learning Model Scores")
print(classification_report(y_true_transfer, y_pred_transfer, target_names=class_labels_transfer, zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_transfer_bin = label_binarize(y_true_transfer, classes=[0, 1, 2])
auc_roc_transfer = roc_auc_score(y_true_transfer_bin, Y_pred_probs_transfer, average='macro', multi_class='ovr')
print(f"Transfer Learning Model AUC-ROC (macro average): {auc_roc_transfer:.4f}")
```

Narrower and Transfer Learning Multi-Branch Model

```
# This is the input layer with 4 channels that includse the images for the 3 classes and 1 for the mask
inputs_4ch = Input(shape=(224, 224, 4), name='input_layer')
# This is to extract only the channels for EfficientNet
inputs_3ch = Lambda(lambda x: x[:, :, :, :3], name='slice_rgb')(inputs_4ch)
```

```
# This is the data generator to load grayscale image and masks and combine them into a 4 channel input
class CombinedImageMaskGenerator(Sequence):
```

```
def __init__(self, df, batch_size, num_classes, shuffle=True):
    self.df = df.reset_index(drop=True)
    self.batch_size = batch_size
    self.num_classes = num_classes
    self.shuffle = shuffle
    self.on_epoch_end()

def __len__(self):
    return int(np.ceil(len(self.df) / self.batch_size))

def on_epoch_end(self):
    self.indices = np.arange(len(self.df))
    if self.shuffle:
        np.random.shuffle(self.indices)

def __getitem__(self, idx):
    batch_indices = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
    batch_x = []
    batch_y = []

    for i in batch_indices:
        row = self.df.iloc[i]
        image_path = row['image_path']
        mask_path = row['mask_path']
        label = row['label']

        # This loads grayscale images and normalize to 0, 1
        image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
        image = cv2.resize(image, (224, 224)) / 255.0

        if pd.notna(mask_path) and os.path.exists(mask_path):
            mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
            mask = cv2.resize(mask, (224, 224)) / 255.0
        else:
            mask = np.zeros_like(image)

        # This will stack the 3 class channels of images and the 1 channel of mask
        combined_input = np.stack([image, image, image, mask], axis=-1)  # (224,224,4)
        batch_x.append(combined_input)

        # This maps label strings to a class index then one-hot encode
        if label == 'Benign':
            y = 0
        elif label == 'Cancer':
            y = 1
        elif label == 'Normal':
            y = 2
        else:
            raise ValueError(f"Unknown label {label}")
```

```
            y_onehot = tf.keras.utils.to_categorical(y, num_classes=self.num_classes)
            batch_y.append(y_onehot)

        return np.array(batch_x, dtype=np.float32), np.array(batch_y, dtype=np.float32)
```

```
# This reloads the CSV created earlier in Loading and Preparing Dataset
df = pd.read_csv("Combined_MiniDDSM_Metadata.csv")
df_train, df_val = train_test_split(df, test_size=0.2, stratify=df['label'], random_state=42)

# This instantiates the data generator for training and validation
batch_size = 16
train_gen = CombinedImageMaskGenerator(df_train, batch_size=batch_size, num_classes=3)
val_gen = CombinedImageMaskGenerator(df_val, batch_size=batch_size, num_classes=3, shuffle=False)
```

```
# This is branch 1
# referenced from earlier Narrower Model for Benign/Normal
# 16 filters with 3×3 kernel and ReLU activation
b1 = Conv2D(16, (3, 3), activation='relu', padding='same')(inputs_4ch)
b1 = BatchNormalization()(b1)
b1 = MaxPooling2D(2, 2)(b1)
# This SpatialDropout2D is added, increasingly as well, to each pooling layer to encourage robustness in spatial feature learning
b1 = SpatialDropout2D(0.2)(b1)

# Filter is being increased to 32
b1 = Conv2D(32, (3, 3), activation='relu', padding='same')(b1)
b1 = BatchNormalization()(b1)
b1 = MaxPooling2D(2, 2)(b1)
b1 = SpatialDropout2D(0.2)(b1)

# Filter is being increased to 64
b1 = Conv2D(64, (3, 3), activation='relu', padding='same')(b1)
b1 = BatchNormalization()(b1)
b1 = MaxPooling2D(2, 2)(b1)
b1 = SpatialDropout2D(0.3)(b1)

# This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
b1 = GlobalAveragePooling2D()(b1)

# This is branch 2
# referenced from earlier transfer Learning with EfficientNetB0 for Cancer
# Load pre-trained EfficientNetB0 for transfer learning on RGB channels
base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
```

```
# This freezes the layers during phase 1 training so that the layers weights wont update during back propagation
base_model.trainable = False

# This applies model to 3 channel input
b2 = base_model(inputs_3ch, training=False)

# This replaces Flatten, averaging the spatial dimensions and reducing the number of parameters
b2 = GlobalAveragePooling2D()(b2)

# This combines the outputs from both branches
combined = Concatenate()([b1, b2])

# Fully connected layer with 128 neurons
combined = Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001))(combined)
# Randomly drops 50% of the nodes to minimize overfitting
combined = Dropout(0.5)(combined)

# Final layer for the 3-classes using softmax activation
outputs = Dense(3, activation='softmax')(combined)

narrower_and_transfer_learning_model = Model(inputs_4ch, outputs, name="narrower_and_transfer_learning_model")

# Compile
narrower_and_transfer_learning_model.compile(
    optimizer=Adam(learning_rate=1e-4),
    # categorical_crossentrop for multi-class classification with one-hot encoded labels
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

narrower_and_transfer_learning_model.summary()
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stopping = callbacks.EarlyStopping(patience=8, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint('narrower_and_transfer_learning_model.keras', save_best_only=True)
# Added reduce_lr to reduce learning rate if the validation loss doesnt improve after consecutive epochs
reduce_lr = callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4, verbose=1)

# This is the phase 1 training with EfficientNetB0 frozen
history_phase1 = narrower_and_transfer_learning_model.fit(
    # Training data
```

```
    train_gen,
    # Validation data
    validation_data=val_gen,
    # epochs
    epochs=20,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint, reduce_lr],
    verbose=1
)

# This will unfreeze top 30% of EfficientNetB0 layers for fine-tuning
fine_tune_at = int(len(base_model.layers) * 0.7)
for layer in base_model.layers[fine_tune_at:]:
    layer.trainable = True

# This recompiles with a lower learning rate to help mitigate over updating pre-trained
weights
narrower_and_transfer_learning_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# This continues the training from where phase 1 stopped
history_phase2 = narrower_and_transfer_learning_model.fit(
    # Training data
    train_gen,
    # Validation data
    validation_data=val_gen,
    # epochs
    initial_epoch=history_phase1.epoch[-1] + 1,
    epochs=50,
    # Class weights to address class imbalance
    class_weight=class_weights_dict,
    callbacks=[early_stopping, checkpoint, reduce_lr],
    verbose=1
)
```

Narrower and Transfer Learning Multi-Branch Model Evaluation

```
history_combined = {}
for key in ['accuracy', 'val_accuracy', 'loss', 'val_loss']:
    history_combined[key] = history_phase1.history[key] + history_phase2.history[key]

# Accuracy
plt.figure(figsize=(12, 5))
plt.plot(history_combined['accuracy'], 'b-', label='Train Accuracy')
plt.plot(history_combined['val_accuracy'], 'r-', label='Validation Accuracy')
```

```
plt.title('Narrower and Transfer Learning Multi-Branch Model: Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# This predicts labels for validation set
label_mapping = {'Benign': 0, 'Cancer': 1, 'Normal': 2}
y_true_natl = df_val['label'].map(label_mapping).values
Y_pred_probs_natl = narrower_and_transfer_learning_model.predict(val_gen, verbose=1)
y_pred_natl = np.argmax(Y_pred_probs_natl, axis=1)
class_labels_natl = ['Benign', 'Cancer', 'Normal']

# Confusion Matrix
cm_natl = confusion_matrix(y_true_natl, y_pred_natl)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_natl, annot=True, fmt='d', xticklabels=class_labels_natl,
yticklabels=class_labels_natl, cmap='YlGnBu')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Narrower and Transfer Learning Multi-Branch Model: Confusion Matrix')
plt.tight_layout()
plt.show()

# This prints classification Report for Precision, Sensitivity (Recall), F1-score
print("Narrower and Transfer Learning Multi-Branch Model Scores:")
print(classification_report(y_true_natl, y_pred_natl, target_names=class_labels_natl,
zero_division=0))

# Area Under the ROC Curve (AUC-ROC)
y_true_natl_bin = label_binarize(y_true_natl, classes=[0, 1, 2])
auc_roc_natl = roc_auc_score(y_true_natl_bin, Y_pred_probs_natl, average='macro',
multi_class='ovr')
print(f"Narrower and Transfer Learning Multi-Branch Model AUC-ROC (macro average):
{auc_roc_natl:.4f}")
```

Modified U-Net Model

```
# This is the image patch size
PATCH_SIZE = 256
# Stride controls the overlap between patches, since 128 is half of 256, the patches overlaps
50% between each other
STRIDE = 128
# This is the diemnsion the images and masks resize to before patching
RESIZE_DIM = 512
# This is the minimum foreground ratio. Note that the ratio is set extremely low to keep more
patches
```

```
min_fg_ratio = 0.0005

# This is to store extracted image and mask patches
img_patches, mask_patches = [], []
# Yes i know, i did this before, but its a new day, new model, besides, better to reinitialize
than to spend time finding a 'could be a prob'
df = pd.read_csv("Combined_MiniDDSM_Metadata.csv")

# This hops through each row in the csv
for idx in range(len(df)):
    img_path = df.loc[idx, 'image_path']
    mask_path = df.loc[idx, 'mask_path']

    if pd.notna(mask_path) and os.path.exists(mask_path):
        # This Loads the grayscale images and masks
        img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

        # This is image resizing and normalization
        img = cv2.resize(img, (RESIZE_DIM, RESIZE_DIM)).astype(np.float32) / 255.0
        mask = cv2.resize(mask, (RESIZE_DIM, RESIZE_DIM)).astype(np.float32) / 255.0

        # This is/are temporary lists for patches of the current image
        img_p, mask_p = [], []

        # Similar to patch extraction, slide a window of patch_size across an image with the
declared stride
        h, w = img.shape
        for y in range(0, h - PATCH_SIZE + 1, STRIDE):
            for x in range(0, w - PATCH_SIZE + 1, STRIDE):
                img_patch = img[y:y+PATCH_SIZE, x:x+PATCH_SIZE]
                mask_patch = mask[y:y+PATCH_SIZE, x:x+PATCH_SIZE]

                # This calculates the foreground ratio and keeps the patch if it has sufficient lesion
area
                fg_ratio = np.mean(mask_patch > 0.1)
                print(f"Patch ({y},{x}) fg_ratio: {fg_ratio:.6f}")
                if fg_ratio > min_fg_ratio:
                    img_p.append(img_patch)
                    mask_p.append(mask_patch)

        # This appends valid patches from the image to the main list
        img_patches.extend(img_p)
        mask_patches.extend(mask_p)
        print(f"Image {idx} produced {len(img_p)} patches.")

print(f"Extracted {len(img_patches)} patches with sufficient lesions.")
```

```
# This cell is for the Laplacian filter
```

```
def apply_laplacian(image):
    # This is to apply histogram equalization to enhance contrast where images must be uint8
    image = cv2.equalizeHist((image * 255).astype(np.uint8))

    # This applies the Laplacian filter to detect edges
    laplacian = cv2.Laplacian(image, cv2.CV_64F)

    # The result is converted to 8-bit absolute scale then normalized to 0,1 for model input
    laplacian = cv2.convertScaleAbs(laplacian)
    laplacian = laplacian.astype(np.float32) / 255.0
    return Laplacian
```

```
# This applies Laplacian edge enhancement to the image patches and convert to type float32
laplacian_patches = np.array([apply_laplacian(p) for p in img_patches], dtype=np.float32)[..., np.newaxis]

# This expands the dimensions of mask patches to shape H, W, 1 for model compatibility
mask_patches = np.array(mask_patches)[..., np.newaxis]

# This splits the data into 80% training and 20% validation
X_train, X_val, y_train, y_val = train_test_split( laplacian_patches, mask_patches, test_size=0.2, random_state=42)

print(f"Train patches: {X_train.shape}, Val patches: {X_val.shape}")
```

```
# This is for metrics, Dice Loss metric
def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)
```

```
# This is to help define two convolutional layers with batch normalization and dropout
def conv_block(x, filters, dropout=0.2):
    x = layers.Conv2D(filters, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(filters, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(dropout)(x)
    return x

def Modified_UNet(input_shape=(PATCH_SIZE, PATCH_SIZE, 1)):
    inputs = layers.Input(shape=input_shape)
```

```
    # This is encoder or contracting path with increasing filters 16, 32, 64, 128, bottleneck
256
    conv_block_1 = conv_block(inputs, 16)
    pool_1 = layers.MaxPooling2D((2, 2))(conv_block_1)
    conv_block_2 = conv_block(pool_1, 32)
    pool_2 = layers.MaxPooling2D((2, 2))(conv_block_2)
    conv_block_3 = conv_block(pool_2, 64)
    pool_3 = layers.MaxPooling2D((2, 2))(conv_block_3)
    conv_block_4 = conv_block(pool_3, 128)
    pool_4 = layers.MaxPooling2D((2, 2))(conv_block_4)
    conv_block_5 = conv_block(pool_4, 256)

    # This is decoder or expanding path with decreasing filters 128, 64, 32, 16
    upconv_6 = layers.Conv2DTranspose(128, (2, 2), strides=(2, 2),
padding='same')(conv_block_5)
    concat_6 = layers.Concatenate()([upconv_6, conv_block_4])
    conv_block_6 = conv_block(concat_6, 128)
    upconv_7 = layers.Conv2DTranspose(64, (2, 2), strides=(2, 2),
padding='same')(conv_block_6)
    concat_7 = layers.Concatenate()([upconv_7, conv_block_3])
    conv_block_7 = conv_block(concat_7, 64)
    upconv_8 = layers.Conv2DTranspose(32, (2, 2), strides=(2, 2),
padding='same')(conv_block_7)
    concat_8 = layers.Concatenate()([upconv_8, conv_block_2])
    conv_block_8 = conv_block(concat_8, 32)
    upconv_9 = layers.Conv2DTranspose(16, (2, 2), strides=(2, 2),
padding='same')(conv_block_8)
    concat_9 = layers.Concatenate()([upconv_9, conv_block_1])
    conv_block_9 = conv_block(concat_9, 16)

    # This is the output layer with sigmoid activation for binary segmentation
    outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(conv_block_9)

    return models.Model(inputs, outputs, name="modified_u_net_model")

model = Modified_UNet()
model.compile(
    optimizer=optimizers.Adam(learning_rate=1e-4),
    # This is Dice loss for segmentation
    loss=dice_loss,
    metrics=['accuracy', dice_coef]
)

model.summary()
```

```
# This is the data generator for patch-based image and mask training
class PatchDataGenerator(Sequence):
    # This initializes the generator with image and mask patches
    def __init__(self, img_patches, mask_patches, batch_size=8, shuffle=True):
```

```
        self.img_patches = img_patches
        self.mask_patches = mask_patches
        self.batch_size = batch_size
        self.shuffle = shuffle
        self.indices = np.arange(len(self.img_patches))
        self.on_epoch_end()

    def __len__(self):
        # This returns the number of batches per epoch
        return int(np.ceil(len(self.img_patches) / self.batch_size))

    def __getitem__(self, idx):
        # This gets the indices for the current batch and extracts them
        indices = self.indices[idx * self.batch_size : (idx + 1) * self.batch_size]
        batch_x = [self.img_patches[i] for i in indices]
        batch_y = [self.mask_patches[i] for i in indices]

        # This converts them to arrays and add channel dimension
        batch_x = np.array(batch_x, dtype=np.float32)[..., np.newaxis]
        batch_y = np.array(batch_y, dtype=np.float32)[..., np.newaxis]
        return batch_x, batch_y

    # This shuffles the dataset indices after each epoch
    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)
```

```
# Early stopping to stop training if no changes after consecutive epochs
early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
# checkpoint acts as a save file to restore best model weights from training
checkpoint = callbacks.ModelCheckpoint("modified_u_net_model.keras", save_best_only=True)
# Added reduce_lr to reduce learning rate if the validation loss doesnt improve after consecutive epochs
reduce_lr = callbacks.ReduceLROnPlateau(monitor='val_loss', patience=5, factor=0.5, verbose=1)

# This creates data generators for training and validation
train_gen = PatchDataGenerator(X_train[...,0], y_train[...,0], batch_size=8, shuffle=True)
val_gen = PatchDataGenerator(X_val[...,0], y_val[...,0], batch_size=8, shuffle=False)

history = model.fit(
    # Training data
    train_gen,
    # Validation data
    validation_data=val_gen,
    # epochs
    epochs=50,
```

```
    callbacks=[early_stop, checkpoint, reduce_lr],
    verbose=1
)
```

Modified U-Net Model Evaluation

```
# This section computes the metrics
# This applies threshold to binarize predictions and flattens
y_pred = (model.predict(X_val, batch_size=8) > 0.5).astype(np.uint8)
y_true_flat = (y_val > 0.5).astype(np.uint8).flatten()
y_pred_flat = y_pred.flatten()

# This is the computation followed by the results printed out
precision = precision_score(y_true_flat, y_pred_flat, zero_division=0)
recall = recall_score(y_true_flat, y_pred_flat, zero_division=0)
f1 = f1_score(y_true_flat, y_pred_flat, zero_division=0)
jaccard = jaccard_score(y_true_flat, y_pred_flat, zero_division=0)
accuracy = accuracy_score(y_true_flat, y_pred_flat)
dice = (2 * np.sum(y_true_flat * y_pred_flat) + 1) / (np.sum(y_true_flat) +
np.sum(y_pred_flat) + 1)

print(f"\nModified U-Net Model Evaluation Results:")
print(f"Precision: {precision:.4f}")
print(f"Recall (Sensitivity): {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Jaccard Index (IoU): {jaccard:.4f}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Dice Score: {dice:.4f}")

# This is a little visualization to view sample predictions
# 3 samples are being picked and 3 images of each sample are showned, input image, ground
truth mask, and predicted mask
for idx in np.random.choice(len(X_val), 3, replace=False):
    plt.figure(figsize=(12,4))

    plt.subplot(1,3,1)
    plt.imshow(X_val[idx].squeeze(), cmap='gray')
    plt.title("Input Image (Laplacian)")

    plt.subplot(1,3,2)
    plt.imshow(y_val[idx].squeeze(), cmap='gray')
    plt.title("Ground Truth Mask")

    plt.subplot(1,3,3)
    plt.imshow(y_pred[idx].squeeze(), cmap='gray')
    plt.title("Predicted Mask")

    plt.show()
```

.gitignore

```
The Complete Mini-DDSM archive/
Combined_MiniDDSM_imgs/
Combined_MiniDDSM_img_Masks/
```

app.py

```
'''
pip install flask tensorflow==2.19.0 keras==3.10.0

python app.py

http://127.0.0.1:5000

'''

import os
import numpy as np
import cv2
from flask import Flask, render_template, request
from tensorflow.keras.layers import Lambda
from tensorflow.keras.models import load_model

# This line suppresses tensorflow warnings, used in debugging due to py, tf, and keras mismatch
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

# This is the models path, take note that lambda was used
MODEL_PATH = "narrower_and_transfer_learning_model.keras"
model = load_model(MODEL_PATH, custom_objects={'Lambda': Lambda}, compile=False, safe_mode=False)

CLASS_NAMES = ['Benign', 'Cancer', 'Normal']

# This are templates and static folders according to flask app templates and examples
app = Flask(__name__, template_folder='templates', static_folder='static')
UPLOAD_FOLDER = os.path.join('static', 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

def preprocess_image(image_path):
    # This right here preprocess uploaded image into (224,224,4) format
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (224, 224)) / 255.0
    mask = np.zeros_like(img)
```

```
    # This stacks 3 grayscale channels and a mask
    combined_input = np.stack([img, img, img, mask], axis=-1)
    return np.expand_dims(combined_input, axis=0).astype(np.float32)

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        file = request.files.get("file")
        if not file or file.filename == "":
            return render_template("index.html", error="No file uploaded.")

            # Save uploaded file into static/uploads for data gathering and future model improvements if needed.
        filename = file.filename
        img_path = os.path.join(UPLOAD_FOLDER, filename)
        file.save(img_path)

        input_tensor = preprocess_image(img_path)
        preds = model.predict(input_tensor, verbose=0)[0]

        # This section maps the results
            results = {CLASS_NAMES[i]: round(float(preds[i] * 100), 2) for i in range(len(CLASS_NAMES))}
        top_class = CLASS_NAMES[np.argmax(preds)]
        top_score = results[top_class]

        uploaded_image = f"{filename}"

        return render_template(
            "index.html",
            uploaded_image=uploaded_image,
            results=results,
            top_class=top_class,
            top_score=top_score
        )

    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True, use_reloader=False)
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Breast Cancer Predictor</title>
```

```
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    <div class="container">
        <h1>Breast Cancer Predictor</h1>

        <p style="margin-left: 2.5%;">This is a deep learning app to predict breast cancer using x-ray mammography images.</p>
        <p style="margin-left: 2.5%; margin-bottom: -0.5%;">How to use:</p>
         <p style="margin-left: 2.5%; margin-bottom: -0.5%;">Step 1: Upload an image by clicking on "Upload Image".</p>
        <p style="margin-left: 2.5%;">Step 2: Click on "Predict" and wait for results.</p>

        <div class="alert">
            ⚠ Important: This app is not a substitute for clinical decision-making and should consult healthcare specialists for more information.
        </div>

        <div class="main-body-container">
            <div class="left-section">
                <form method="POST" enctype="multipart/form-data" class="upload-form">
                    <input type="file" name="file" accept="image/*" id="actual-upload-img-btn" hidden />
                    <label for="actual-upload-img-btn" class="for-actual-upload-img-btn">Upload Image</label>
                    <button type="submit">Predict</button>
                </form>
            </div>
            <div class="right-section">
                <p>Uploaded File: <span id="img-file-name-display"></span></p>
            </div>
        </div>

        {% if uploaded_image %}
        <div class="main-body-container">
            <div class="left-section">
                <div class="image-frame">
                    <img src="{{ url_for('static', filename='uploads/' + uploaded_image) }}" alt="Uploaded Image" class="uploaded-img">
                </div>
            </div>
            <div class="right-section">
                <h3>Predictions:</h3>
                {% for cls, prob in results.items() %}
                    <div class="progress-bar">
                        <div class="fill">{{ "%.1f"|format(prob) }}%</div>
                        <span>{{ cls }}</span>
                    </div>
                {% endfor %}
```

```
                                    <h2 class="likely">Likely {{ top_class }} ({{ "%.1f"|format(top_score) }}%)</h2>

            <div class="disclaimer">
              <h4>Disclaimer & Terms</h4>
             <p>This application is meant solely for educational, research, and demonstration use. It is not a medical instrument, and its forecasts should not be viewed as a clinical assessment. By utilizing this tool, you agree that:</p>
              <ul>
                  <li>The outcomes are produced by an AI system trained on historical data and might not apply to all cases.</li><br>
                      <li>No medical or personal information is kept or transmitted externally.</li><br>
                 <li>You remain fully responsible for any decisions made based on the results of this application.</li>
              </ul>
            </div>
          </div>
        </div>
        {% endif %}

        <footer>
          <div>
                <p>Documentation: https://github.com/WZhengJie99/Deep-Learning-Breast-Cancer-Detection</p>
            <p>&copy; Wong Zheng Jie 2025. All rights reserved.</p>
          </div>
        </footer>

    </div>
    <script>
        const fileInput = document.getElementById('actual-upload-img-btn');
        const fileNameDisplay = document.getElementById('img-file-name-display');

        fileInput.addEventListener('change', function() {
          if (this.files && this.files.length > 0) {
            fileNameDisplay.textContent = this.files[0].name;
          } else {
            fileNameDisplay.textContent = '';
          }
        });
    </script>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f7f7f7;
    margin: 0;
    padding: 0;
}

.container {
    margin: auto;
    background-color: #fff;
}

h1 {
    color: #333;
    text-align: left;
    margin: 0 0 0 2.5%;
}

.alert {
    background-color: #e74c3c;
    color: white;
    padding: 10px;
    text-align: center;
    margin: 15px 0;
    border-radius: 4px;
}

.upload-form {
    text-align: left;
    margin: 1% 0 0 0;
}

.upload-form input[type="file"] {
    margin: 10px;
}

.for-actual-upload-img-btn {
    padding: 8px;
    background-color: #a2d5c6;
    color: #333;
    border: none;
    cursor: pointer;
}

.upload-form button {
    padding: 8px;
    background-color: #a2d5c6;
```

```
    color: #000000;
    border: none;
    cursor: pointer;
    margin-left: 50px;
    font-size: 15.5px;
}

.for-actual-upload-img-btn:hover {
    background-color: #7fc5b6;
}

.upload-form button:hover {
    background-color: #7fc5b6;
}

.main-body-container {
    display: flex;
    gap: 10px;
}

.left-section {
    flex: 1;
    display: flex;
    justify-content: center;
    align-items: center;
}

.image-frame {
    border: 2px solid #ccc;
    width: 300px;
    height: 400px;
    display: flex;
    justify-content: center;
    align-items: center;
}

.uploaded-img {
    max-width: 100%;
    max-height: 100%;
}

.right-section {
    flex: 1;
    margin: 0 10% 0 0;
}

.progress-bar {
    position: relative;
    background-color: #eee;
    border-radius: 4px;
```

```
    margin: 10px 0;
    height: 25px;
}

.progress-bar .fill {
    background-color: #4db6ac;
    height: 100%;
    text-align: right;
    padding-right: 5px;
    line-height: 25px;
    color: #fff;
    font-weight: bold;
    border-radius: 4px 0 0 4px;
}

.progress-bar span {
    position: absolute;
    left: 10px;
    top: 0;
    line-height: 25px;
    color: #333;
    font-weight: bold;
}

.likely {
    margin: 20px 0 0 0;
    font-size: 28px;
    font-weight: bold;
    color: #333;
}

.disclaimer {
    font-size: 14px;
    color: #555;
}

.disclaimer h4 {
    margin: 20px 0 0 0;
    font-size: 16px;
}

footer {
    text-align: center;
    padding: 5px;
    background-color: #4db6ac;
    margin: 50px 0 0 0;
}
```

WZJ_Cancer_Predictor.bat

```
@echo off

REM This checks the Python version because the .keras model uses python 3.10.18
python --version
IF %ERRORLEVEL% NEQ 0 (
    echo Python not found. Please install Python 3.10.18 first.
    pause
    exit /b
)

REM python -m venv wzj_cancer_predictor_env
REM call wzj_env\Scripts\activate.bat

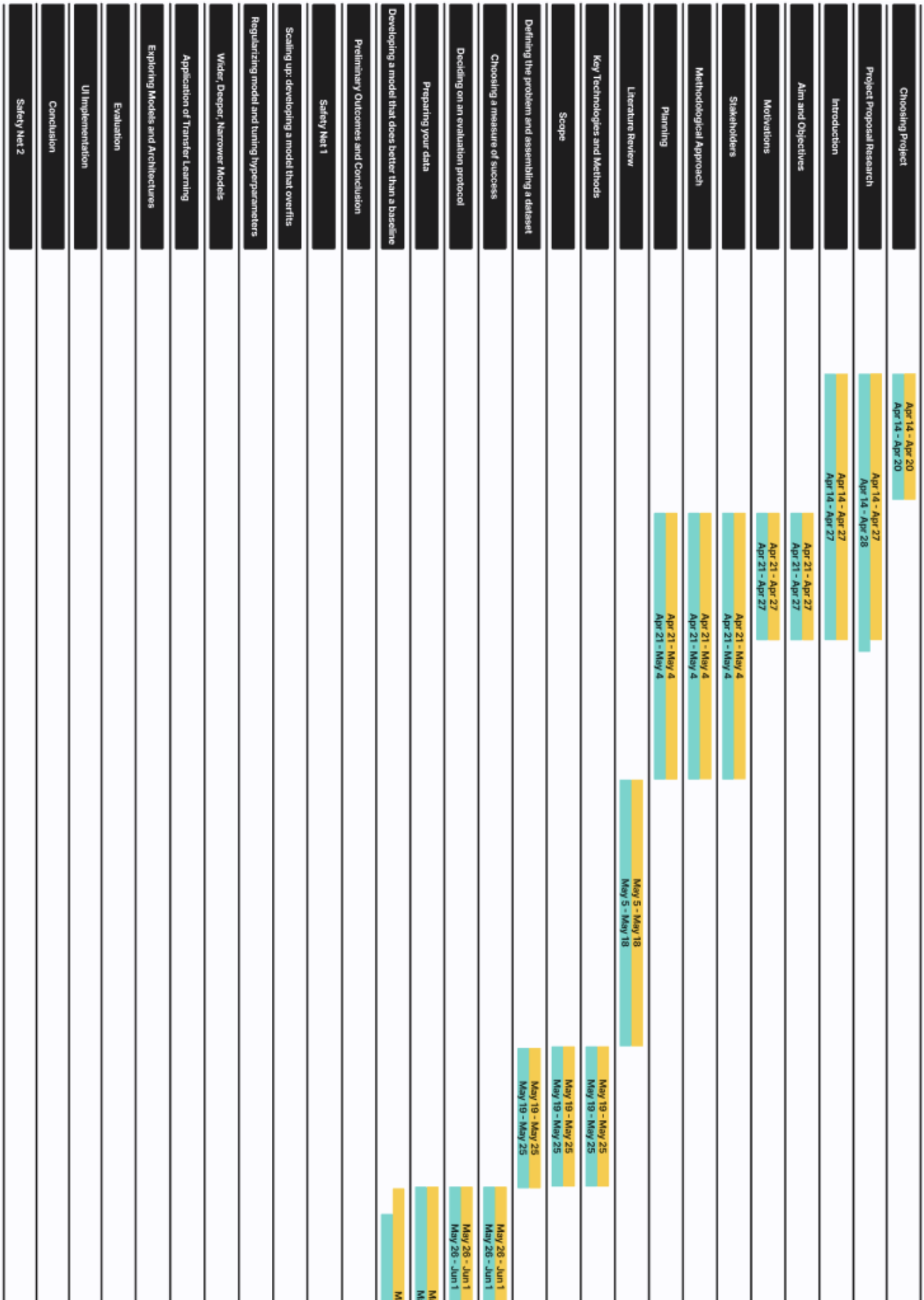
REM This section installs the required libraries for the .keras model to work, specifically tensorflow==2.19.0 keras==3.10.0
python -m pip install --upgrade pip
pip install flask tensorflow==2.19.0 keras==3.10.0 opencv-python-headless numpy

REM This starts the python script
python app.py

pause
```

19. Appendix H – Supplementary Materials

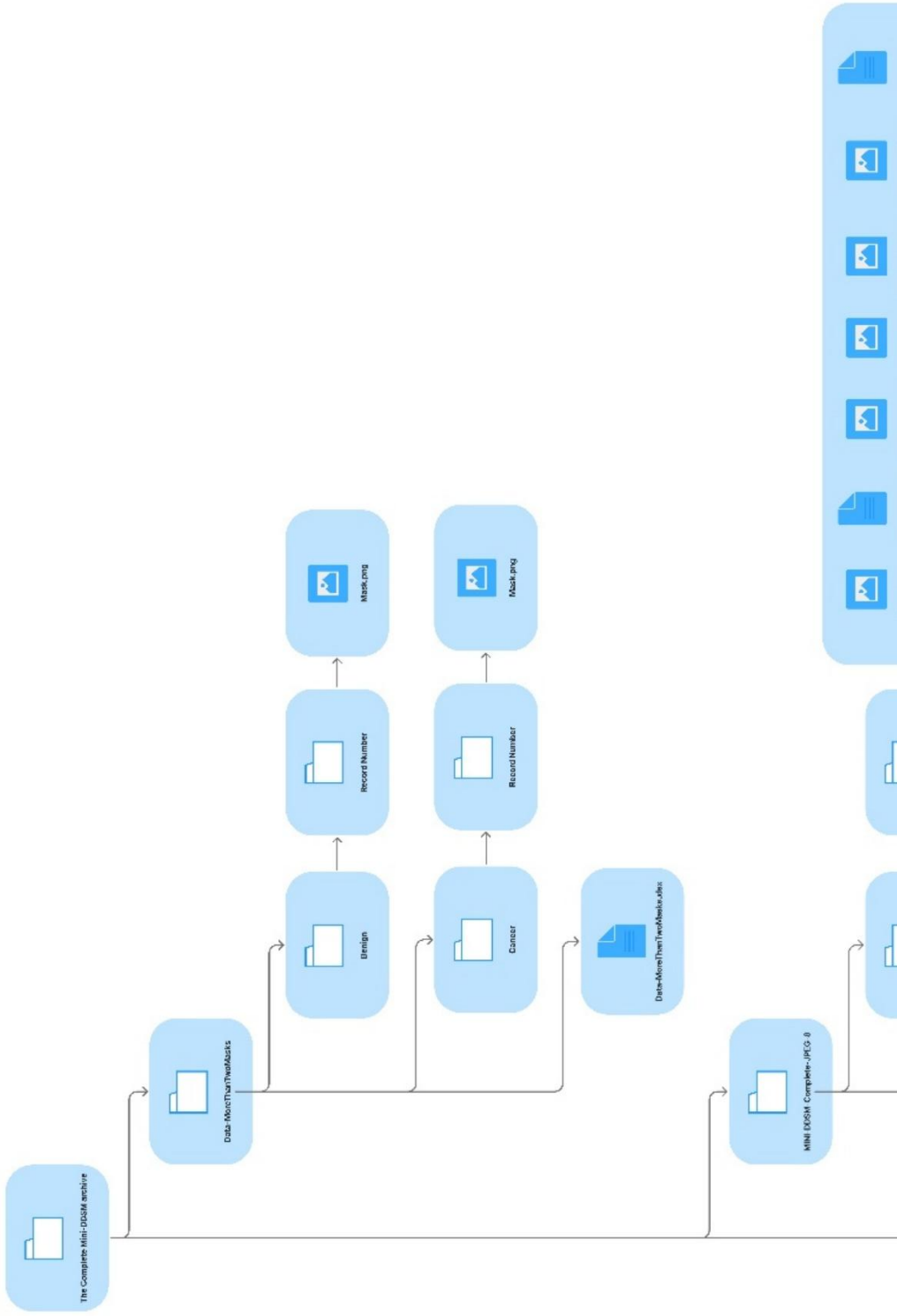
19.1 Supplementary Figures

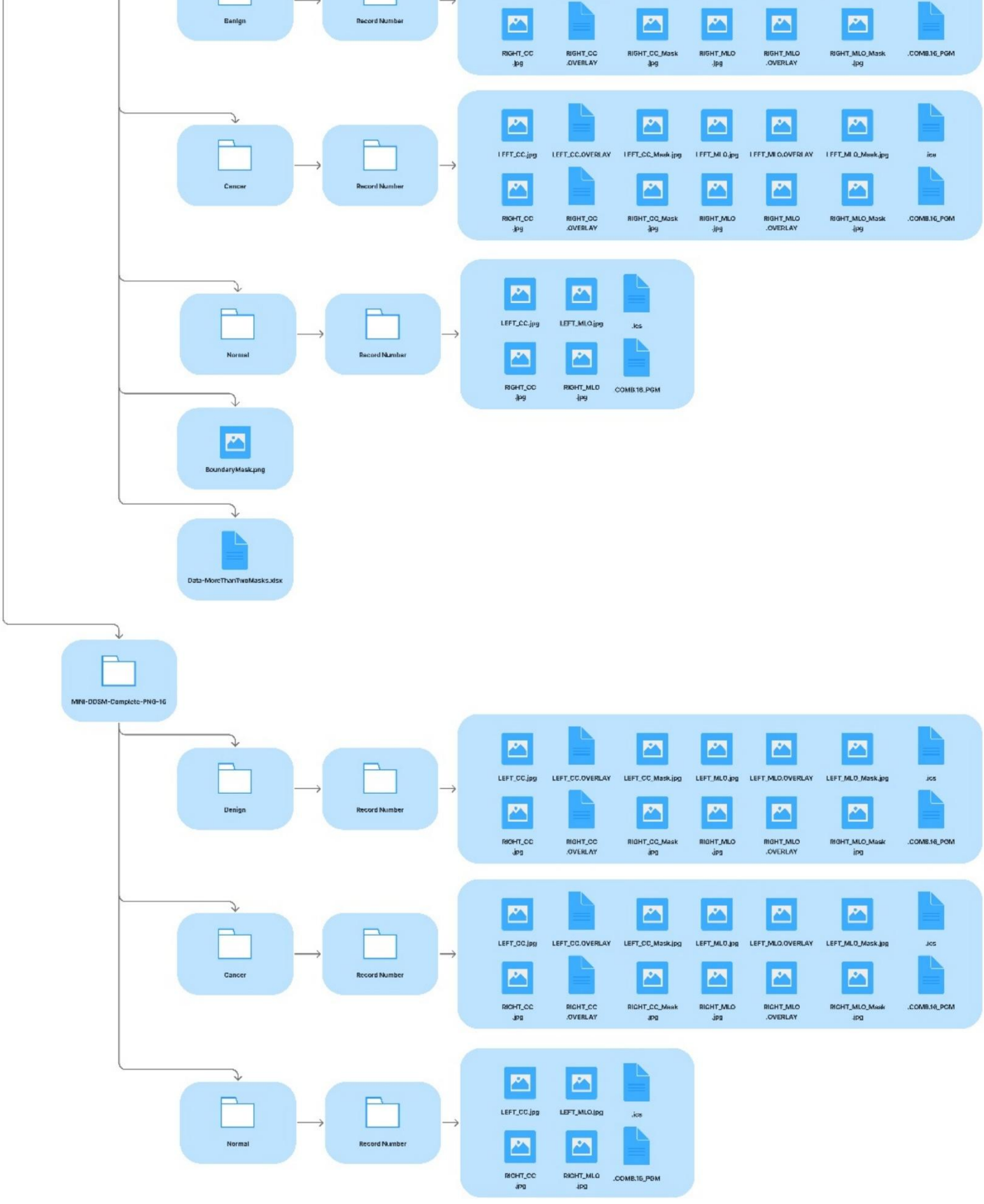



June
July
August
/02
06/03 - 06/09
06/10 - 06/16
06/17 - 06/23
06/24 - 06/30
07/01 - 07/07
07/08 - 07/14
07/15 - 07/21
07/22 - 07/28
07/29 - 08/04
08/05 - 08/11
08/12 - 08/18
Jun 9 - Jun 16
ay 26 - Jun 8
ay 26 - Jun 8
ay 26 - Jun 8
May 27 - Jun 8
Jun 2 - Jun 8
Jun 2 - Jun 8
Jun 9 - Jun 16
Jun 9 - Jun 16
Jun 17 - Jun 22
Jun 20 - Jul 25
Jun 23 - Jul 29
Jun 23 - Jul 29
Jun 30 - Jul 13
Jun 30 - Jul 13
Jul 14 - Jul 27
Jul 14 - Jul 27
Jul 28 - Aug 17
Jul 28 - Aug 17
Jul 24 - Aug 4
June
July
August
/02
06/03 - 06/09
06/10 - 06/16
06/17 - 06/23
06/24 - 06/30
07/01 - 07/07
07/08 - 07/14
07/15 - 07/21
07/22 - 07/28
07/29 - 08/04
08/05 - 08/11
08/12 - 08/18

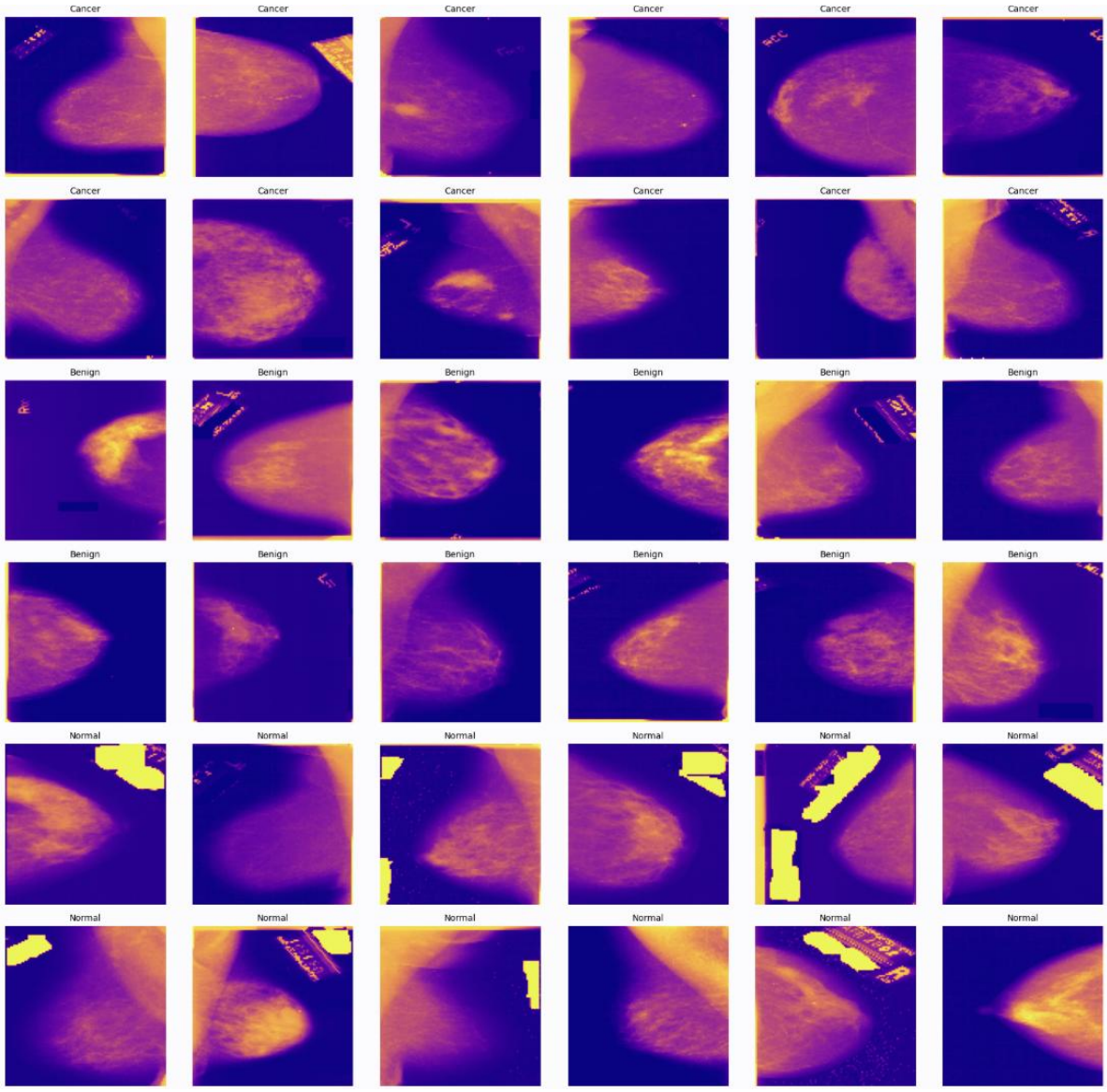


Appendix Figure 19.1.1 Gantt Chart Upscaled



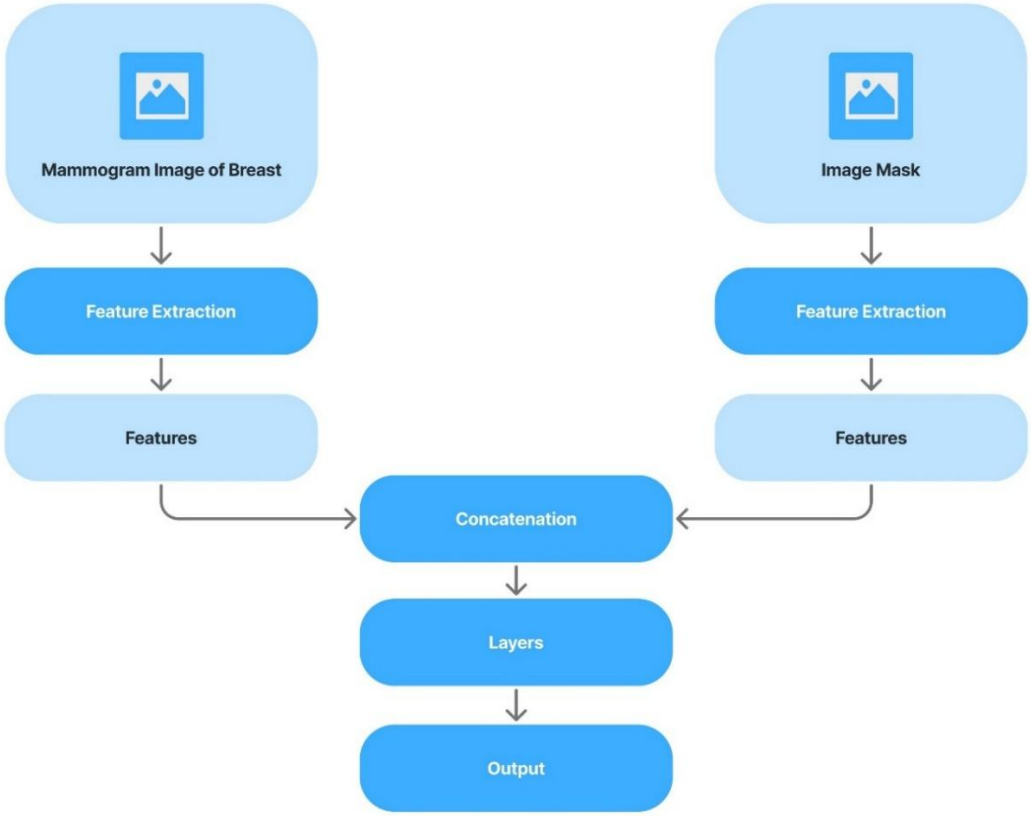


Appendix Figure 19.1.2 The Complete Mini-DDSM archive Folder Structure Enlarged



Appendix Figure 19.1.3 6 x 6 Grid Comparison of Randomly Selected Mammogram Images

The grid above shows randomly selected samples of mammogram images from the Cancer, Benign, and Non-Cancer classes. These samples illustrate the visual intricacy and variability within classes, especially in cancerous cases. (See Appendix Figure 21.1.3)



Appendix Figure 19.1.4 Preliminary Report Dual Input Hybrid CNN Rough Workflow Discussion Idea

19.2 Supplementary Tables

Evaluation Metric	Naive Baseline	Baseline
Accuracy	0.48	0.46
Macro Precision	0.47	0.45
Macro Recall	0.48	0.46
Macro F1-Score	0.45	0.45
AUC-ROC (macro)	0.62	0.58
Benign F1-Score	0.53	0.52
Cancer F1-Score	0.21	0.24
Normal F1-Score	0.63	0.58

Appendix Table 19.2.1 Preliminary Report Naïve Baseline and Baseline Table Comparison

Based on the table comparison, while cancer detection improved slightly for the baseline model, the overall scores dipped. Some of the possible reasons could be due to additional model complexity adding to overfitting to the training data or training time is not sufficient for the baseline model to outdo the naive baseline model. While the baseline model underperformed overall, the slight improvement in cancer class sensitivity is considered a crucial aspect in clinical detection. (See Appendix Table 21.2.1)

20. Appendix References

[85] Wikipedia (2024) Kanban (development). Last edited: 30 May 2024. Available at: https://en.wikipedia.org/wiki/Kanban_%28development%29 (Accessed: 2025).

[86] gov.uk (2025) Your guide to NHS breast screening. Last updated: 14 May 2025. Available at: https://www.gov.uk/government/publications/breast-screening-helping-women-decide/nhs-breast-screening-helping-you-decide (Accessed: 2025).

[87] Wang, L. (2017) 'Early Diagnosis of Breast Cancer', Sensors, 17, 1572. doi: https://doi.org/10.3390/s17071572 (Accessed: 2025).

[88] Gatting, L., Ahmed, S., Meccheri, P., Newlands, R., Kehagia, A.A. and Waller, J. (2024) 'Acceptability of artificial intelligence in breast screening: focus groups with the screening-eligible population in England', BMJ Public Health, 2, e000892. doi: 10.1136/bmjph-2024-000892. [Online] Available at: https://doi.org/10.1136/bmjph-2024-000892 (Accessed: 2025).

[89] NHS England (2025) Breast screening (mammogram). [Online] Available at: https://www.nhs.uk/conditions/breast-screening-mammogram/ (Accessed: 2025).

[90] Pesapane, F., Rotili, A., Valconi, E. et al. (2023) 'Women's perceptions and attitudes to the use of AI in breast cancer screening: a survey in a cancer referral centre', British Journal of Radiology. doi: https://doi.org/10.1259/bjr.20220569 (Accessed: 2025).

[91] Shih, Y.-C.T., Sabik, L.M., Stout, N.K., Halpern, M.T., Lipscomb, J., Ramsey, S. and Ritzwoller, D.P. (2022) 'Health economics research in cancer screening: research opportunities, challenges, and future directions', JNCI Monographs, pp. 42–50. doi: https://doi.org/10.1093/jncimonographs/lgac008 (Accessed: 2025).

[92] Hill, H., Roadevin, C., Duffy, S., Mandrik, O. and Brentnall, A. (2024) 'Cost-effectiveness of AI for risk-stratified breast cancer screening', JAMA Network Open, 7(9), e2431715. doi: https://doi.org/10.1001/jamanetworkopen.2024.31715. Available at: https://jamanetwork.com/journals/jamanetworkopen/fullarticle/2823253 (Accessed: 2025).

[93] Brentnall, A.R., Atakpa, E.C., Hill, H. et al. (2023) 'An optimization framework to guide the choice of thresholds for risk-based cancer screening', npj Digital Medicine, 6, 223. doi: https://doi.org/10.1038/s41746-023-00967-9 (Accessed: 2025).

[94] Alesha, W., Okolie, C., Everitt, J., Hookway, A., Shaw, H., Little, K., Lewis, R., Cooper, A. and Edwards, A. (2024) 'The effectiveness and cost-effectiveness of

community diagnostic centres: a rapid review', International Journal of Public Health, 69. doi: https://doi.org/10.3389/ijph.2024.1606243. Available at: https://www.ssph-journal.org/journals/international-journal-of-public-health/articles/10.3389/ijph.2024.1606243/full (Accessed: 2025).

[95] Sharafaddini, A.M., Esfahani, K.K. and Mansouri, N. (2024) 'Deep learning approaches to detect breast cancer: a comprehensive review', Multimedia Tools and Applications. doi: https://doi.org/10.1007/s11042-024-20011-6. Available at: https://link.springer.com/article/10.1007/s11042-024-20011-6 (Accessed: 2025).

[96] Elahi, R. and Nazari, M. (2024) 'An updated overview of radiomics-based artificial intelligence (AI) methods in breast cancer screening and diagnosis', Radiological Physics and Technology, 17(4), pp. 795–818. Available at: https://arxiv.org/abs/2406.14735 (Accessed: 2025).

[97] Thakur, N., Kumar, P. and Kumar, A. (2024) 'A systematic review of machine and deep learning techniques for the identification and classification of breast cancer through medical image modalities', Multimedia Tools and Applications, 83(12), pp. 35849–35942. doi: https://doi.org/10.1007/s11042-023-16634-w. Available at: https://www.proquest.com/scholarly-journals/systematic-review-machine-deep-learning/docview/3030965077/se-2 (Accessed: 2025).

[98] Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. Cambridge, MA: MIT Press. Available at: https://www.deeplearningbook.org (Accessed: 2025).

[99] Stanford University (2025) CS231n: Convolutional Neural Networks for Visual Recognition. [Online] Available at: https://cs231n.github.io/convolutional-networks/ (Accessed: 2025).

[100] Wu, J. (2017) 'Introduction to convolutional neural networks', National Key Lab for Novel Software Technology, Nanjing University, China, 5.23, p. 495. Available at: https://project.inria.fr/quidiasante/files/2021/06/CNN.pdf (Accessed: 2025).

[101] Godoy, D. (2025) dl-visuals: Convolutions. GitHub. Available at: https://github.com/dvgodoy/dl-visuals/tree/main/Convolutions (Accessed: 2025).

[102] Gholamalinejad, H. and Khosravi, H. (2020) 'Pooling methods in deep neural networks: a review', arXiv preprint arXiv:2009.07485. doi: https://doi.org/10.48550/arXiv.2009.07485 (Accessed: 2025).

[103] TensorFlow (n.d.). TensorFlow. Available at: https://www.tensorflow.org/ (Accessed: 2025).

[104] Keras Project (n.d.). Keras: Deep learning for humans. Available at: https://keras.io/ (Accessed: 2025).

[105] Jayathilaka, C. (2020). Agile Methodology. Medium, 30 July. Available at: https://medium.com/@chathmini96/agile-methodology-30ec4cdf3fc (Accessed: 2025).